

Sistemas Operacionais

Introdução

O que é um Sistema Operacional?

- Um software que abstrai as complexidades do hardware de um usuário/programador
- SO visto como uma **Máquina Virtual** (Extendida)
 - Detalhes de hardware tais como velocidade de gravação e posicionamento de cabeça de gravação de um HD tornam-se completamente transparente ao usuário
- SO visto como um **Gerenciador de Recursos**
 - Provê acesso ordenado aos elementos de hardware
 - Monitora quem está usando qual recurso a fim de aceitar solicitações de recursos, contabilizar estatísticas de uso, e mediar solicitações conflitantes de programadores e usuários distintos

Sistema de Computador em Camadas?

- O SO representa a interface entre programas aplicativos bem como utilitários e o hardware
- Os utilitários normalmente vêm com o SO para facilitar o desenvolvimento de programas
- Tarefas como gerenciamento de arquivos, controle de I/O e criação de aplicativos fazem parte dos utilitários

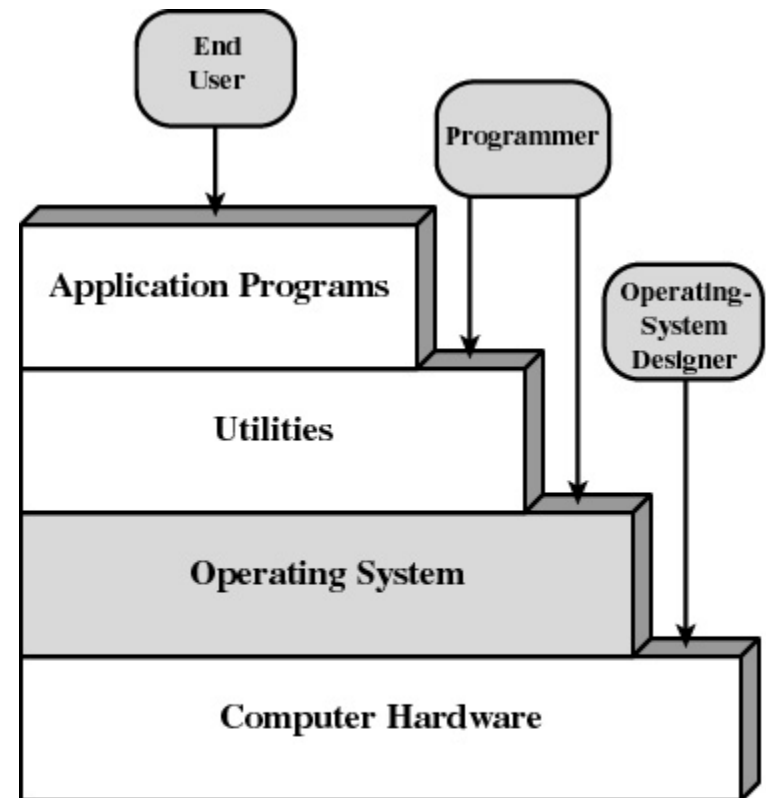


Figure 2.1 Layers and Views of a Computer System

Evolução dos Sistemas Operacionais

■ Anos 40

- Programador é também operador
- Programação é feita inteiramente em linguagem de máquina por intermédio de painéis de ligação (*plugboards*)
- Painéis de interligação evoluem para cartões perfurados
- Um programa (job) em execução ocupa toda a máquina

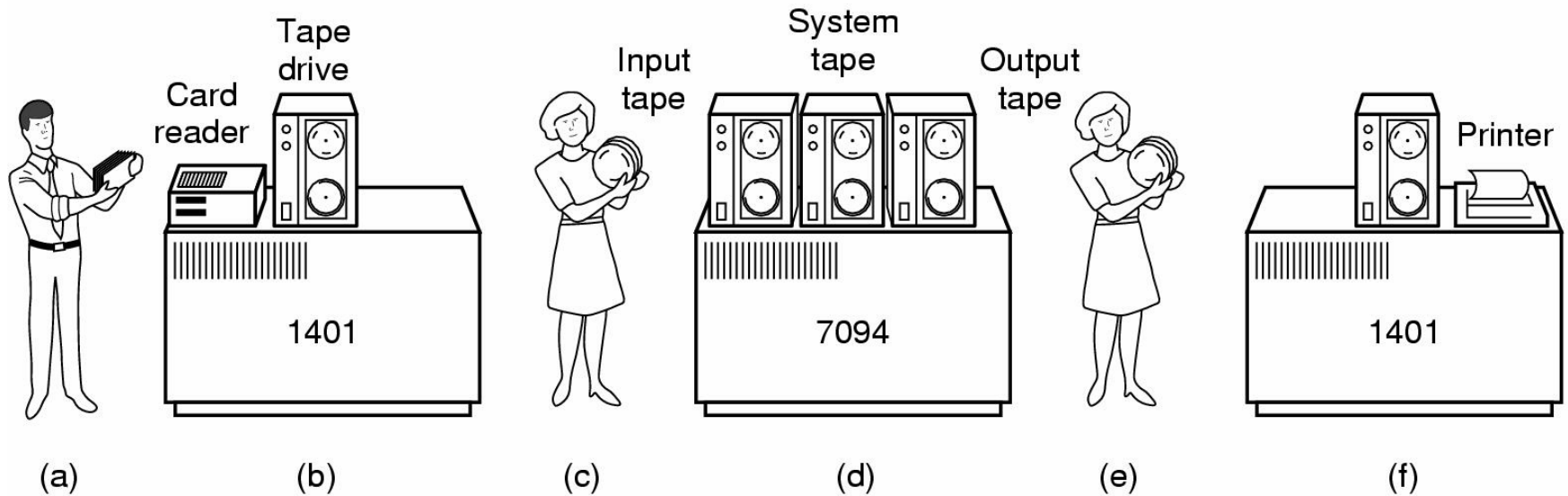
Evolução dos Sistemas Operacionais

■ Anos 50

- O transistor possibilita a criação de computadores mais confiáveis
- Programador não precisa mais operar a máquina
- Introdução do conceito de processamento de lote (**batch**) com o programa **monitor**
- Processamento de **lote** reduz o tempo entre **jobs** seqüenciais

Evolução dos Sistemas Operacionais

■ Anos 50 → Sistema Batch



Evolução dos Sistemas Operacionais

■ Anos 60

- Multiprogramação para reduzir o tempo em que o processador fica ocioso
- Memória é dividida em partições para que cada **job** fique em uma partição
- O trabalho de manuseio com fitas magnéticas é reduzido drasticamente (***spooling***)

Partições da memória

Job 1
Job 2
Job 3
Sistema Operacional

Evolução dos Sistemas Operacionais

■ Anos 70

- *Timesharing* → usuários compartilham um processador de forma transparente por meio de terminais
- Interactividade é conseguida de forma eficiente

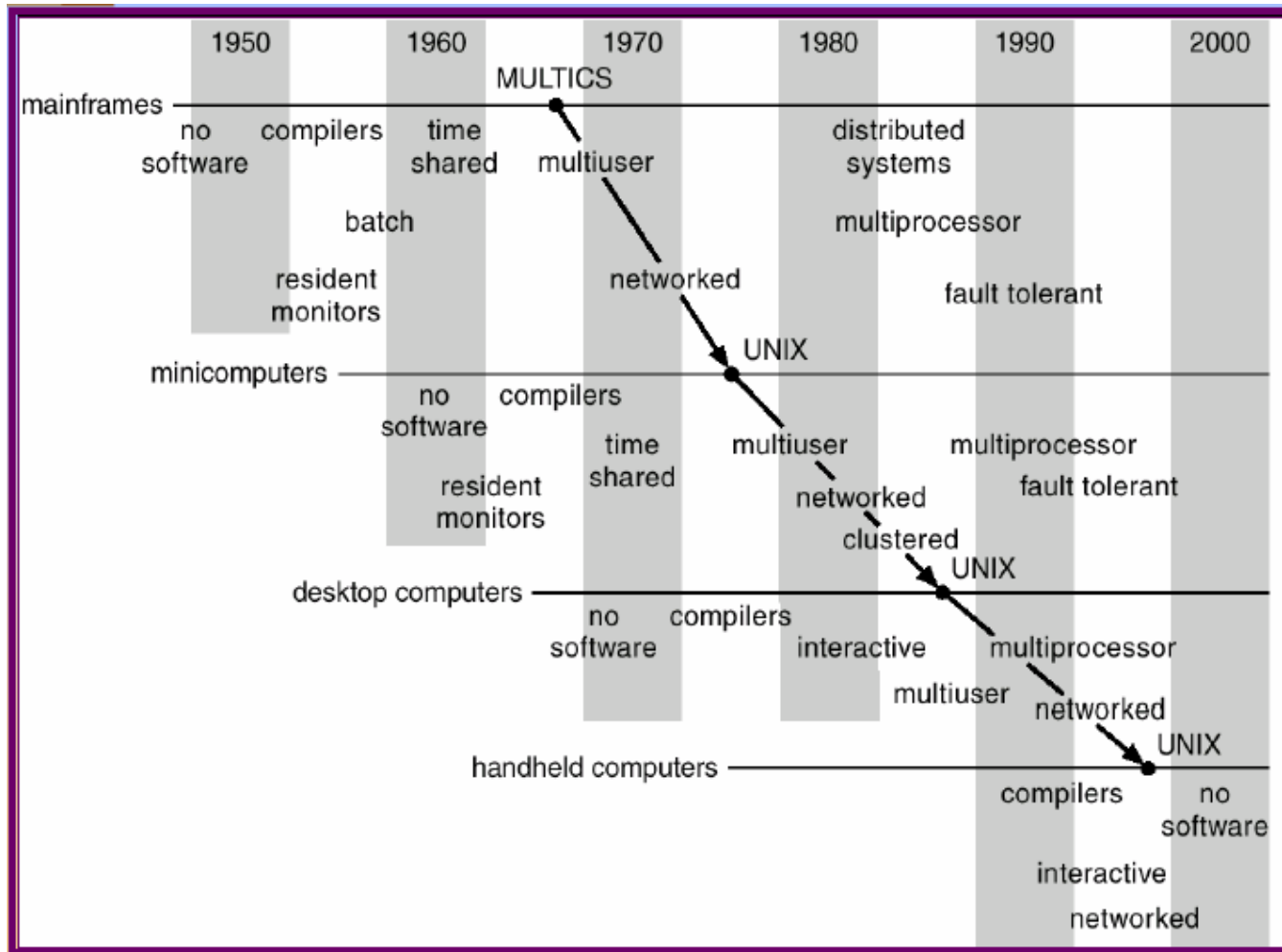
■ Anos 80

- Além de eficiência, conveniência para o usuário é procurada

■ Desde os anos 90

- Computadores PC tornam-se populares
- Sistemas Operacionais que controlam banco de dados
- Sistemas distribuídos
- Sistemas Operacionais de tempo real
- Paralelismo

Evolução dos Sistemas Operacionais (cont.)

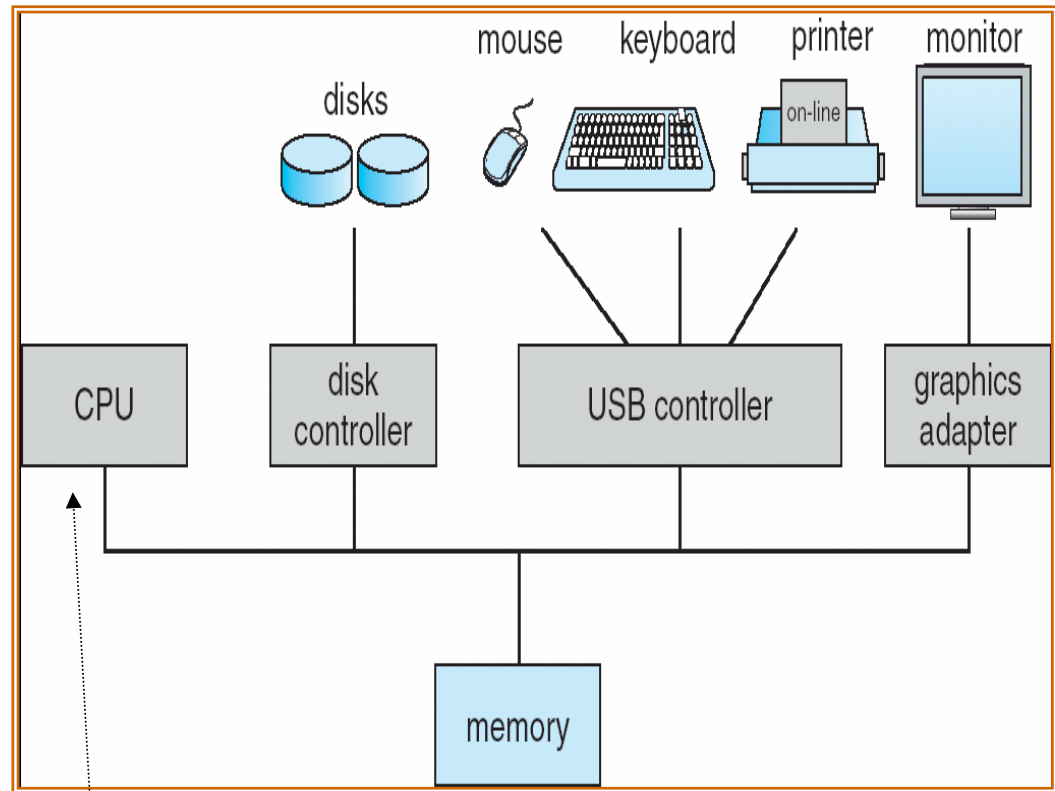


Tipos de Sistemas Operacionais

- **Main frames**
 - Jobs em batch, transações, multiprogramação
- **Servidores**
 - Usuários conectados remotamente por redes
- **Ambientes multiprocessados**
 - Extensão de SO em servidores (programação paralela)
- **Computadores pessoais**
 - Prioriza interface com o usuário
- **Aplicações em tempo real**
 - Confiabilidade de resposta previsível é priorizada
- **Sistemas embutidos**
 - Limitados em memória, potência. Ex.: palmtop, celular, etc.
- **Smart cards**
 - Muito limitados, mas alguns podem rodar até JAVA !

Revisão de Hardware

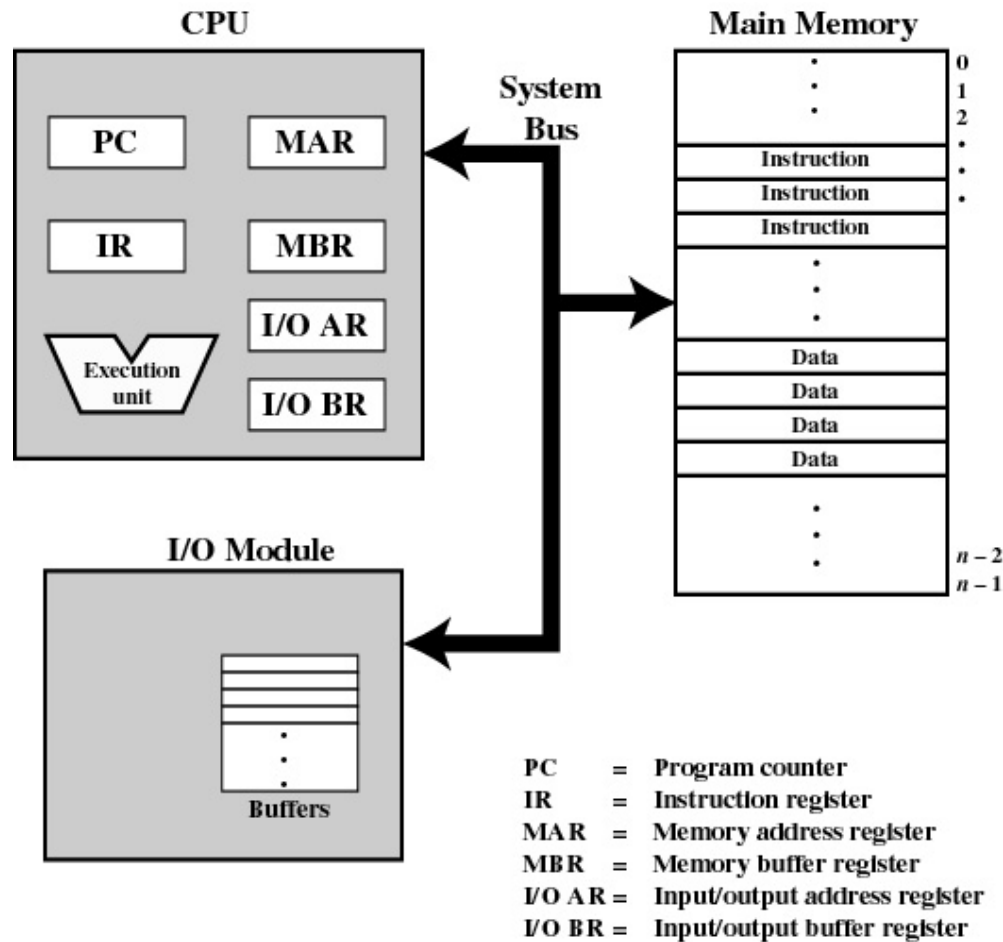
- **Todo hardware é composto de:**
 - Processador (CPU)
 - Memória
 - Periféricos (I/O)
 - Barramento (bus)
- Cada periférico tem seu controlador
- A comunicação entre todos os elementos se dá pelo barramento



Coração do computador !

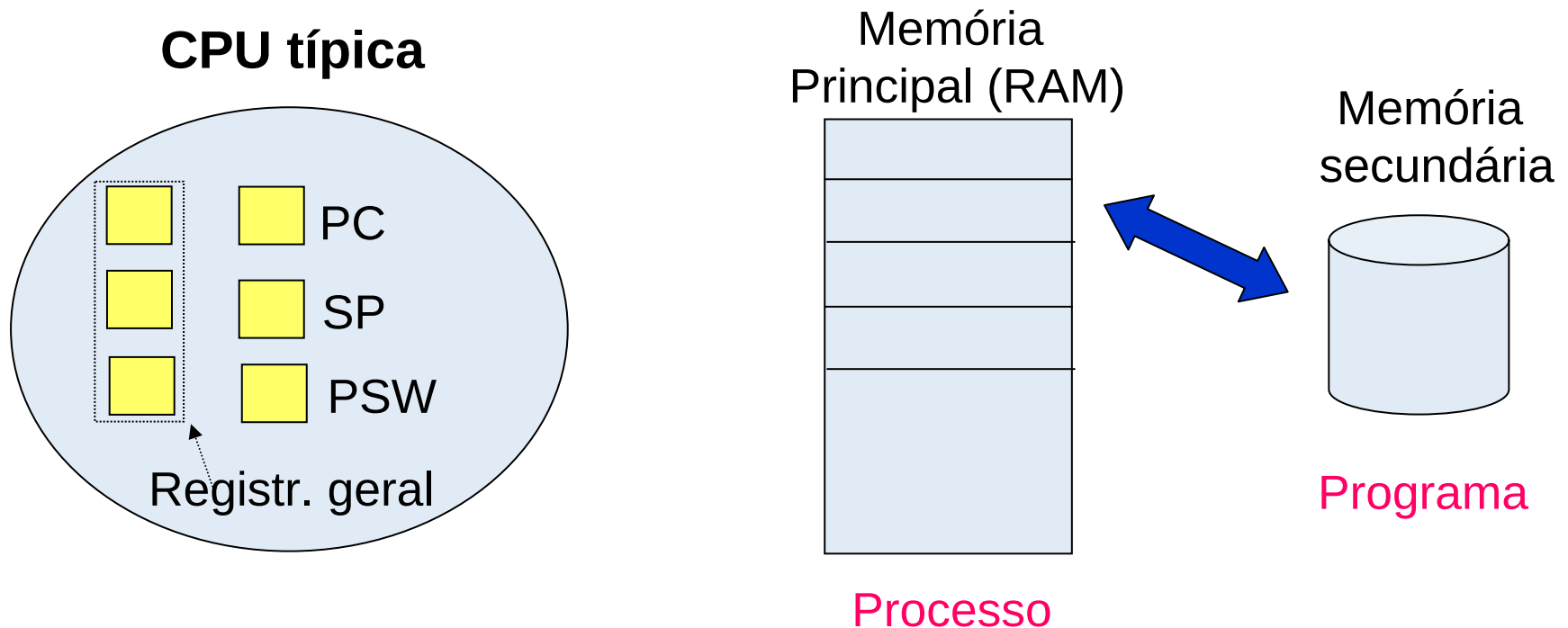
Revisão de Hardware (cont.)

- elementos básicos de um sistema de computador



Revisão de Hardware: Processos

- Processos são programas em execução
- Cada CPU tem seu conjunto de instruções próprias
- CPU possui: registradores de uso geral, registrador PC, registrador de pilha (SP), e registrador PSW



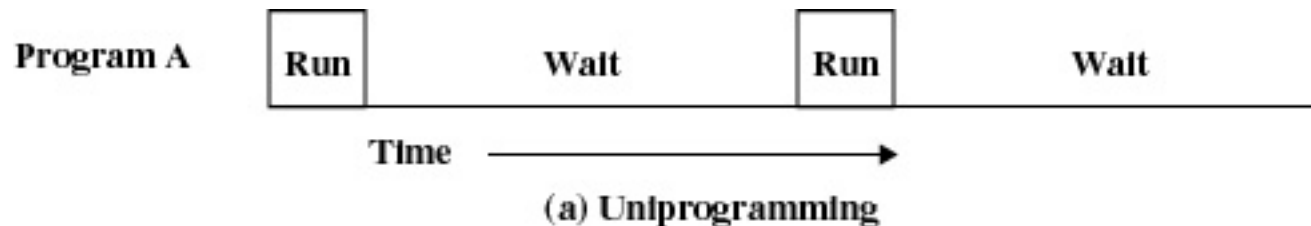
Revisão de Hardware: Processos (cont.)

- Processos podem rodar em dois modos:
 - Kernel
 - Usuário
- Um bit no registrador PSW controla o modo do processo
- Para obter acesso aos periféricos o processo precisa chavear para modo kernel
- Esse chaveamento é feito pelas chamadas de sistema (*systems calls*)
- Existem várias *systems calls* oferecidas pelo SO
- Depois de executado uma *systems calls*, o processo retorna ao modo usuário

Revisão de Hardware: Processos (cont.)

Systema monoprogramável

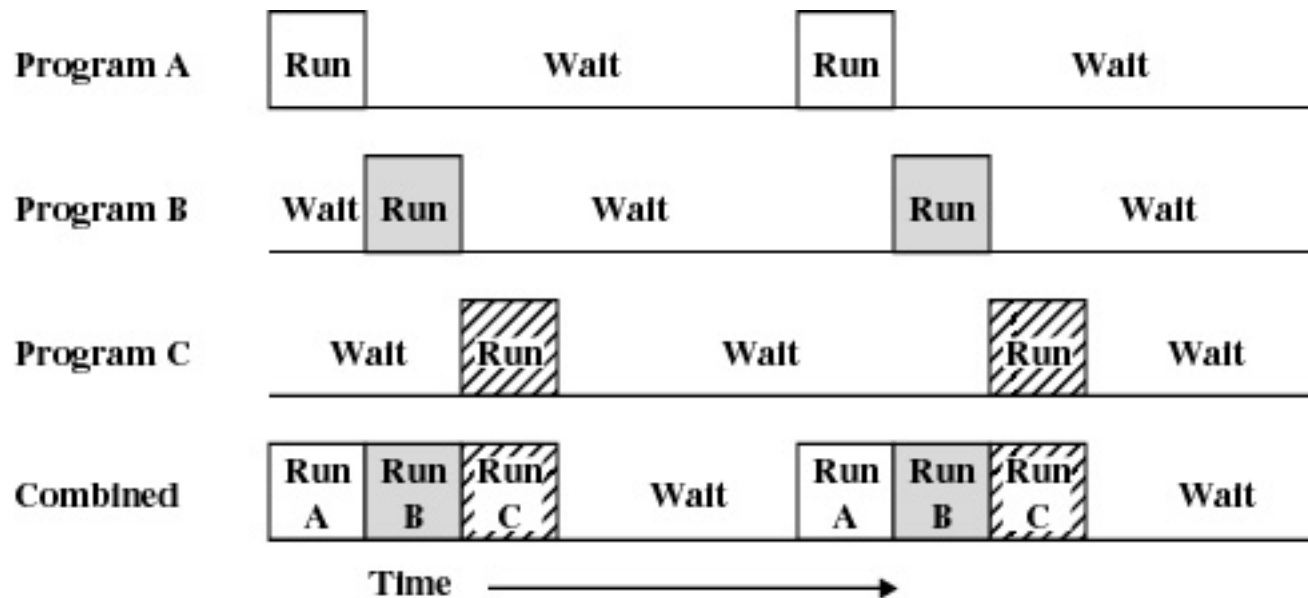
- Processo precisa esperar pelo fim da operação de I/O para prosseguir !!!



Revisão de Hardware: Processos (cont.)

Systema multiprogramável

- Quando um processo precisa esperar pelo fim da operação de I/O, o processador pode rodar outro processo !!!



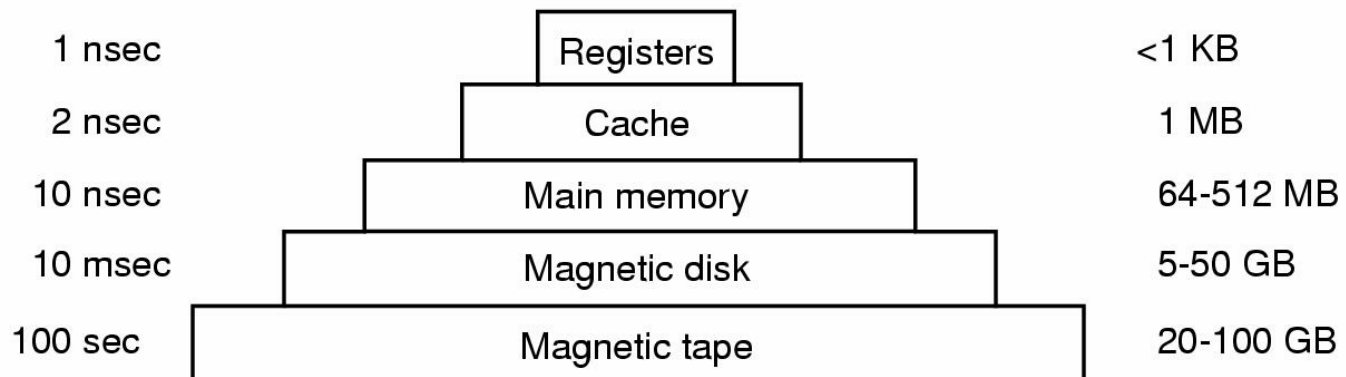
(c) Multiprogramming with three programs

Revisão de Hardware: memória

- As memórias atuais não satisfazem todos os requisitos ao mesmo tempo, que são:
 - Extremamente rápidas
 - Muito espaçosas
 - De baixíssimo custo
- Combinação de várias tecnologias
- Velocidades de acesso são inversamente proporcionais à capacidade de armazenamento

Typical access time

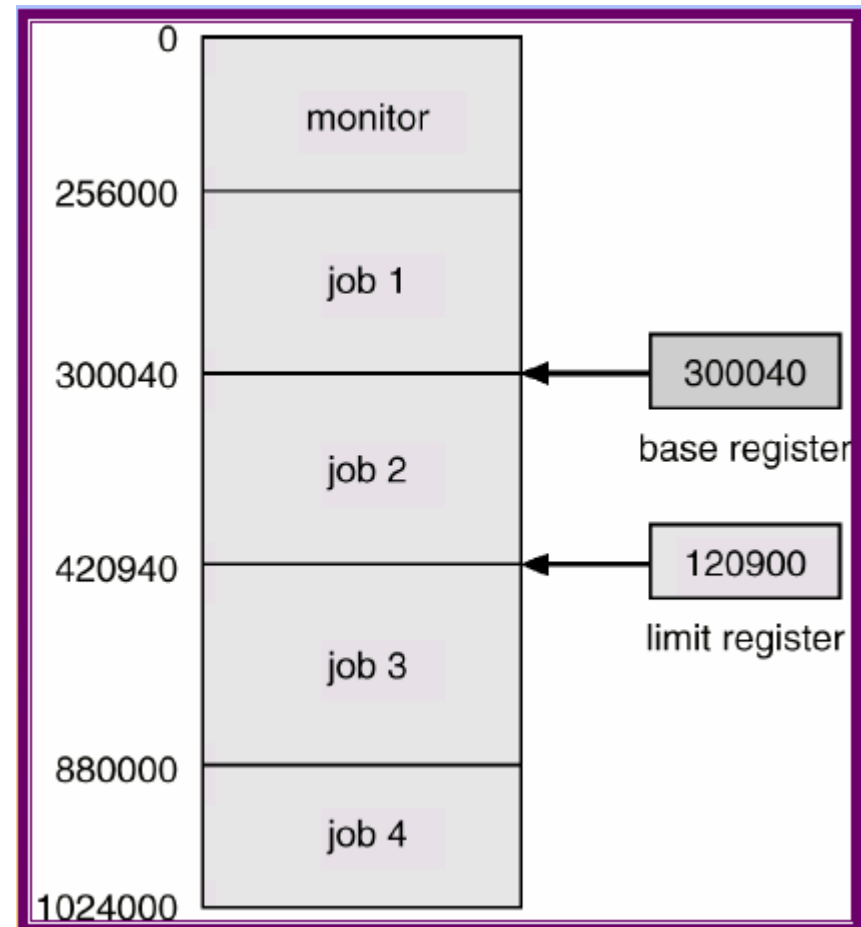
Typical capacity



Revisão de Hardware: memória (cont.)

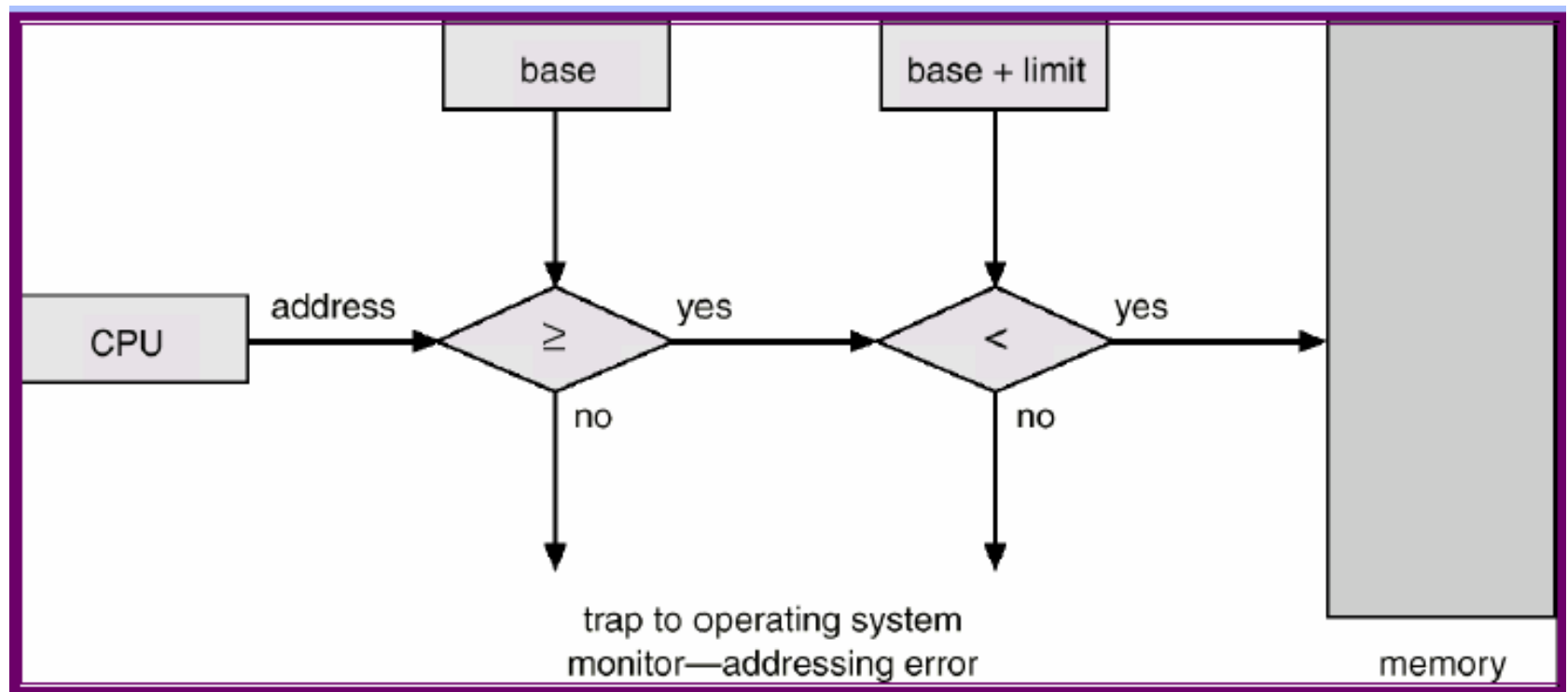
- Proteção de região de memória é fundamental em sistema multiprogramados
- Registrador base
 - Define o endereço inicial da região alocada ao proc.
- Registrador limite
 - Define o tamanho da região desde sua base
- Violação de proteção gera uma chamada ao SO

A MMU (Memory Management Unit) é quem gerência a memória



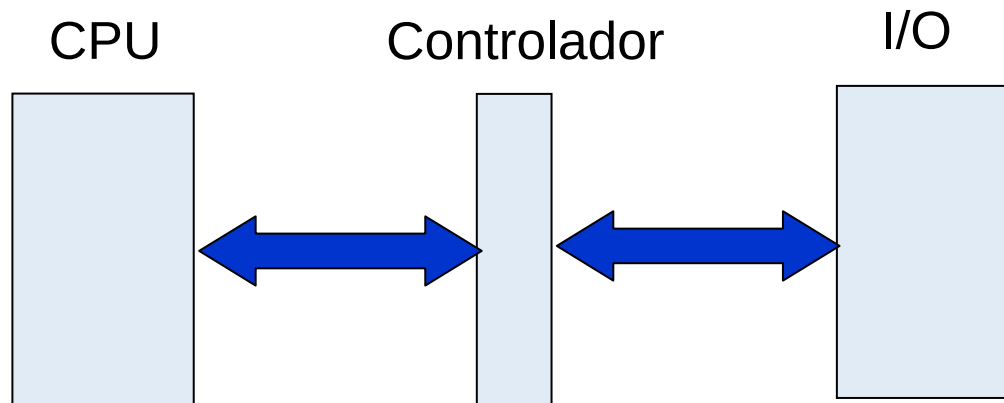
Revisão de Hardware: memória (cont.)

Se endereço vindo da CPU extrapola os limites, a MMU gera uma interrupção (trap) ao SO



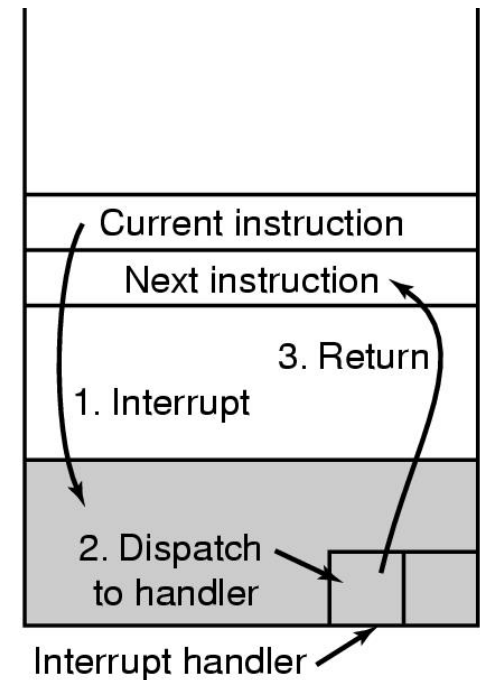
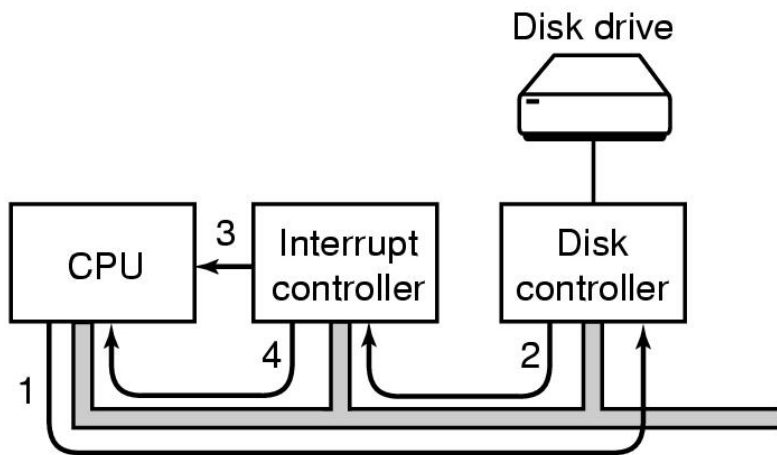
Revisão de Hardware: I/O

- Detalhes de complicações, tais como ler um arquivo em disco, ficam “escondidos” dos processos em modo usuário
- Interface padrão é usada
- Drivers são necessários adaptar mecanismos ao SO
- Drivers executam em modo kernel (são inseridos no kernel)
- Normalmente há instruções especiais IN e OUT para comunicação com os I/Os



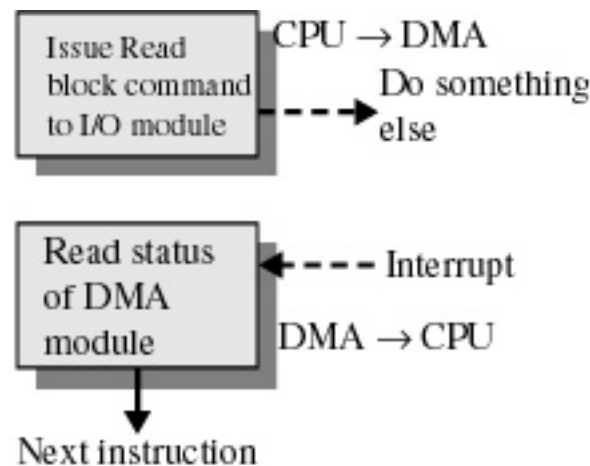
Revisão de Hardware: I/O (cont.)

- **Interrupções** são usadas para uma eficiente comunicação entre periféricos e a CPU
- CPU solicita serviço do I/O e continua com outras atividades. Uma sinalização é gerada pelo I/O depois
- Sob interrupção → PC e PSW são guardados na pilha



Revisão de Hardware: I/O (cont.)

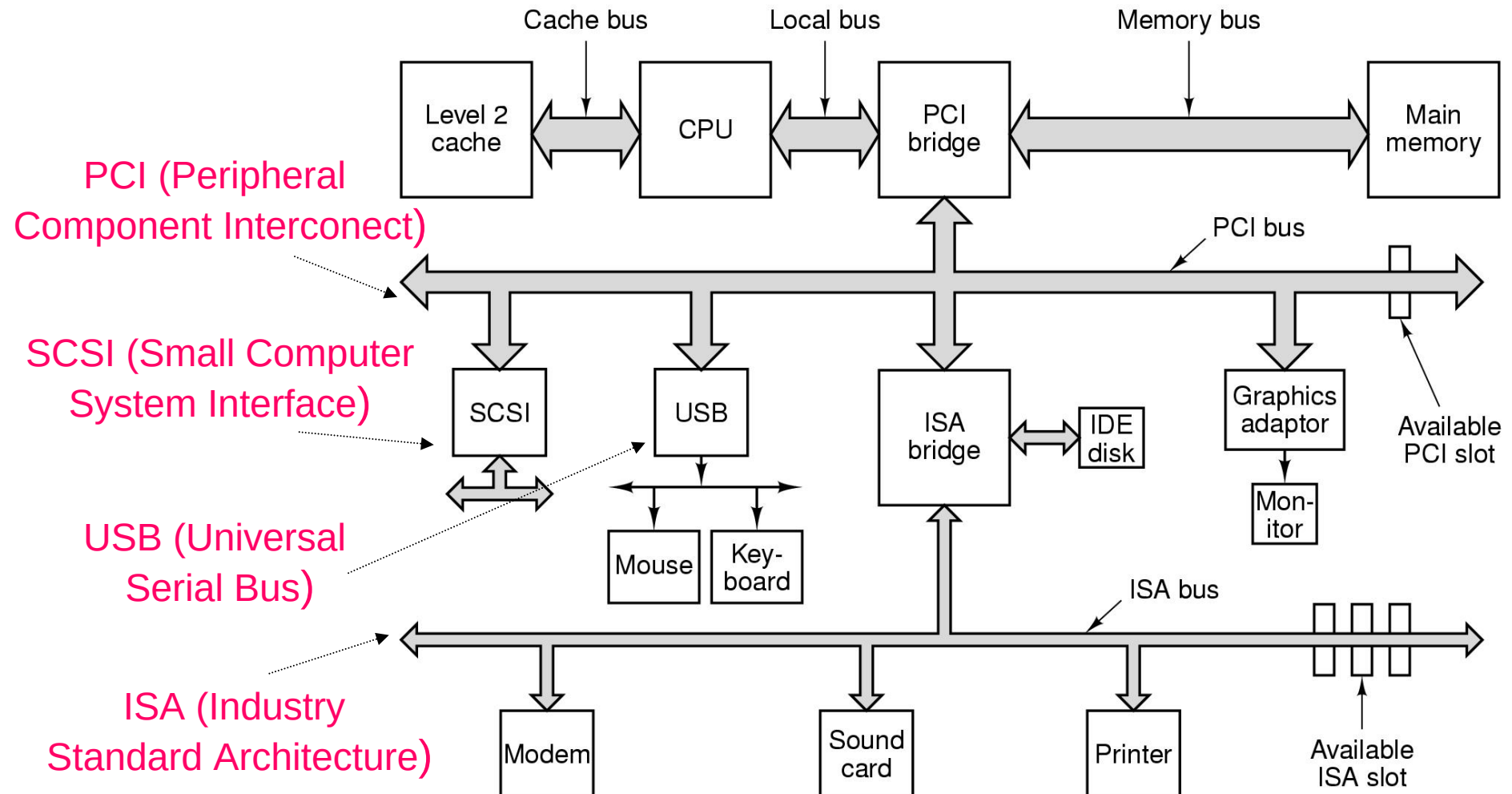
- **DMA (Direct Memory Access)** é usada para se prover acesso direto à memória sem a necessidade de intervenção da CPU (só no início)
- O controlador de disco recebe autorização da CPU para proceder com a leitura/gravação na memória diretamente
- CPU fica livre para realizar outras tarefas



(c) Direct memory access

Revisão de Hardware: Barramentos (Buses)

- Sistemas atuais possui vários barramentos para troca de informação entre os seus elementos (Ex.: **Pentium**)



System Calls (chamadas de sistema)

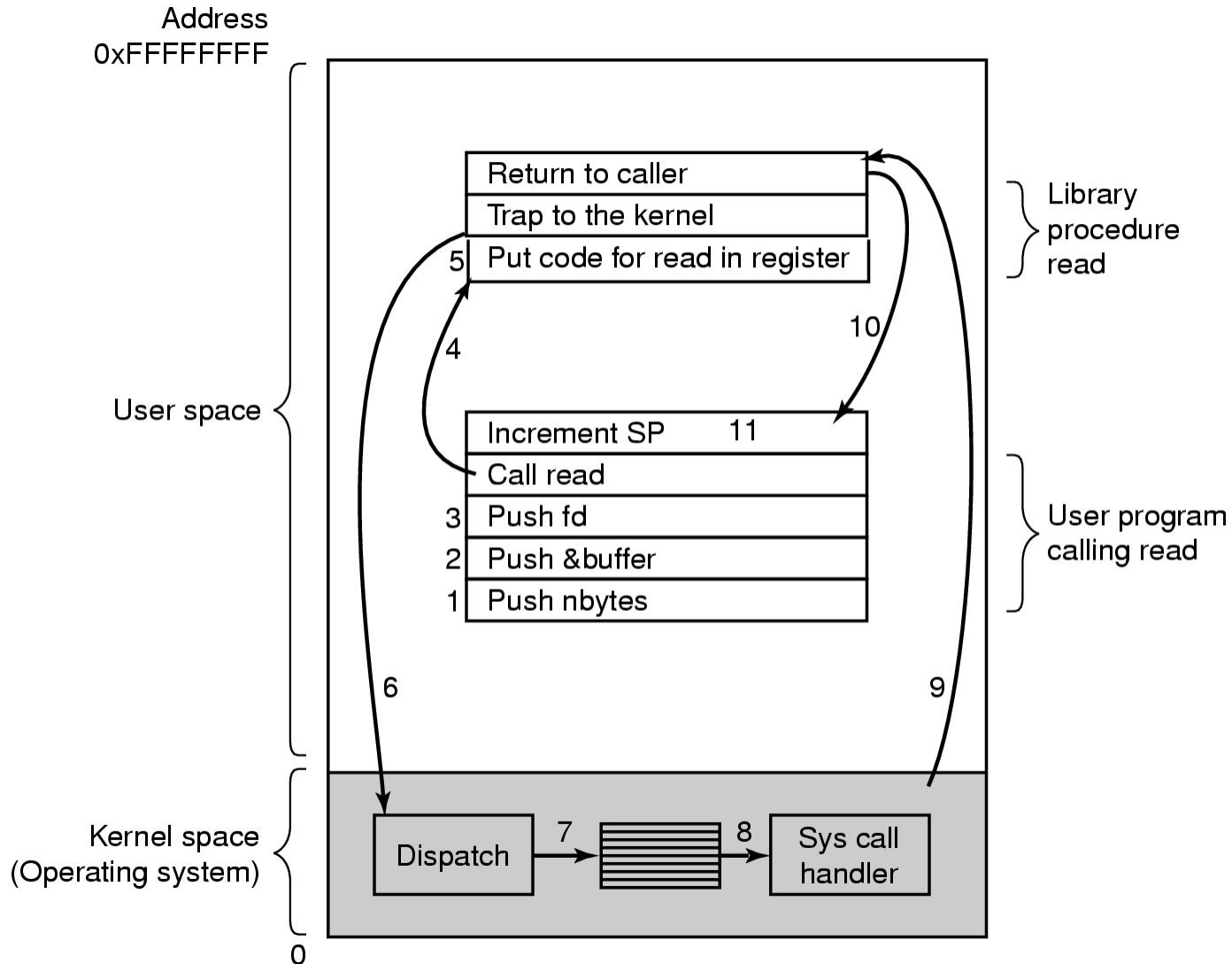
- Representam a interface entre o sistema operacional e os programas de usuários
- O SO provê várias *Systems Calls* (rodam em mod kernel) principalmente para tarefas associadas a periféricos
- Exemplo de uma *Systems Call* para leitura de arquivo
`read (fd, buffer, nbytes)`

fd=descrição do arquivo,

buffer=local para onde o dado lido deve ser armazenado,
nbytes=quantidade de bytes a serem lidos

Onze passos são necessários para a realização desta *system call*, como mostrado a seguir

System Calls (chamadas de sistema) (cont.)



System Calls para gerenciamento de processos

System Calls do UNIX

Process management

Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

System Calls para gerenciamento de arquivos

File management

Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

System Calls para gerenciamento de diretórios

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Outras System Calls

Miscellaneous

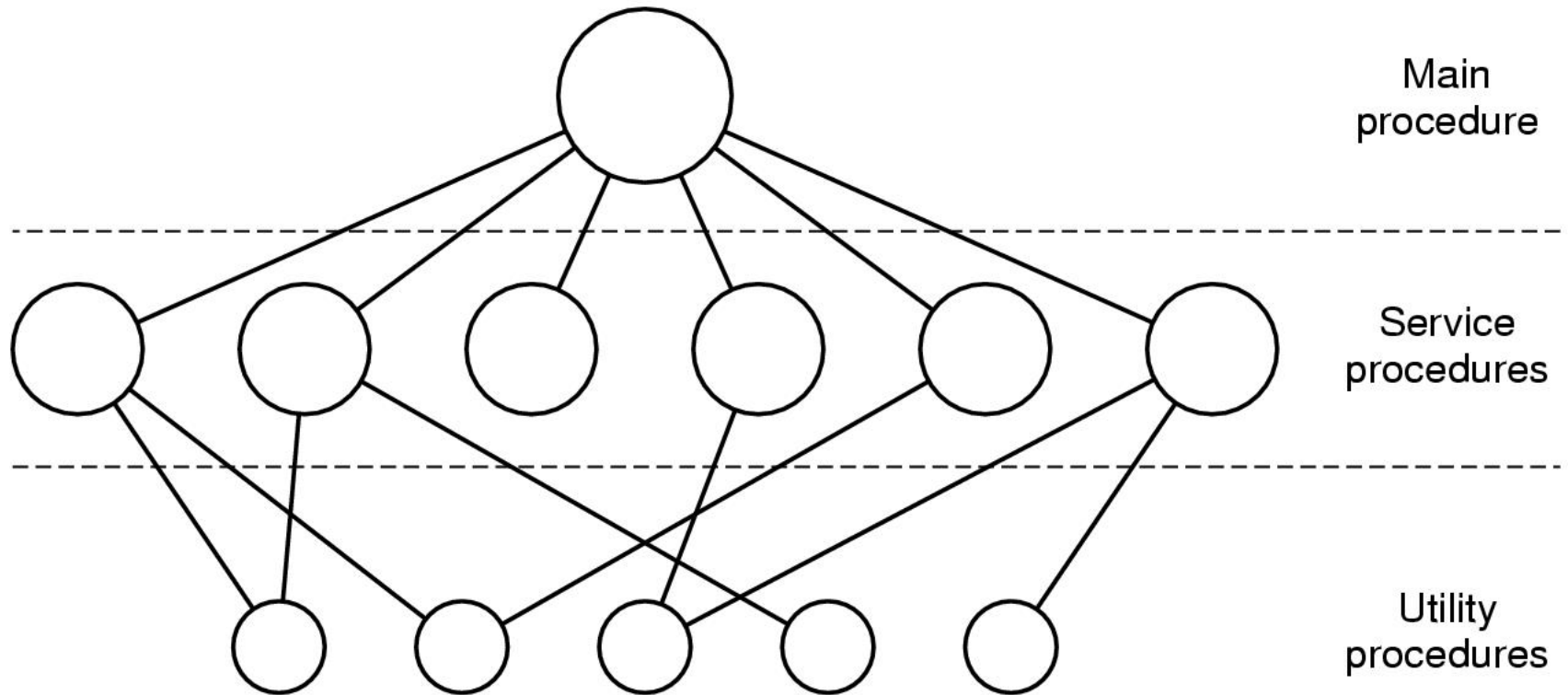
Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

System Calls do Windows

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Estruturas do Sistema Operacional

- **Monolítico:** desestruturado porque os processos se comunicam diretamente entre si



Estruturas do Sistema Operacional (cont.)

- **Em camadas:**

- cada camada desenvolve uma atividade específica
- Há uma interface padrão para comunicação entre as camadas adjacentes

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004

Sistemas Operacionais

Processos e Threads

Conteúdo

- Processos
- Threads
- InterProcess Communication
- Algoritmos de escalonamento

PROCESSOS

- Processo é nome que se dá a um programa em execução
- Um processo controla os valores atuais dos registradores da CPU, bem como variáveis em memória
- Cada processo possui sua própria “CPU virtual”
- Em sistemas multiprogramáveis, os processos dividem a atenção da CPU
- Processos são comutados (chaveados) a fim de criar a ilusão de que cada usuário tem a CPU exclusivamente para si

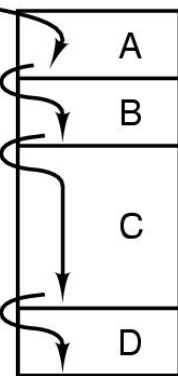
Processos (cont.)

- Exemplo de 4 processos competindo pela CPU
- Somente um processo por vez, pois só há uma CPU e assim só um registrador PC real

Cada processo possui seu próprio
“registrador PC lógico”

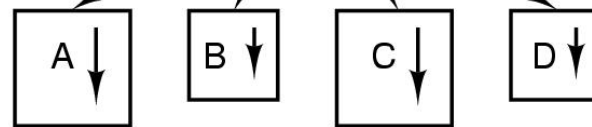
Chavamento de contexto

One program counter



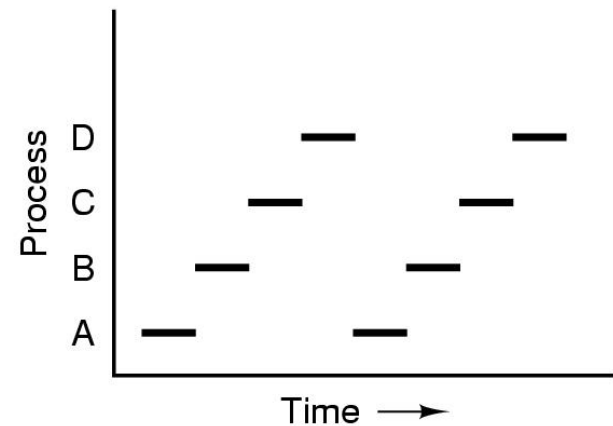
(a)

Four program counters



(b)

Ilusão de que os 4 processos
estão em paralelo



(c)

Criação de um Processo

Processos são criados por 4 eventos principais:

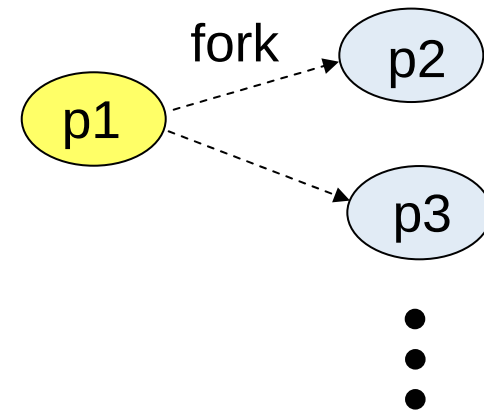
- Inicialização do sistema
 - Processos *foreground* e *background* (*daemons*)
- Processos em execução que usam uma “*system call*” para criar outros processos
 - Ex.: leitura de grandes quantidades de dados via um *buffer*
- Solicitação de usuário para criação de um novo processo
 - Ex.: disparo de novos processos via teclado ou mouse
- Inicialização de um *job* em sistemas *batch*
 - Para cada job um processo é criado pelo processo SO

Criação de um Processo (cont.)

- Processos são sempre criados por um processo em execução via a “system call” correspondente

- *System call:*

- No unix → fork
- No windows → CreateProcess



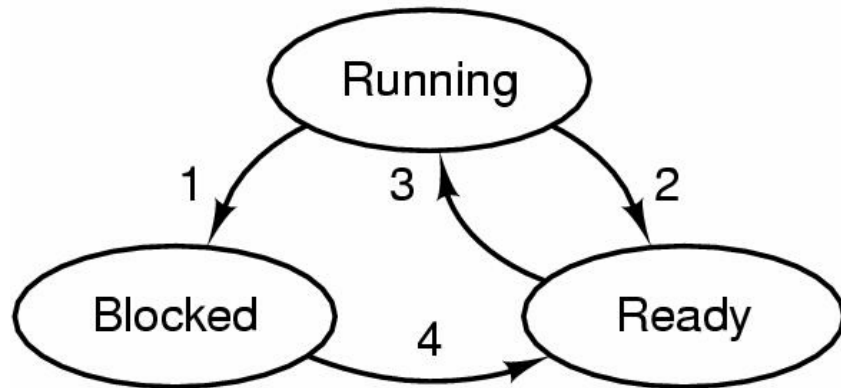
- Quando criado, um processo filho tem o seu próprio espaço de endereçamento
 - No unix, isso ocorre em duas etapas

Destruição de um Processo

Há 4 situações em que um processo é terminado

- Finalização normal ao término da sua tarefa (**voluntário**)
 - O próprio processo fecha seus arquivos abertos, e termina
- Finalização por erro detectado durante a sua execução (**voluntário**)
 - Processo detecta um erro externo e aborta execução
- Finalização por erro fatal (**involuntariamente**)
 - Processo contém um erro (bug)
- Finalização por outro processo (**involuntariamente**)
 - O outro processo tem que ter privilégio para tal

Estados de um Processo



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Um processo pode se encontrar nos seguintes estados

- Execução (running)
 - Processo ocupa a CPU (exclusivamente)
- Pronto (ready)
 - Processo está pronto para ocupar a CPU, mas deve esperar pela sua vez em fila
- Bloqueado (blocked)
 - Processo não pode executar enquanto um evento externo (por exemplo, a resposta do controlador de HD) não ocorrer

Informações agregadas a um Processo

- O SO possui uma tabela contendo o ID de cada processo
- Cada registro da tabela refere-se a um processo
- Tais registros são chamados de PCB (Process Control Block)

PCB1
PCB2
PCB3
⋮

← Tabela de processos

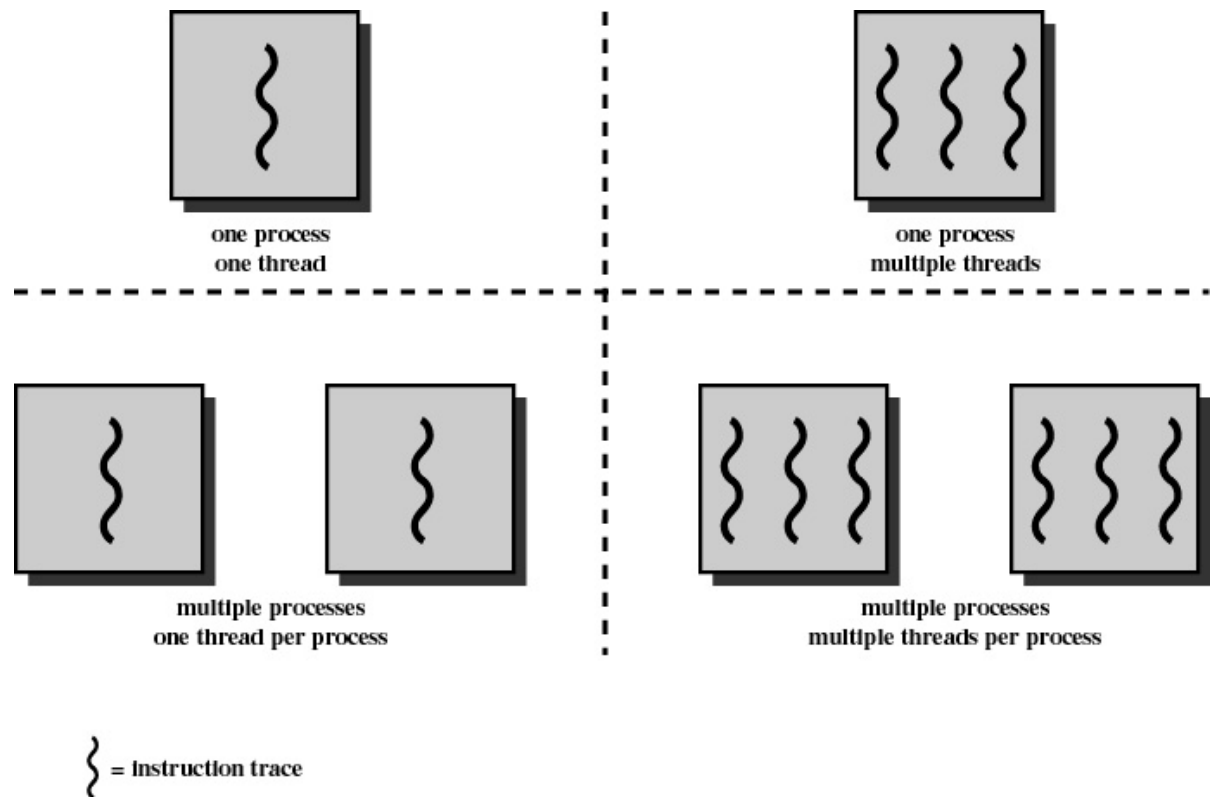
- Todas as informações relevantes aos processos estão nos seus PCBs. Ex.: o registrador PC, o registrador de pilha, os arquivos abertos, o usuário dono do processo, etc

THREADS

- As threads otimizam o desempenho dos processos
 - Várias tarefas podem ser executadas “em paralelo” dentro do processo
 - Chaveamento entre processos é muito dispendioso quando desnecessário

Nesse contexto,
os processos agrupam
recursos e as threads
executam as tarefas junto
à CPU

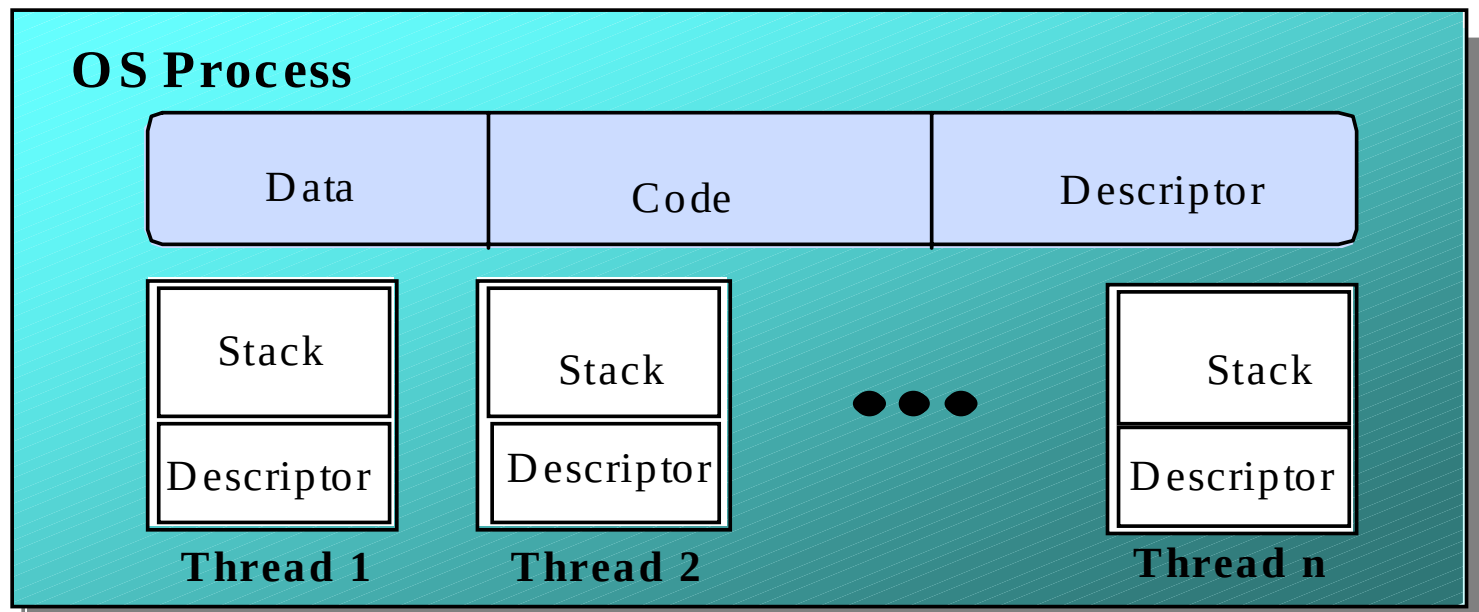
Uma thread representa
um fluxo de execução
dentro do processo



Multithreading

info compartilhada por
todas as threads

PROCESSO



chaveamento
entre threads
é muito mais
simples e
rápido!

as threads trabalham em cooperação e
pertencem a um mesmo usuário

Características das Threads

- Tal qual um processo, uma thread pode estar em um dos seguintes estados:
 - Execução
 - Pronto
 - Bloqueado
- Uma thread pode criar outras threads, mas o espaço de endereçamento é comum a todas
- Todas as threads de um processo podem acessar/alterar um arquivo ou alocar memória
- Cada thread tem a sua própria pilha para que o chaveamento entre threads ocorra eficientemente
- Como as threads são cooperativas, elas podem ceder a CPU umas às outras voluntariamente

Motivação para uso das Threads

- Muitas aplicações possuem múltiplas atividades que devem ser realizadas ao mesmo tempo
- Algumas dessas threads podem estar bloqueadas em alguns momentos
- Decompondo tais aplicações em múltiplas threads seqüenciais (rodando “em paralelo”) facilita a programação
- O fato de se compartilhar o espaço de endereçamento e todos os dados, torna as threads insubstituíveis
- A criação de uma thread pode ser até 100 vezes mais rápido do que a criação de um processo
- Threads são muito úteis em sistemas multiprocessados, onde o paralelismo real ocorre

Exemplo de uso das Threads

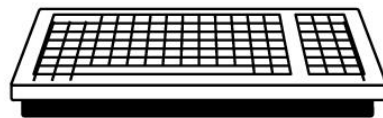
■ 1- PROCESSADOR DE TEXTO (EX.: WORD)

Cada thread trabalha independentemente

Controla formatação do texto na tela do micro

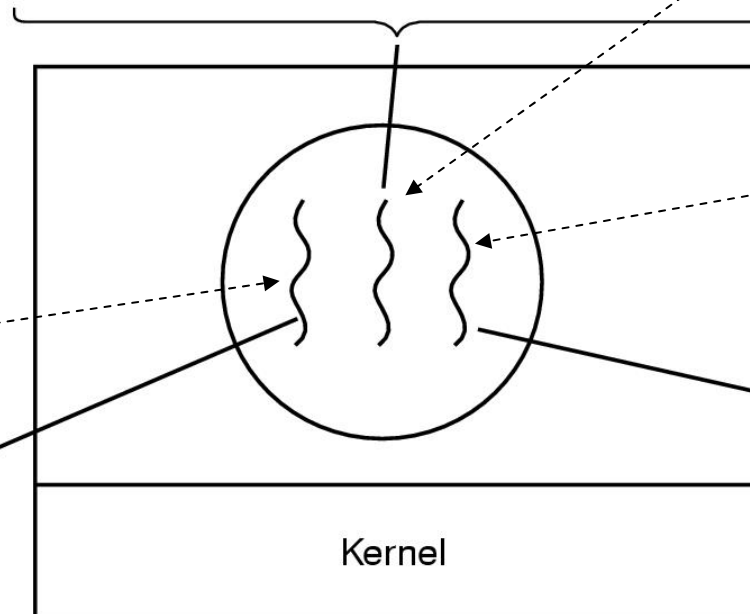
Usar 3 processos não funcionaria aqui: dados devem ser manipulados simultaneamente !

Controla info vinda do teclado

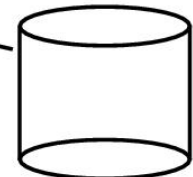


Keyboard

Four score and seven years ago, our fathers brought forth upon this continent a new nation: conceived in liberty, and dedicated to the proposition that all men are created equal.	Now we are engaged in a great civil war testing whether that nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battlefield of that war.	We have come to dedicate a portion of that field as a final resting place for those who here gave their lives that this nation might live. It is altogether fitting and proper that we should do this.	But, in a larger sense, we cannot consecrate we cannot hallow this ground. The brave men, living and dead, who struggled here have consecrated it, far above our poor power to add or detract. The world will little note, nor long remember, what we say here, but it can never forget what they did here.	It is for us the living, rather, to be dedicated here to the unfinished work which they who fought here have thus far so nobly advanced. It is rather for us to be here dedicated to the great task remaining before us, that from these honored dead we take increased devotion to that cause for which they gave the last full measure of devotion, that we here highly resolve that these dead shall not have died in vain that this nation, under God, shall have a new birth of freedom and that government of the people, by the people, for the people, shall not perish from the earth.
--	---	--	---	---



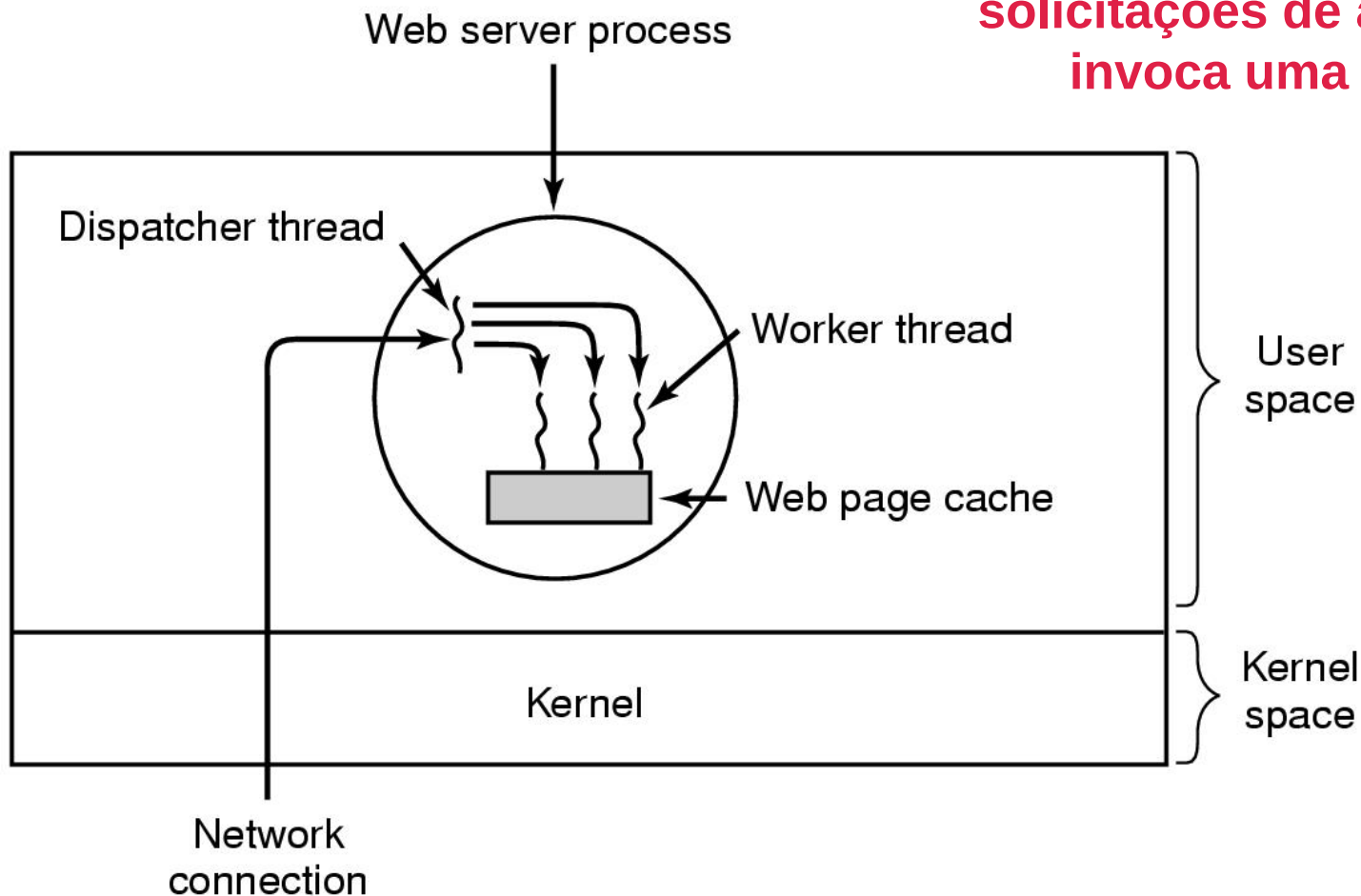
Controla backup automático



Disk

Outro exemplo de uso das Threads

■ 2- Um web server com multithread



A thread Dispatcher monitora solicitações de acesso à página e invoca uma thread worker

Usa página em cache se possível, **SENÃO** busca no Disco e fica bloqueado até recepção do disco



Possibilita ilusão de paralelismo

Implementação de Threads

- Threads podem ser implementadas como:
 - Biblioteca de pacotes em espaço de usuário
 - Pacote gerenciado pelo kernel
 - Pacote híbrido

IPC (InterProcess Communications)

- Processos normalmente se comunicam
 - Uma padronização para troca de info é necessária
 - Proteção entre os processos precisa ser garantida
 - Sincronização é fundamental
- Threads também se comunicam entre si, e as soluções são similares às apresentadas aqui p/ processos

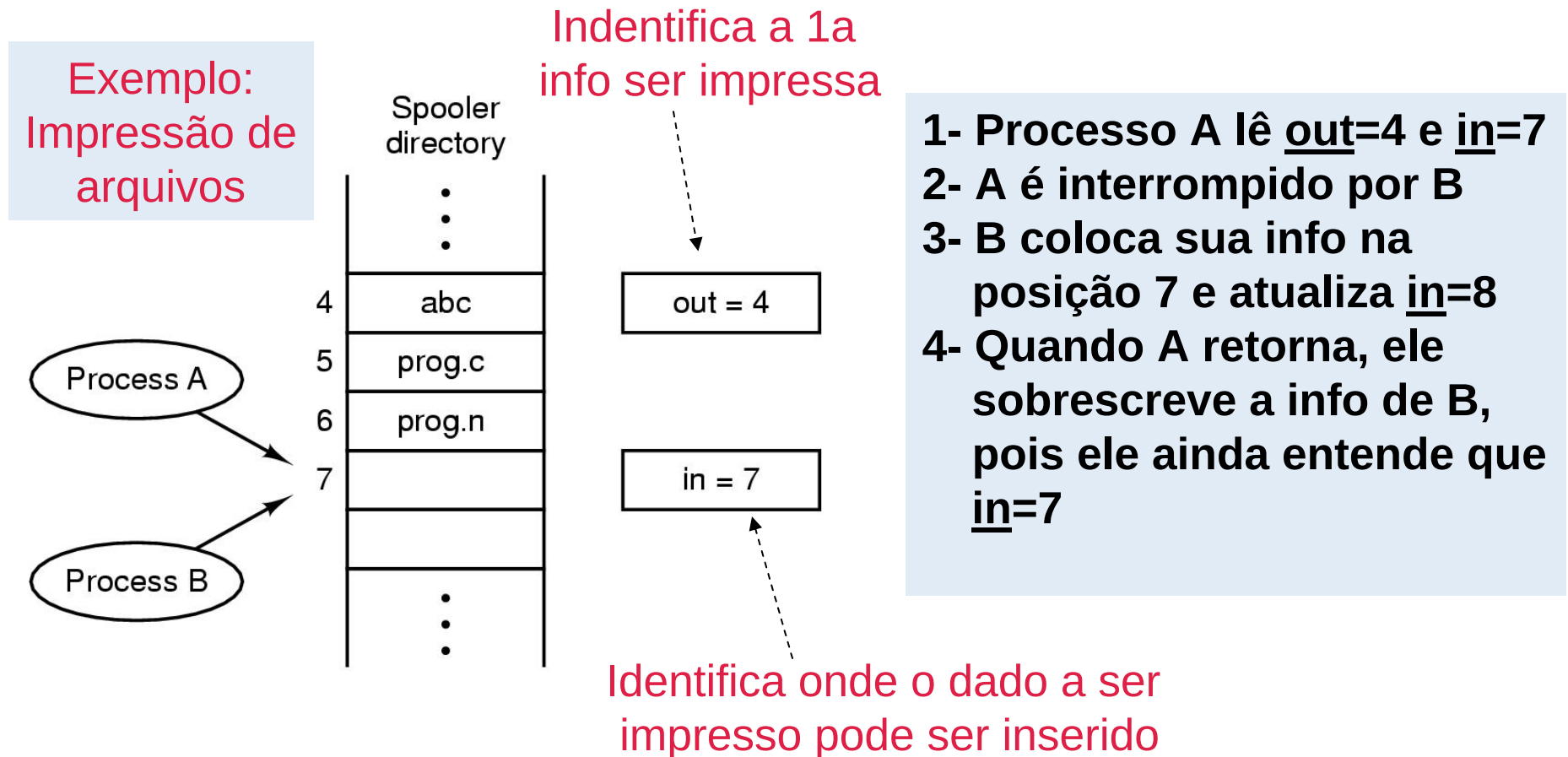
Processo A

Processo B



O Problema da Seção Crítica

- Quando dois ou mais processos compartilham um recurso, pode haver problemas, como ilustrado aqui:

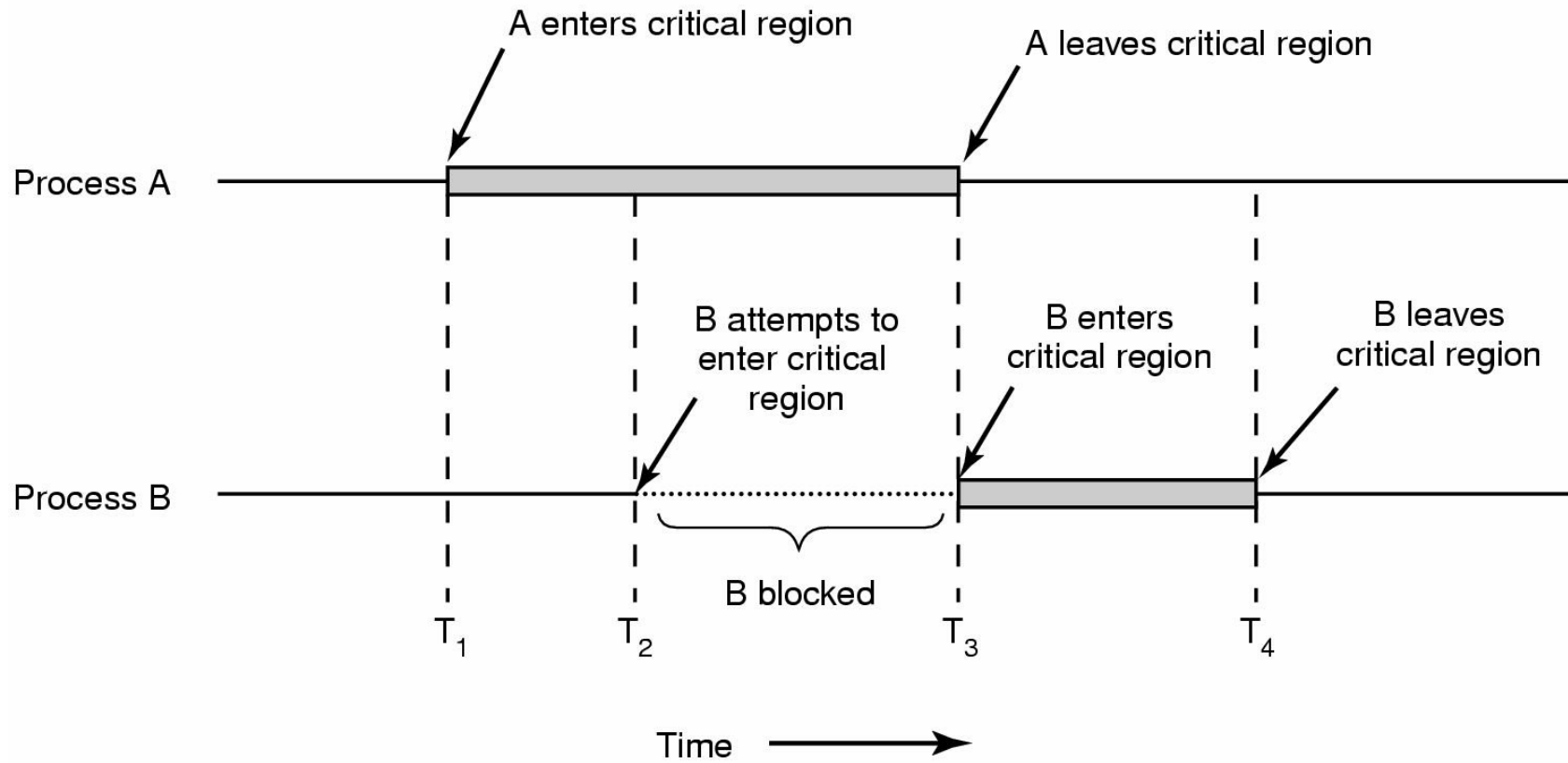


Solução ao Problema da Seção Crítica

- Evitar que mais de um processo acesse a seção crítica ao mesmo tempo
 - **EXCLUSÃO MÚTUA**
- Uma solução completa e eficiente deve satisfazer as 4 condições seguintes:
 - A seção crítica não deve jamais estar sendo acessada por dois processos simultaneamente
 - Nenhuma consideração a respeito da velocidade ou quantidade de CPUs deve existir
 - Nenhum processo rodando fora de sua seção crítica pode bloquear outros processos
 - Nenhum processo deveria esperar indefinidamente para acessar a sua seção crítica

Solução ao Problema da Seção Crítica (cont.)

- A exclusão mútua evitando conflitos de acesso à seção (região) crítica



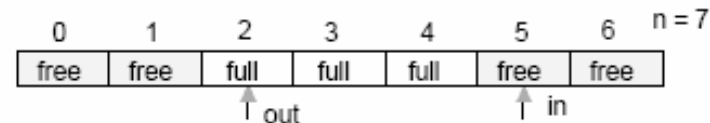
O Problema do produtor Consumidor

- Um produtor produz dados que são consumidos por um consumidor (ex.:, spooler e impressora)
- Os dados produzidos e ainda não consumidos são armazenados em um BUFFER
- Podem haver vários produtores e consumidores
- Idealmente o BUFFER é de tamanho infinito

Variáveis:

“in” → onde dado dever ser Inserido

“out” → dado a ser retirado



```
var buffer: array[0..n-1] of items;    /* circular array */
in = 0
out = 0

/* producer */
repeat forever
  ...
  produce item nextp
  ...
  while (in+1 mod n == out)
    do nothing
  buffer[in] = nextp
  in = in+1 mod n
end repeat

/* consumer */
repeat forever
  while (in == out)
    do nothing
  nextc = buffer[out]
  out = out+1 mod n
  ...
  consume item nextc
  ...
end repeat
```

Algoritmos para Exclusão mútua

- Desabilitando interrupções
- Busy Waiting
 - Variável lock
 - Instrução Test and Set
- Semáforos
- Passagem de mensagens

Desabilitando interrupções

- Quando um processo entra na sua seção crítica, o mesmo desabilita todas as interrupções
- Fica assim garantido a esse processo acesso exclusivo à seção crítica
- Ao sair da seção crítica, as interrupções são habilitadas
- Problemas
 - Há que se permitir que processos de usuários desabilitem interrupções
 - Em sistemas multiprocessados (mais de uma CPU), somente uma CPU terá as interrupções suspensas, os processos nas demais CPUs podem acessar a seção crítica normalmente
- Somente viável para processos do sistema!

Busy Waiting – variáveis de trava (*lock*)

- Existe uma variável *lock* que é inicializada com zero
- Um processo desejando acessar a seção crítica verifica se *lock*=0
- Se sim, o processo entra na seção crítica e ajusta *lock*=1
- Demais processos devem aguardar até que *lock* retorne a zero
- Apresenta os mesmos problemas vistos no exemplo do servidor de impressão
- “*Busy waiting*” significa um processo executando um loop para detectar “seção crítica livre”
- ***Busy waiting*** → desperdício de CPU

Instrução TSL (hardware)

- TSL (Test and Set Lock) existe na maioria dos computadores

Ocorre busy waiting!

TSL RX, LOCK ($RX \leftarrow LOCK$ e $LOCK \leftarrow 1$)

- TSL é indivisível, não pode ser interrompida

enter_region:

TSL REGISTER, LOCK	copy lock to register and set lock to 1
CMP REGISTER, #0	was lock zero?
JNE enter_region	if it was non zero, lock was set, so loop
RET	return to caller; critical region entered

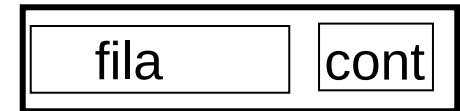
leave_region:

MOVE LOCK, #0	store a 0 in lock
RET	return to caller

Semáforo (S)

- Possui um contador (cont) e uma fila
- Somente duas operações: UP e DOWN

semáforo



```
DOWN(S){
    s.count-- ;
    if(s.count<0){
        coloque o processo na s.fila;
        entre em estado de bloqueio;
    } else {entra na seção crítica}
}
```

```
UP(S){
    s.count++ ;
    if(s.count<=0){
        remove um processo P da s.fila;
        coloque o processo P na lista de pronto;
    }
}
```

* Inicialmente s.cont=1, a cada nova solicitação pela seção crítica o seu valor é decrementado

* O valor int. do contador representa qtidade de acessos pendentes

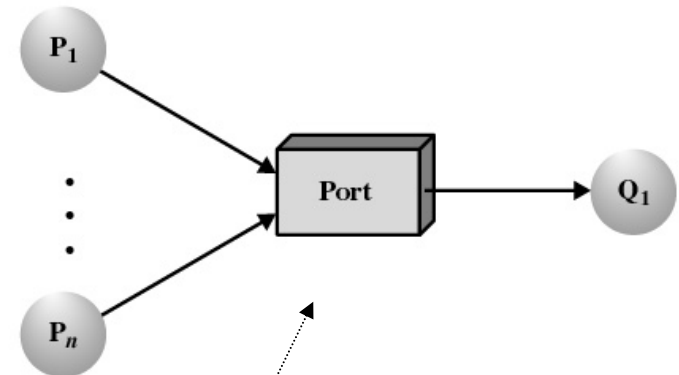
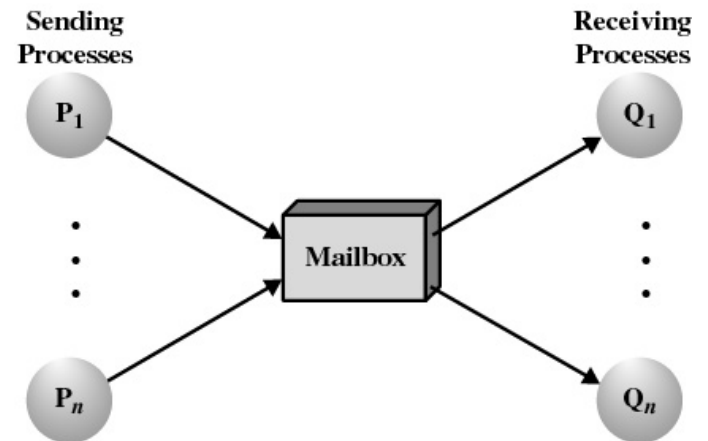
Ex.: após 3 solicitações simultâneas

s.fila: 2 processos

s.count: -2

Passagem de Mensagens

- Mais apropriado à sistemas distribuídos e multiprocessados com memória compartilhada
- Mensagens
 - `Send(destino,mensagem)`
 - `Receive(origem,mensagem)`
- Perda de mensagens pode ocorrer
- Troca de mensagens pode ser
 - Direta
 - Indireta via caixas de mensagens ou portas



comum em arquiteturas
cliente servidor

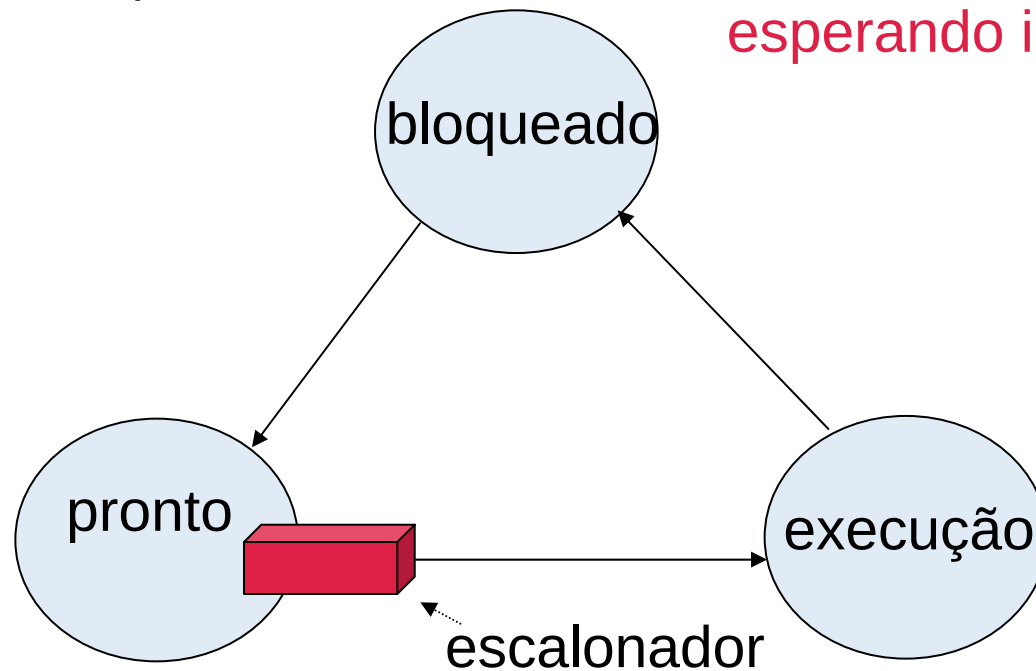
Algoritmos de Escalonamento

- Na multiprogramação, o escalonador decide qual processo deve obter a CPU

- Aspectos importantes
 - Tempo de resposta
 - Throughput
 - Eficiência do processador

Um bom escalonador !

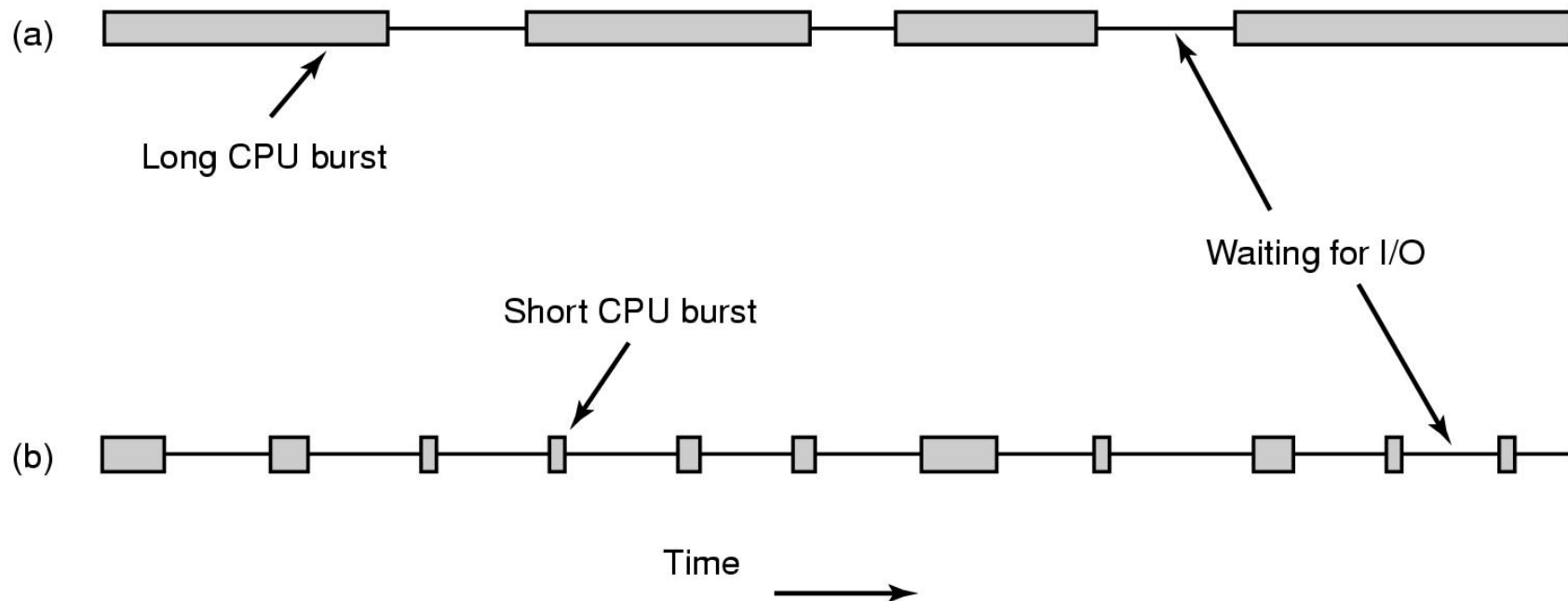
- Prioriza processos importantes
- Não deixa processos esperando indefinidamente



Processos CPU-bound e I/O-bound

- Há processos que permanecem a maior parte do tempo em execução, enquanto outros mais esperam por sinalização de periféricos
 - (a) Processo CPU-bound
 - (b) Processo I/O-bound

Escalonador deve considerar tais características !!!



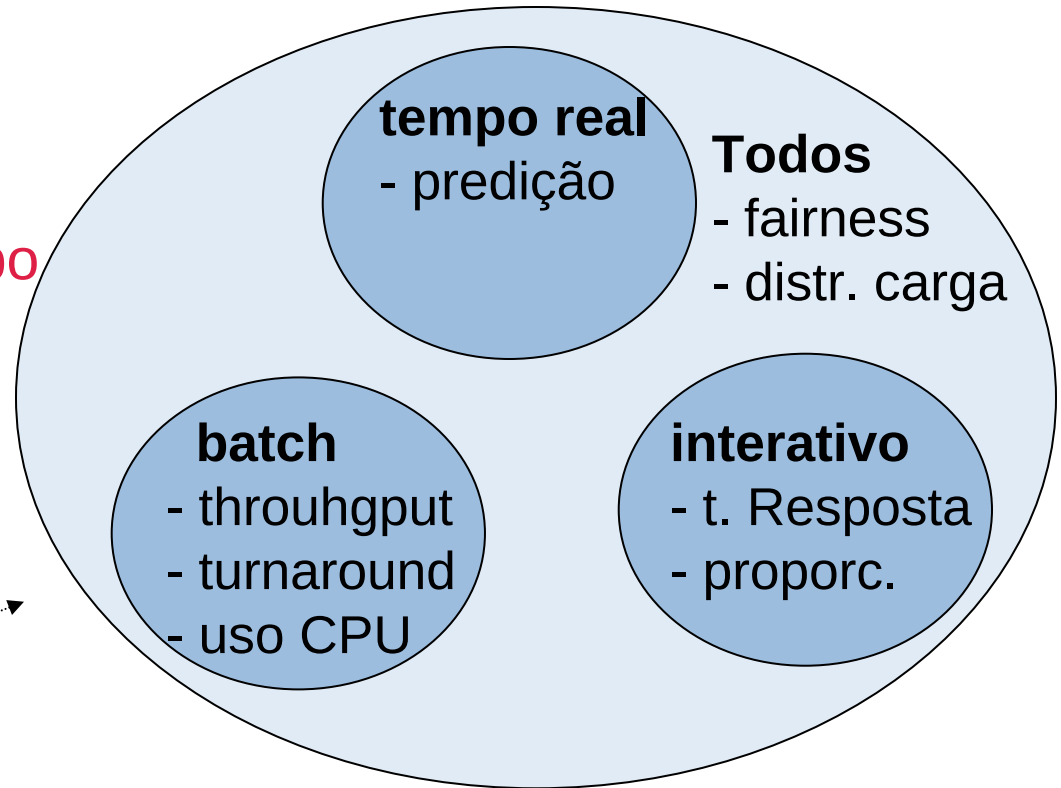
Categorias de Escalonadores

- Escalonadores específicos para os sistemas
 - **Batch** → preempção não é necessária
 - **Interativo** → preempção é necessária
 - **Tempo real** → processos são normalmente limitados por definição

Preempção:

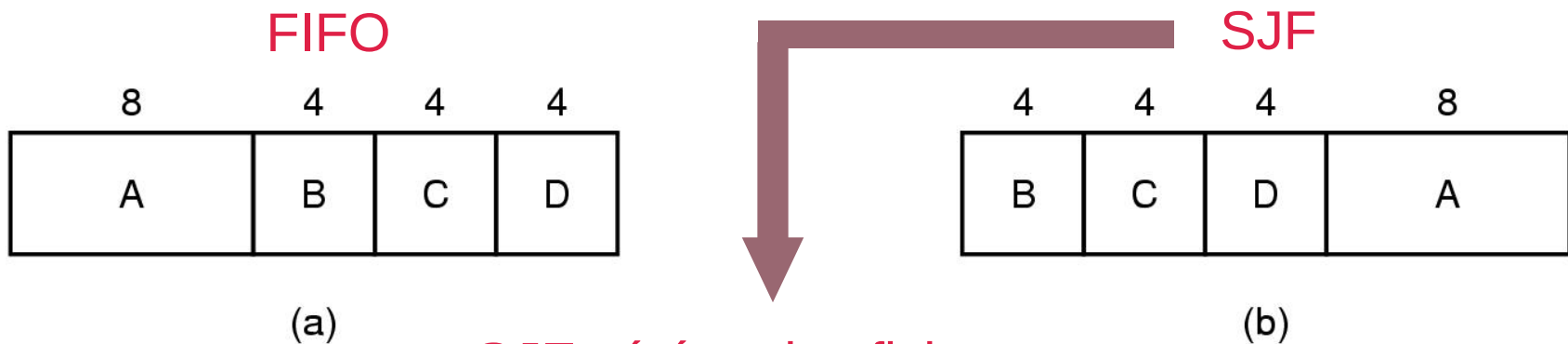
possibilidade de um processo ser retirado de execução pelo escalonador por limite de tempo

Principal objetivo de cada escalonador



O Escalonador Shortest Job First (SJF)

- O processo é selecionado pelo tamanho, o menor primeiro
- Adequado para sistemas *batch*
- Não preemptivo
- Tempo de execução dos processos deve ser conhecido a priori



SJF só é mais eficiente se todos os processos estiverem disponíveis simultaneamente

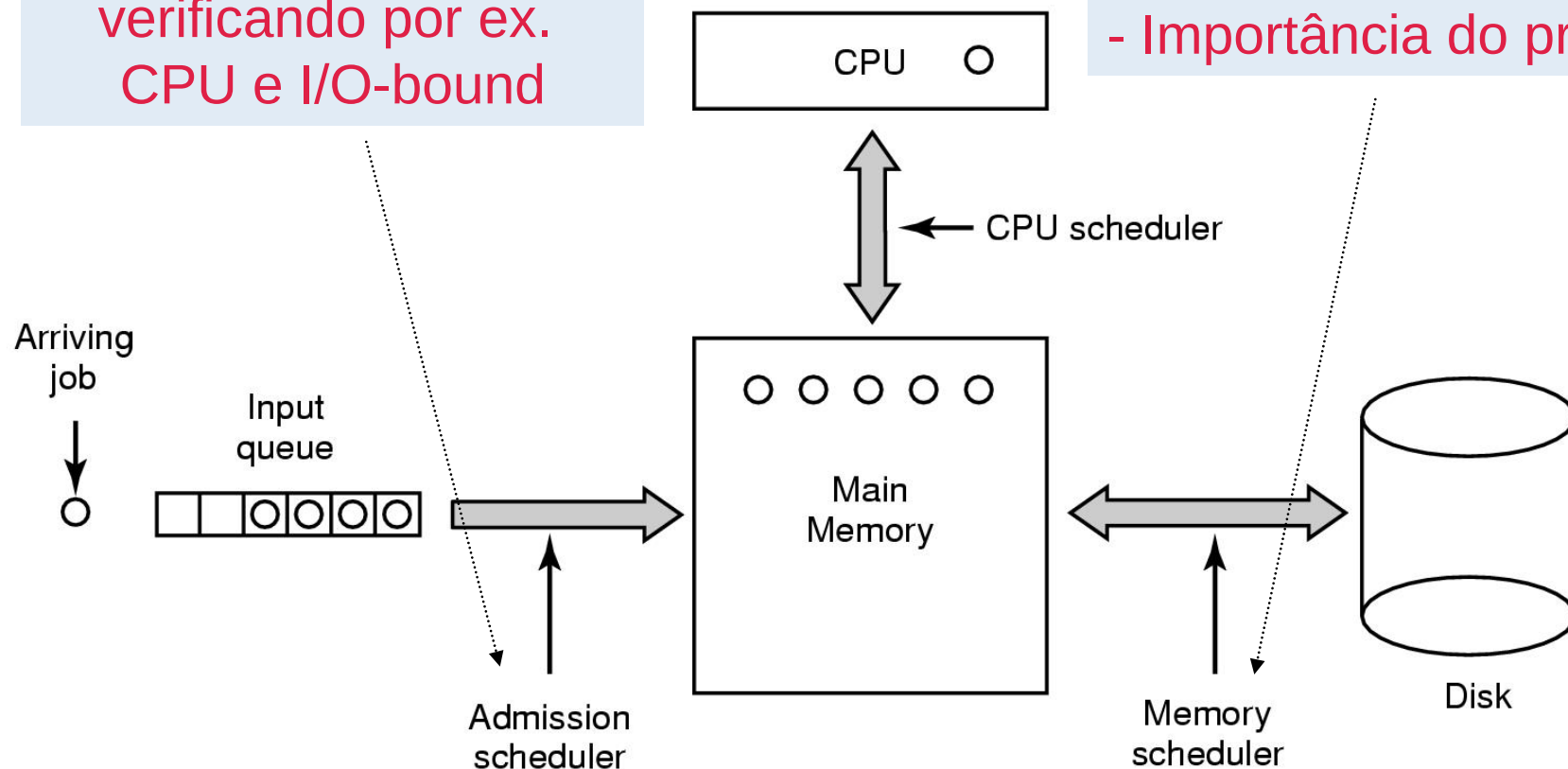
O Escalonador de 3 níveis

- Possui três escalonadores distintos:
 - de admissão, de memória e de CPU

Seleciona os processos verificando por ex. CPU e I/O-bound

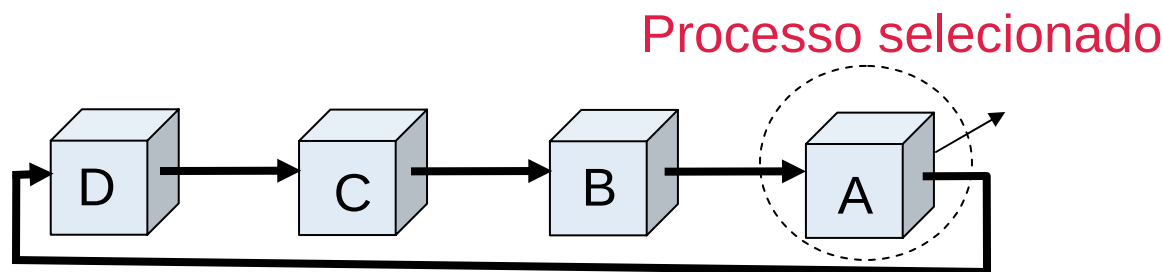
periodicamente verifica os processos

- Duração no disco
- Tempo de uso da CPU
- Tamanho do processo
- Importância do processo



O Escalonador Circular (Round-Robin)

- Um dos mais simples, antigo, justo (*fair*) e usado algoritmo
- A cada processo é atribuído uma fatia de tempo (**quantum**)
- Há uma fila com os processos prontos para execução
- Processos que usam a CPU além da sua fatia são colocados no final da fila
- Definição do valor do quantum é crítico
 - Se muito pequeno, ocorre perda de rendimento devido chaveamento de contexto
 - Se muito grande, torna operações interativas muito lentas



O Escalonador com prioridades

- Cada processo tem uma prioridade
- O de maior prioridade é escolhido em primeiro lugar
- Normalmente processos têm suas prioridades reduzidas a cada execução beneficiando processos de menor prioridade
- Podem ser agrupados em classes de prioridade

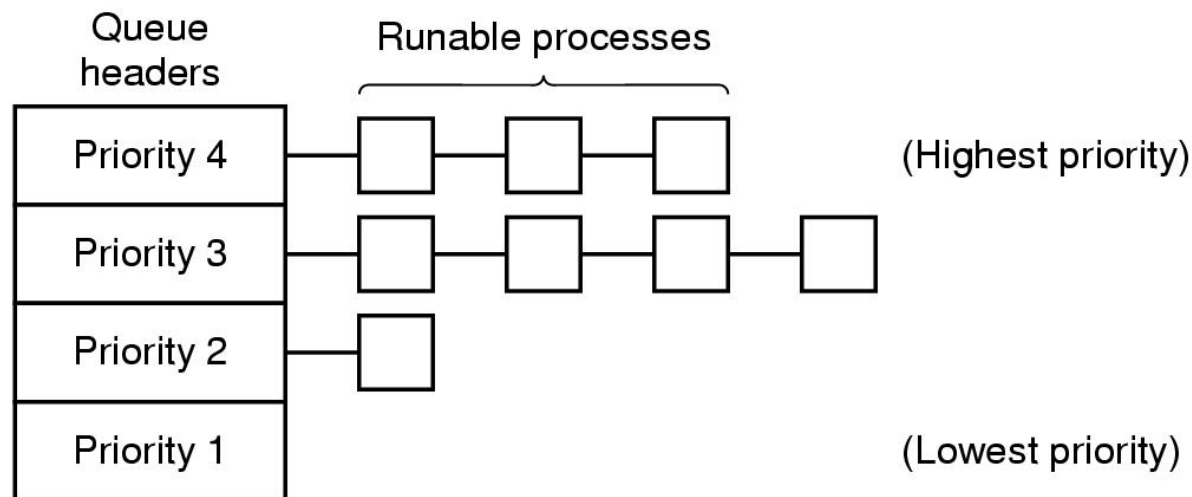
Como ajustar prioridades ?

Exemplo:

Fatia de tempo: 50 ms

1- Uso da fatia: 1ms
Prioridade: $50/1=50$

2- Uso da fatia: 25ms
Prioridade: $50/25=2$



Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004

Sistemas Operacionais

Deadlock

Conteúdo

- Deadlock e recursos
- Modelagem de deadlock
- Detecção de deadlock
- Recuperação de deadlock
- Evitando deadlock
- Prevenção de deadlock

DEADLOCK

- Envolve necessidade conflitante por recursos entre dois ou mais processos

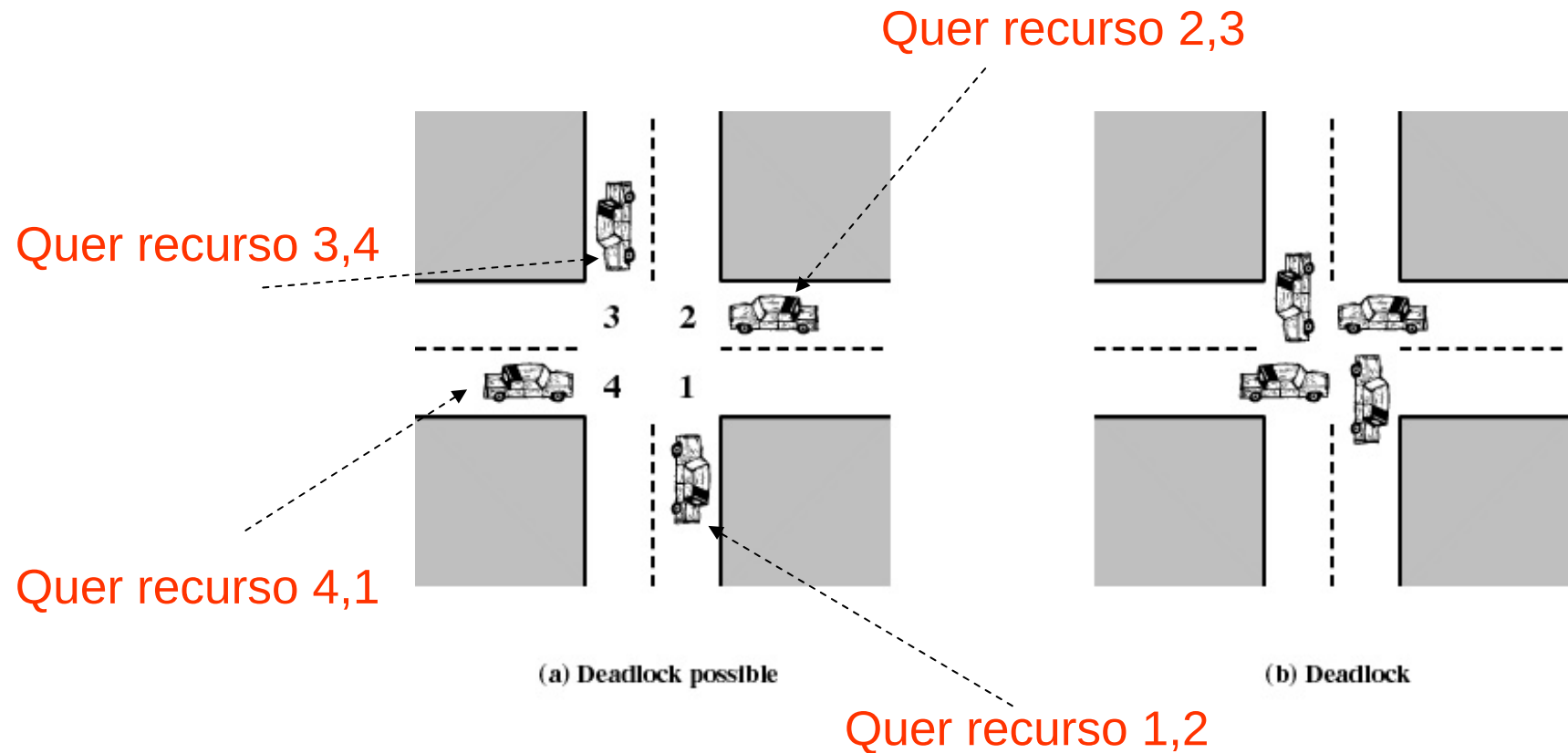


Figure 6.1 Illustration of Deadlock

Recursos

- Deadlock pode ocorrer quando processos têm acesso exclusivo a recursos do sistema
- Recursos podem ser:
 - Um **dispositivo de hardware** → impressora, fita magnética, etc
 - Uma **informação** → registro travado em um banco de dados, arquivo, etc
- Em geral, um recursos pode ser composto por várias instâncias (ex.: HD, impressora, etc)
- Um recurso pode ser utilizado por apenas um processo por vez

Recursos (cont.)

- Processos com acesso a recursos negados são colocados em estado de bloqueio
- A ordem em que os recursos são requeridos é importante

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

sem deadlock

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

deadlock em potencial

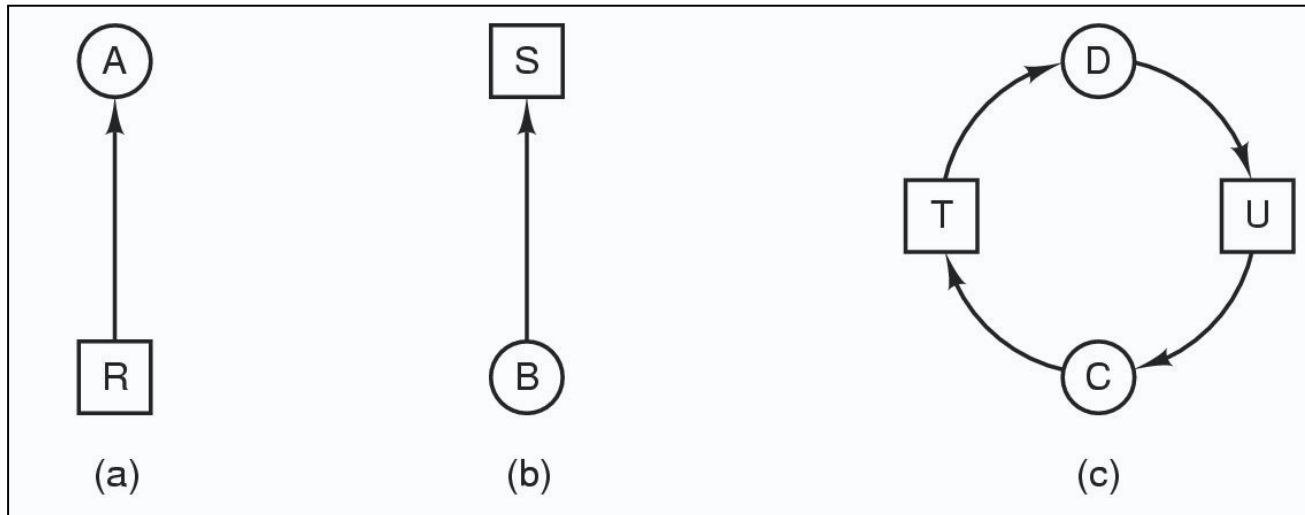
- 1- Processo_A
obtem rec_1
- 2- Processo_B
obtem rec_2
- 3- proc_A bloq.
por rec_2
- 4- proc_B bloq.
Por rec_1
- 5- deadlock ...

DEADLOCK: definição formal

- “Um conjunto de processos está em estado de *deadlock* se cada um de seus processos está esperando por um evento que somente outro processo no conjunto pode causar”
- Condições para ocorrência de deadlock
 - **Exclusão mútua:** somente um processo por vez pode usar um recurso
 - **Posse e espera:** processos já de posse de recursos podem requerer novos recursos
 - **Sem preempção:** nenhum processo pode ser forçado a liberar um recurso, mas deve fazê-lo voluntariamente
 - **Espera circular:** há uma cadeia circular de 2 ou mais processos, os quais esperam pelo recurso do processo seguinte na cadeia

DEADLOCK: modelagem

- Uso da teoria dos grafos



- a) Recurso R atribuído ao processo A
- b) Processo B requerendo/esperando pelo recurso S
- c) Processo C e D em deadlock sobre os recursos T e U

DEADLOCK: modelagem (cont.)

A
Request R
Request S
Release R
Release S

(a)

B
Request S
Request T
Release S
Release T

(b)

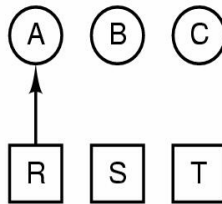
C
Request T
Request R
Release T
Release R

(c)

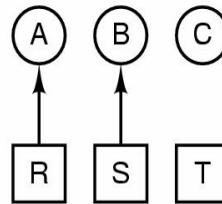
Escalonador:
Circular (round-robin) !

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R
deadlock

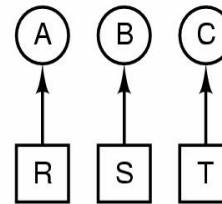
(d)



(e)

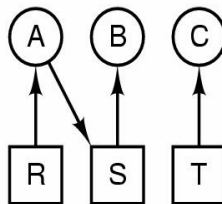


(f)

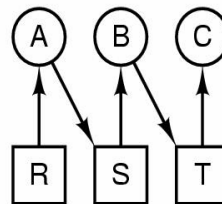


(g)

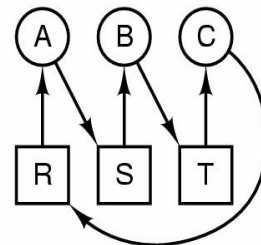
A ordem de execução
dos processos leva o
sistema a uma espera
circular → deadlock



(h)



(i)

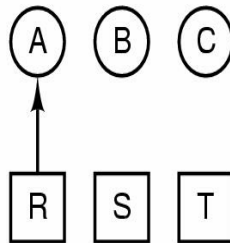


(j)

DEADLOCK: modelagem (cont.)

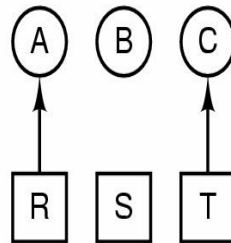
Mudando a ordem de execução dos processos previne o deadlock !!!

1. A requests R
 2. C requests T
 3. A requests S
 4. C requests R
 5. A releases R
 6. A releases S
- no deadlock

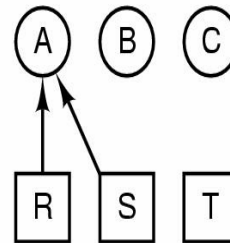


(k)

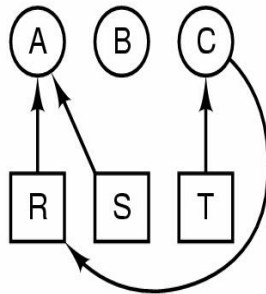
(l)



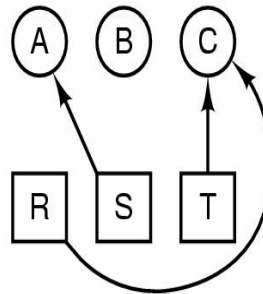
(m)



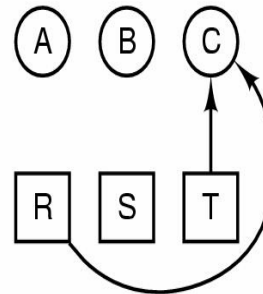
(n)



(o)



(p)



(q)

DEADLOCK: detecção c/ múltiplos recursos p/ tipo

Vetor de recursos existentes

Resources in existence
($E_1, E_2, E_3, \dots, E_m$)

Current allocation matrix

C_{11}	C_{12}	C_{13}	$\dots$	C_{1m}
C_{21}	C_{22}	C_{23}	$\dots$	C_{2m}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
C_{n1}	C_{n2}	C_{n3}	$\dots$	C_{nm}

Row n is current allocation to process n

Matriz de alocação atual

Resources available
($A_1, A_2, A_3, \dots, A_m$)

Vetor de recursos disponíveis

Request matrix

R_{11}	R_{12}	R_{13}	$\dots$	R_{1m}
R_{21}	R_{22}	R_{23}	$\dots$	R_{2m}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
R_{n1}	R_{n2}	R_{n3}	$\dots$	R_{nm}

Row 2 is what process 2 needs

Matriz de pedido

$m \rightarrow$ nr. de recursos
 $n \rightarrow$ nr. de proc

■ Algoritmo baseado em matriz

1. Procure por um processo desmarcado, P_i , para o qual a i -ésima linha da matriz R é menor ou igual a A
2. Se encontrado, adicione a i -ésima linha de C a A , marque o processo e retorne ao procedimento 1
3. Se não existe tal processo, o algoritmo termina

→ Quando o algoritmo termina, todos os processos desmarcados estão em deadlock

DEADLOCK: exemplo de detecção

- 3 processos e 4 classes de recursos

Execução do algoritmo

- Verificar linha a linha da matriz R
- P1 não pode ser atendido: 0 CD-Rom disponível
- P2 também não: nenhum scanner livre
- P3 é atendido, roda e finalmente retorna todos seus recursos
→ $A=(2\ 2\ 2\ 0)$
- P2 pode então rodar e retornar
→ $A=(4\ 2\ 2\ 1)$
- O processo remanescente pode então rodar → SEM DEADLOCK

	Tape drives	Plotters	Scanners	CD Roms
$E =$	4	2	3	1
$A =$	2	1	0	0

P1 → tem um scanner

P2 → tem 2 TDs e um CD-Rom

P3 → tem 1 plotter e 2 scanners

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix} \begin{matrix} P1 \\ P2 \\ P3 \end{matrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Cada processo precisa de recursos adicionais !

DEADLOCK: exercício

- Considerando-se o exemplo anterior, como ficaria a situação se o processo P2 necessitasse dos recursos abaixo?
 - 1 CD-ROM
 - 2 TDs
 - 1 plotter

DEADLOCK: aplicabilidade da detecção

Quando a detecção deve ser realizada?

- Sempre que um recurso for solicitado
 - rápido, mas dispendioso em termos de uso de CPU
- Periodicamente
 - lento
- Somente quando o uso da CPU cair abaixo de um certo limite
 - quando alguns processos se encontram em deadlock, provavelmente poucos processos estão em execução e assim a CPU deve estar bem ociosa

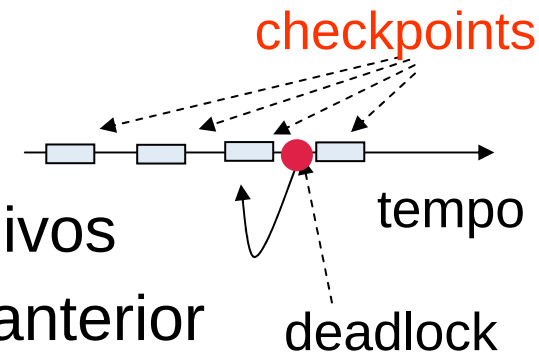
Recuperação de deadlock

■ Por preempção

- Retire o recurso de algum outro processo
- Dificuldade: nem sempre é viável para se implementar

■ Por retrocesso a checkpoints

- A medida que um processo roda, seus dados são armazenados em arquivos
- Sob deadlock, o ponto imediatamente anterior ao uso do recurso causando deadlock é retornado

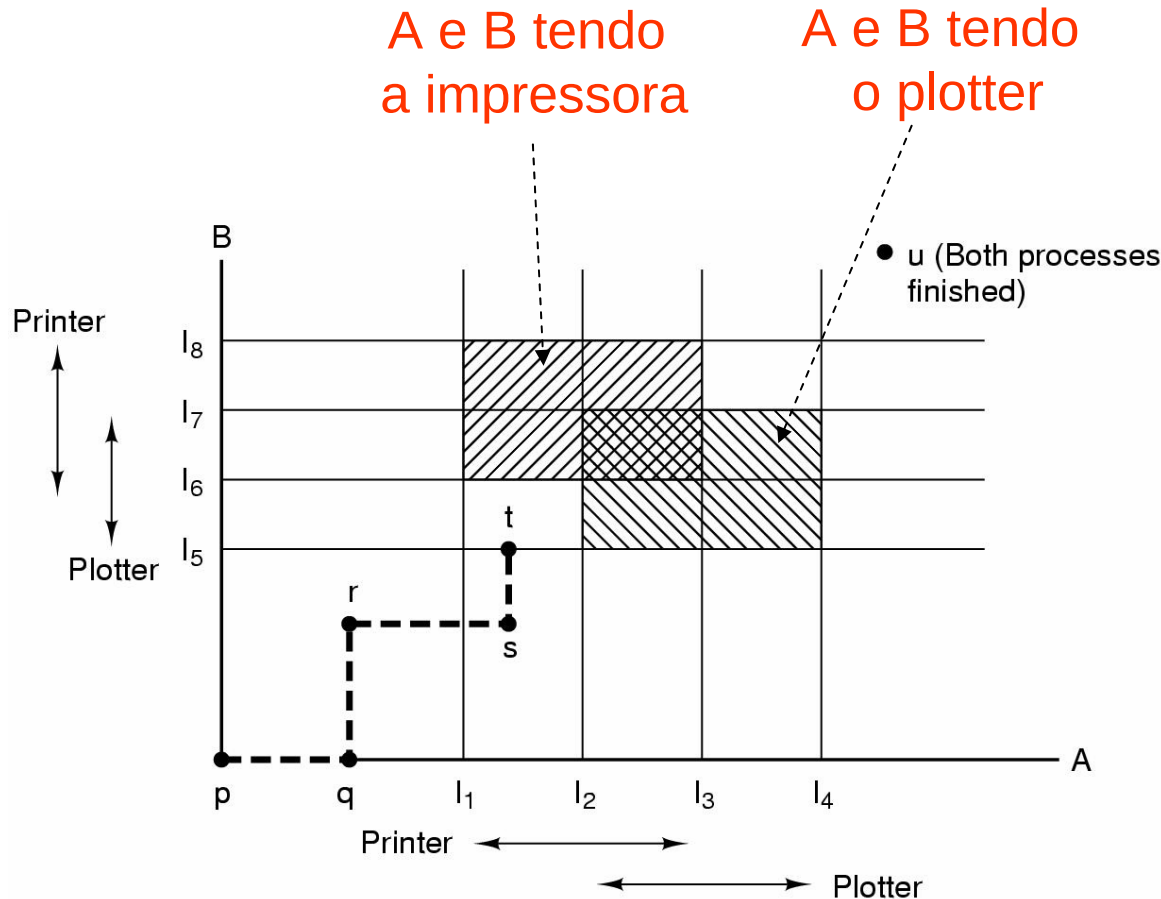


■ Por destruição de processos

- Um processo no loop de deadlock ou outro processo que possua um recurso necessário ao loop
- Cautela na escolha do processo a ser destruído

Evitando deadlocks: trajetória de recursos

- Evita-se deadlocks tendo-se certas info de antemão, alocando-se recursos apropriadamente
- No ponto t, B solicita o plotter, se receber o sistema irá a deadlock
- A região riscada (escura) é insegura
- A exclusão mútua evita que se entre nessa região
- O mais sensato em t, é bloquear B até que A esteja em l4
- Depois de l4, u pode ser alcançado sem deadlock



Evitando deadlocks: estados (in)seguro

O estado (a) é seguro !

Has Max		
A	3	9
B	2	4
C	2	7

Free: 3

(a)

$$3+2+2=7 \rightarrow 10-7=3$$

Has Max		
A	3	9
B	4	4
C	2	7

Free: 1

(b)

Has Max		
A	3	9
B	0	—
C	2	7

Free: 5

(c)

Has Max		
A	3	9
B	0	—
C	7	7

Free: 0

(d)

Has Max		
A	3	9
B	0	—
C	0	—

Free: 7

(e)

Neste exemplo, há um total de 10 instâncias do recurso em questão

Estado seguro

- Não há deadlock e existe uma ordem de escalonamento em que todos os processos podem rodar completamente ainda que precisem de todos os seus recursos imediatamente

Estado inseguro

- Há um deadlock potencial que ocorrerá se todos os recursos de cada processo forem necessários

Evitando deadlocks: estados (in)seguro (cont.)

A e C precisam de 5 recursos, mas só existem 4 !!!

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2

(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0

(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4

(d)

O estado (b) é inseguro !

A recebe mais um recurso

B recebe mais dois recursos

B termina e libera seus recursos

- A ordem de escalonamento pode ser importante
- No exemplo acima, A não deveria ter recebido o recurso adicional
- Estado inseguro não é sinônimo de deadlock

Evitando deadlocks: Banker's algoritmo

seguro

Has Max

A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

seguro

Has Max

A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

inseguro

Has Max

A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

- Modelo baseado num sistema bancário de uma cidade pequena
- Clientes → processos, unidades → recursos, banq. → sistema oper
- Este algoritmo verifica cada solicitação por recurso, e somente atende a solicitação se a mesma induzir o sistema a um estado seguro
- Solicitação não atendidas são colocadas em espera

Exercício

- Na figura (b) abaixo, se D requerer uma unidade a mais, isso levará a um estado seguro ou inseguro? E se a solicitação partir de C ao invés de D?

Has Max		
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

Has Max		
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

Has Max		
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

Prevenção de deadlocks

- As técnicas apresentadas para se evitar deadlocks são difíceis de serem implementadas por exigirem conhecimento sobre solicitações futuras
- A solução é evitar que pelo menos uma das quatro condições necessárias para ocorrência de deadlock ocorra, ou seja:
 - Atacando a condição de Exclusão mútua
 - Atacando a condição de Posse e espera
 - Atacando a condição de Sem preempção
 - Atacando a condição de Espera circular

Atacando a condição de exclusão mútua

- Se nenhum recurso fosse atribuído exclusivamente a um processo não haveria deadlocks
- Sem exclusão mútua pode-se criar o caos (ex.: acesso direto à impressora)
- A técnica de spooling para a impressora elimina o problema de exclusão mútua, pois só o “daemon” do spooler efetivamente acessa a impressora
- Problema é que nem todo processo pode usar a técnica de spooling
- **Recomendado:**
 - Somente atribuir recursos quando absolutamente necessário
 - Mínimo possível de processos devem ter privilégio para solicitar o recurso

Atacando a condição de posse e espera

- Pode-se eliminar o deadlock se um processo de posse de um recurso ficar impossibilitado de solicitar mais recursos
 - Processos requerem todos seus recursos antes de rodarem
 - Recursos disponíveis → processo roda por completo
 - Não disponíveis → processo espera
- Problemas
 - Total de recursos necessários só são sabidos qdo processo começa (→ Banker algoritmo seria usado)
 - Recursos não são usados otimizadamente (longas esp.)
- Variantes
 - Processo libera todos recursos em sua posse e, então solicita todos os recursos necessários de uma só vez

Atacando a condição de não preempção

- Solução pouco viável
- Certos recursos não podem ser removidos forçadamente sem prejuízos
- Ex.: uma impressora
 - É inviável parar uma impressão de um processo para começar a de outro processo !!!

Atacando a condição de espera circular

1. Imagesetter
2. Scanner
3. Plotter
4. Tape drive
5. CD Rom drive

(a)

se $i > j$, A não pode
Requerer j

Se $j > i$, B não pode
Requerer i

Então, loop é impossível



(b)



- Processos devem requerer recursos segundo uma ordem numérica pré-estabelecida
- Assim, os gráficos de recurso não têm loop
- pode ser difícil encontrar uma ordem que satisfaça a todos

Resumo da prevenção de deadlock

- Técnicas existentes

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Starvation

- Pode ocorrer em sistemas em que processos pequenos são escalonados prioritariamente
- Os processos longos podem ser adiados indefinidamente, embora não estejam bloqueados nem em deadlock
- Solução mais viável → escalonamento do tipo FIFO

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004

Sistemas Operacionais

Gerência de Memória

Conteúdo

- Mapeamento de memória
- Endereços lógico e físico
- Carga e ligamento dinâmicos
- Overlay e Swapping
- Alocação de memória
- Fragmentação
- Paginação
- Paginação hierárquica
- Segmentação
- Segmentação com Paginação

Memória

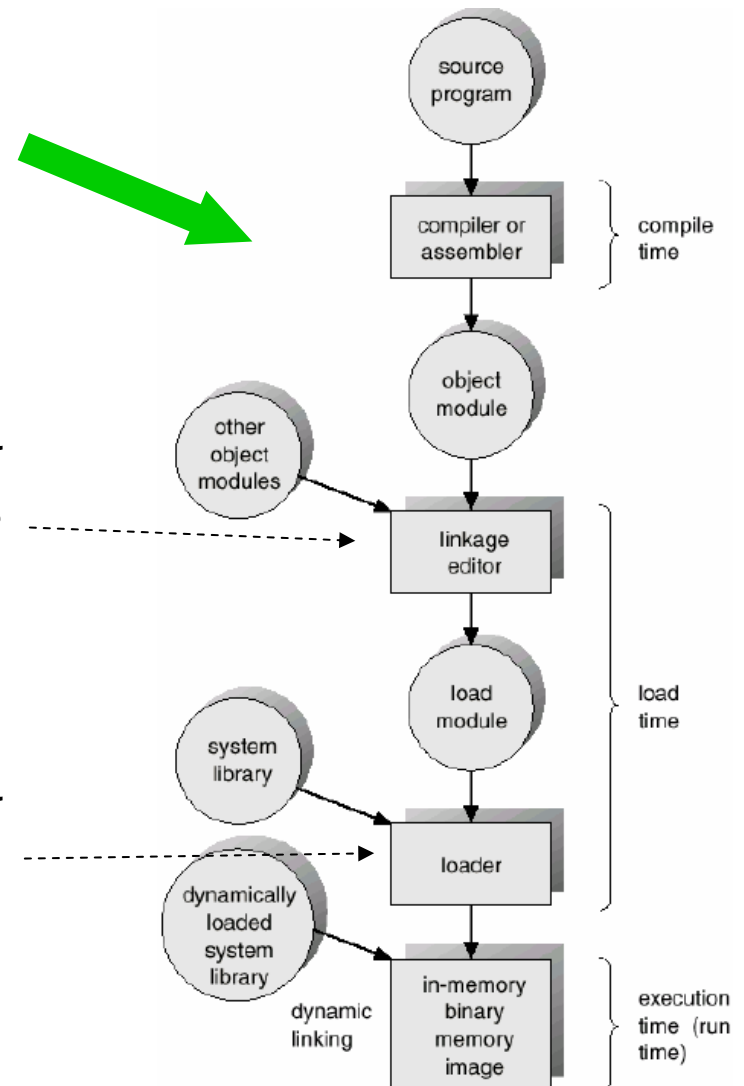
- A CPU busca instruções da memória guiada pelo valor no registrador PC
- Uma instrução pode requerer acessos adicionais à memória para leitura ou escrita
- Portanto, um programa precisa ser carregado em memória para ser executado
- **Fila de entrada** contém grupo de processos em disco (HD) a ser carregado na memória

Mapeamento de endereços de memória

- Normalmente, os programas de usuários podem estar carregados em qualquer parte disponível da memória
- Como tais programas usam endereços simbólicos, um mapeamento p/ end. real é preciso
- Três momentos possíveis para o mapeamento
 - **Na compilação:** se a localização de memória é conhecida a priori, o código absoluto pode ser gerado, mas mudanças no endereço inicial requer recompilação do código (ex.: MS-DOS)
 - **Na carga:** Compilador tem que gerar um código relocável. Mudanças requerem somente recarga do código do usuário
 - **Na execução:** mapeamento é adiado até a execução. Requer suporte de hardware tal como *registradores base e limite*

Mapeamento de endereços de memória (cont.)

- As diversas fases até um programa ser carregado em memória
- O *linker* (ligador) agrupa todos os códigos objetos necessários ao programa final
- O *loader* (carregador) carrega o programa na memória, com as bibliotecas apropriadas do sistema

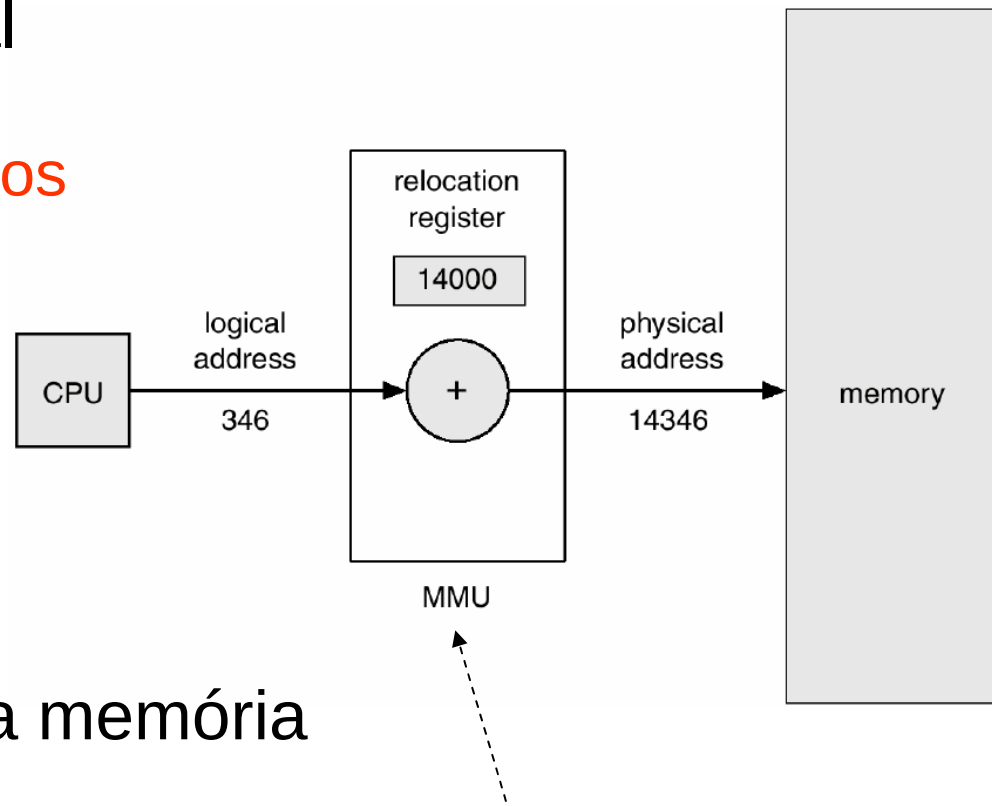


Endereços lógico e físico e a MMU

- **Endereço lógico:**
gerado pela CPU, também
chamado de end. virtual

MMU:
unidade de hardware que
mapeia end. lógico em físico

Ambos endereços são iguais nos
mapeamentos realizados na
compilação e na carga.
Só diferem nos
mapeamentos na execução.



- **Endereço físico:**
endereço real visto pela memória

MMU: Memory Management Unit

Carga dinâmica (dynamic loading)

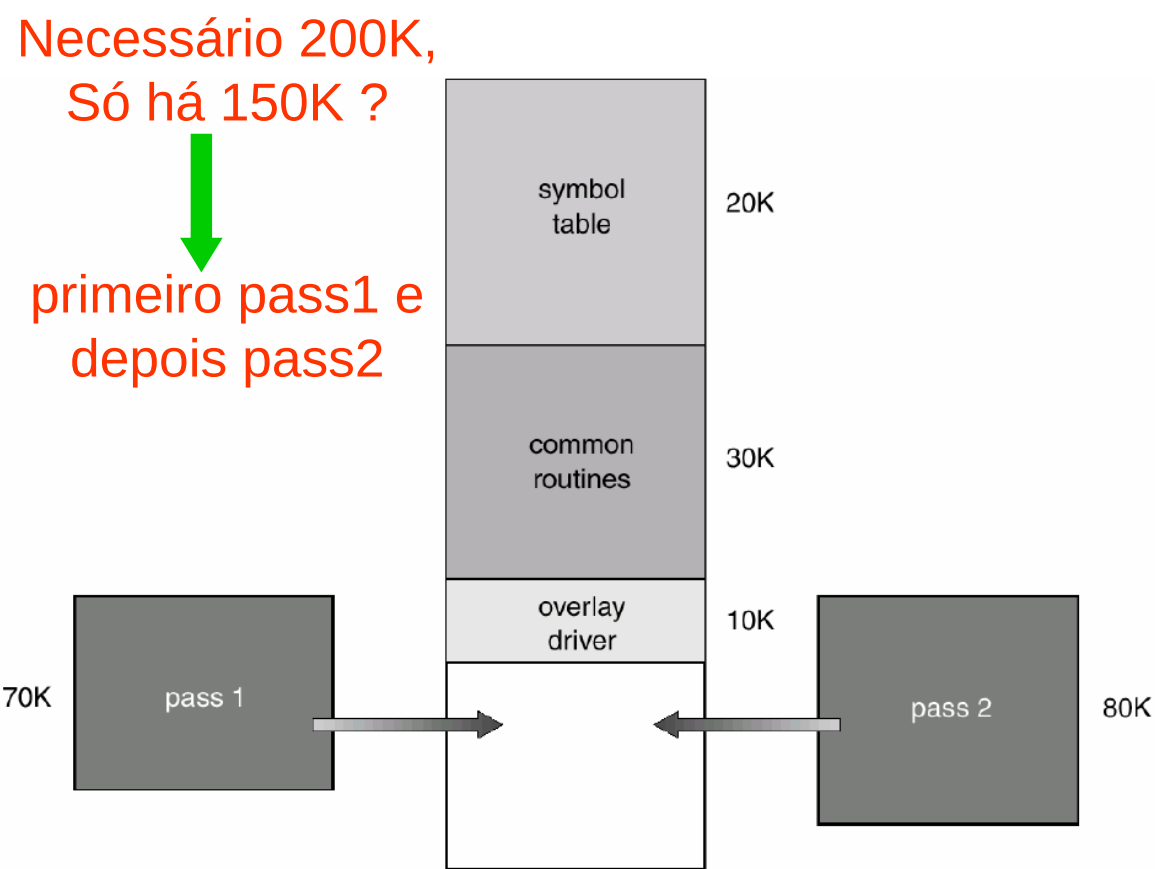
- Rotinas em programas somente são carregadas na memória quando solicitadas
- Provê melhor utilização de memória, pois rotinas que não são utilizadas não necessitam ser carregadas nunca
- Nenhum suporte especial do sistema operacional é necessário.
- Os usuários são responsáveis por desenvolver programas para aproveitar tais recursos

Ligamento dinâmico (dynamic linking)

- Ao invés da carga, o ligamento é adiado até o momento de execução
- Muito útil com bibliotecas do sistema, para que os programas utilizem uma única biblioteca
- Um pequeno pedaço de código, denominado **stub**, é usado para localizar a rotina própria de biblioteca residente em memória
- O **stub** substitui-se com o endereço da rotina e a executa
- O sistema operacional tem que verificar se a rotina está no endereço de memória do processo
- Este método é também chamado: **shared libraries**
 - No windows → **DLLs**

A Técnica Overlay

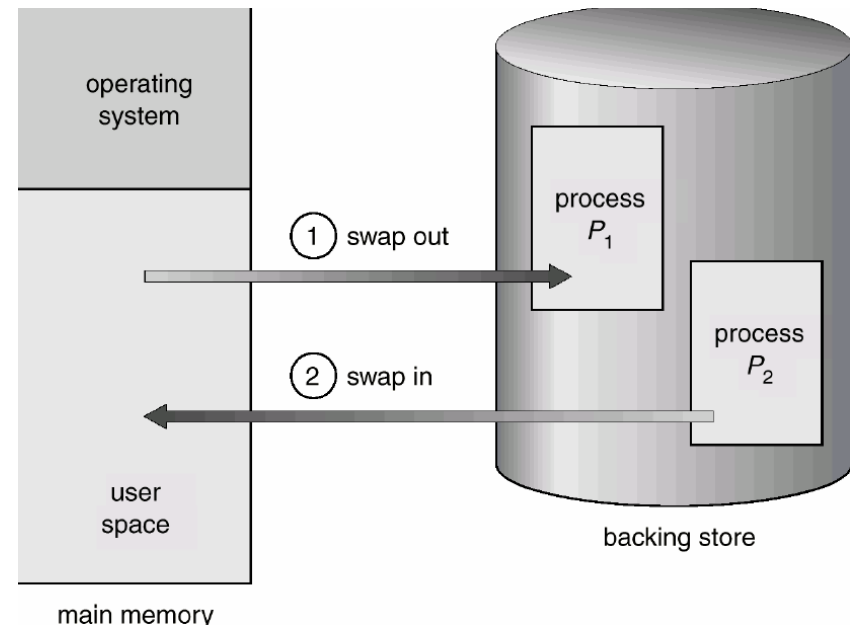
- A idéia é manter em memória somente o que for necessário o tempo todo



- Necessário qdo o processo é maior que a qtidade de memória a ele alocada
- Dispensa suporte especial do SO, mas a programação é complexa
- Outras técnicas são mais usadas !!!

A Técnica Swapping

- Processos podem ser retirados da memória para o disco (swap out) e vice-versa (swap in)
- Prioridade pode ser usada na escolha de qual processo a ser retirado (roll out, roll in)
- Tempo de transferência é significativo
- Tempo do escalonador (quantum) deve ser ajustado adequadamente



Alocação de memória contígua

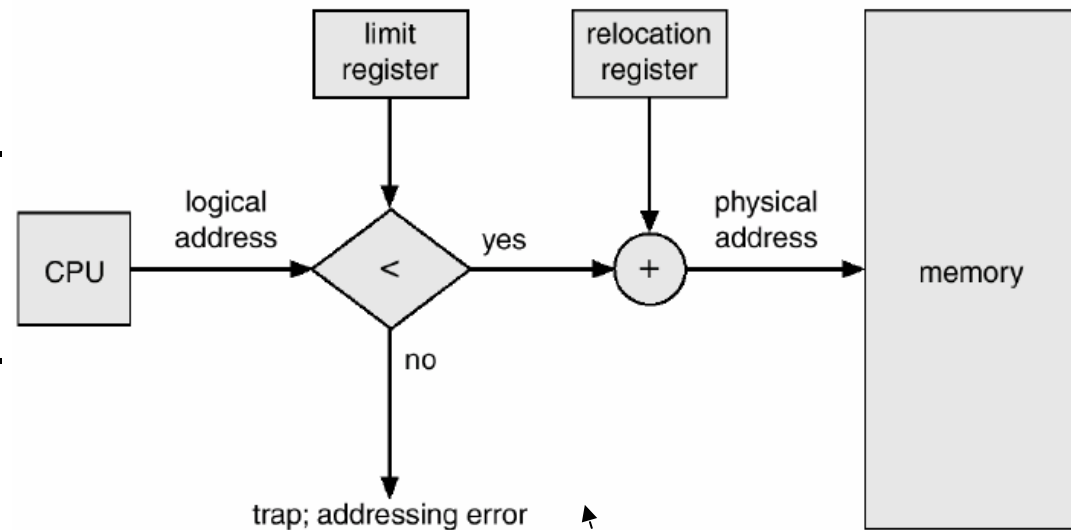
- Cada processo é carregado em uma única partição
- Registradores de relocação e limite asseguram proteção de memória para cada processo

- **Registrador de relocação**

→ possui o menor end. físico

- **Registrador de limite**

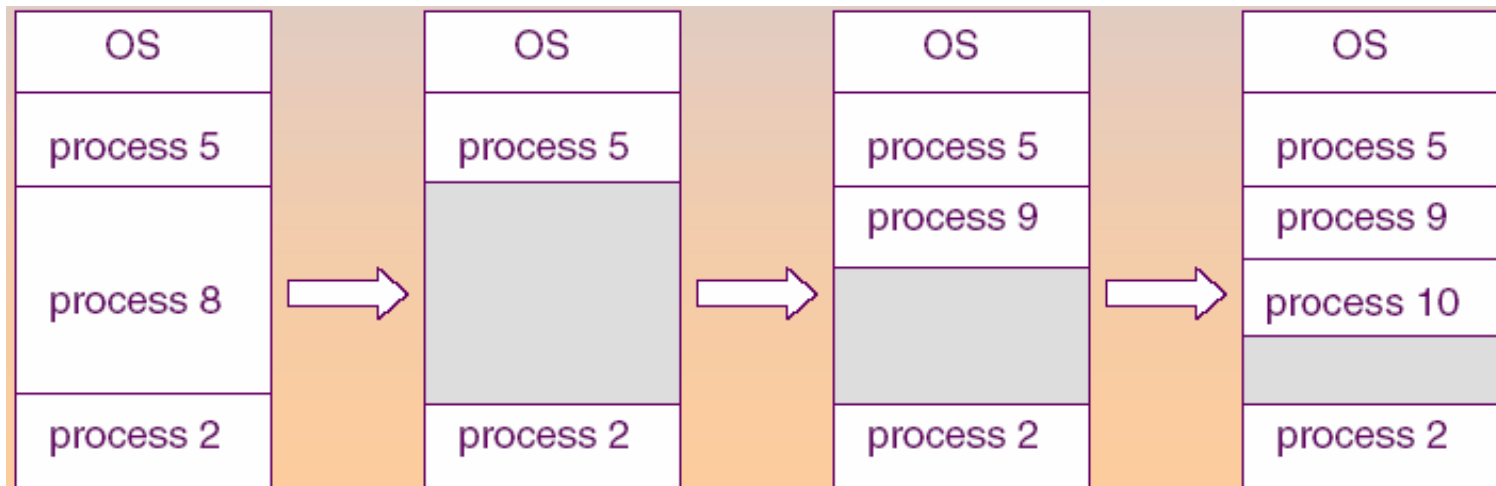
→ define faixa de end. lógicos



operações realizadas
pela MMU

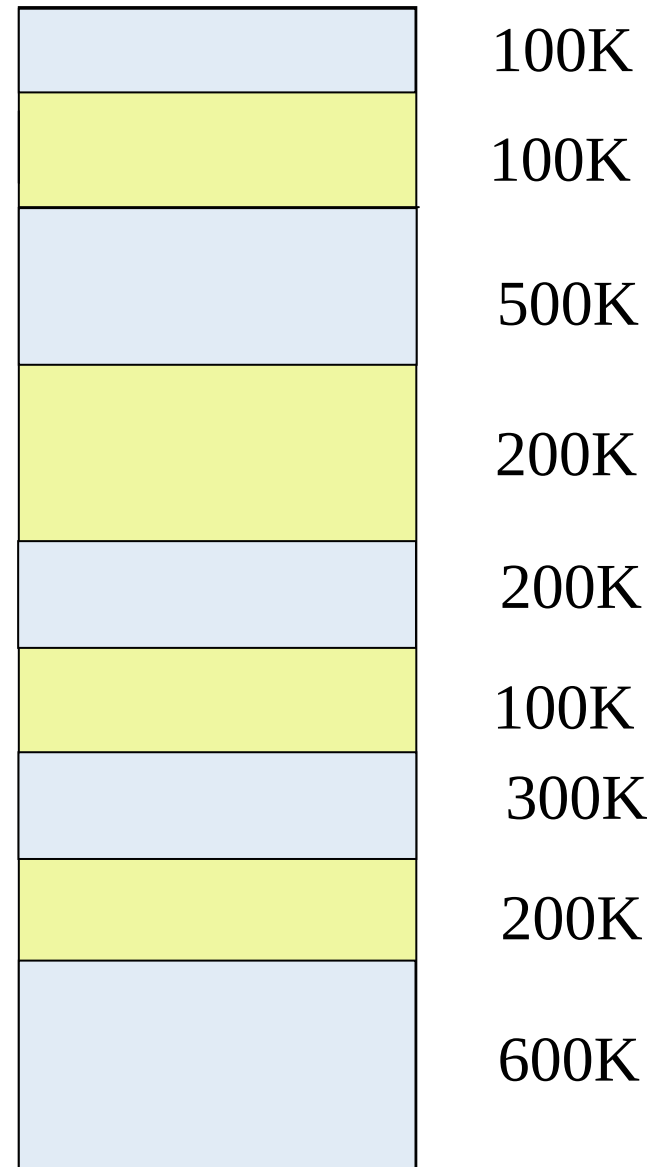
Alocação de memória contígua (cont.)

- Memória torna-se fragmentada
- Novos processos são carregados nas lacunas existentes
- SO mantém info sobre: partições ocupadas e livres
- Algoritmos utilizados para alocação de partições
 - **First-fit** → primeira lacuna suficiente encontrada
 - **Best-fit** → menor lacuna suficiente encontrada
 - **Worst-fit** → maior lacuna encontrada



Exercício

- Dado um esquema de memória com partições de: 100, 500, 200, 300, 600 KB (nesta ordem), pergunta-se:
- Como os algoritmos **first fit**, **best fit** e **worst fit** alocariam processos de tamanho 212, 417, 112 e 426 KB (nesta ordem)?
- Qual algoritmo usa a memória de forma mais eficiente?



Fragmentação

Dois tipos de fragmentação podem ocorrer

- **Fragmentação externa**

- Espaço de memória total é suficiente para alocar um processo, mas não é contíguo

- **Fragmentação interna**

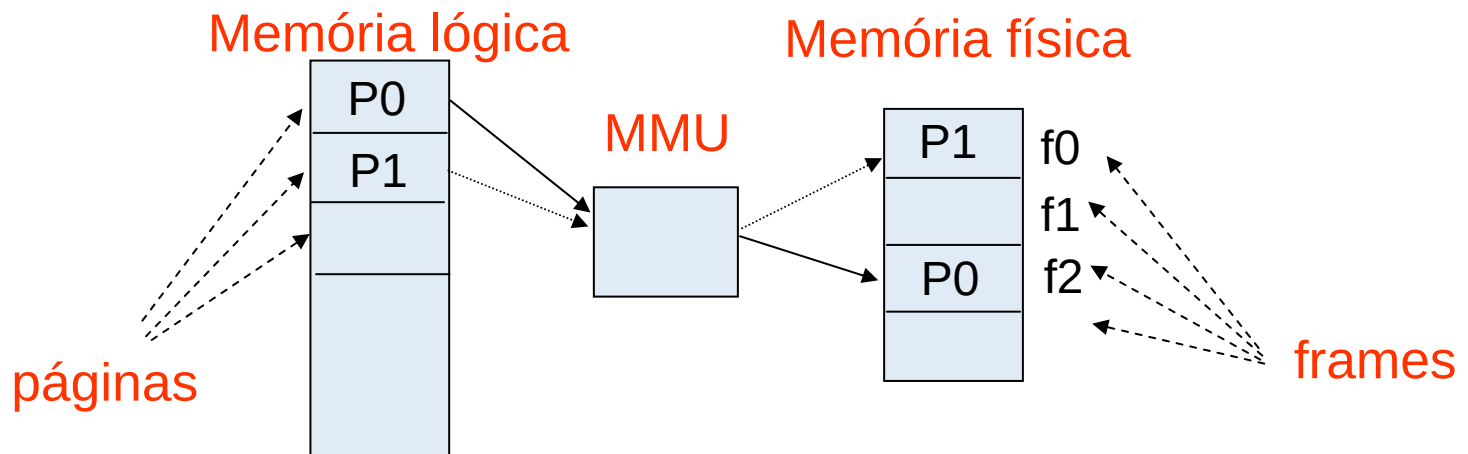
- Partição alocada é um pouco maior do que o processo carregado

- **Fragmentação externa reduzida por compactação**

- Reorganizar toda a memória de modo a juntar todas as lacunas em um único bloco
- Compactação só é possível se a relocação é dinâmica, e feita em tempo de execução

PAGINAÇÃO

- O endereçamento lógico pode ser não contíguo
- **Memória lógica** → dividida em blocos de mesmo tamanho chamados de **páginas**
- **Memória física** → dividida em blocos, de tamanho iguais às páginas, denominados de **frames**
- Um programa cujo tamanho é de n páginas, precisa de n frames livres para ser executado
- Uma **tabela de páginas** é necessário

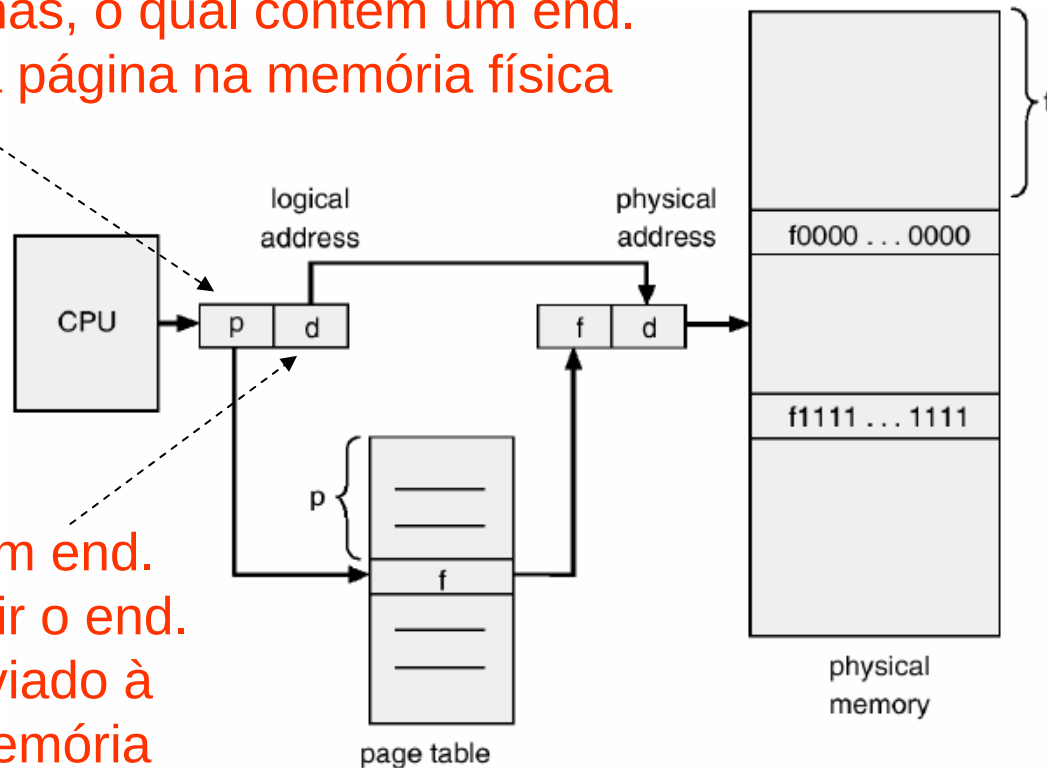


Esquema de translação de endereços

- O endereço gerado pela CPU divide-se em:
 - Número de página (p)
 - Deslocamento dentro da página (d)

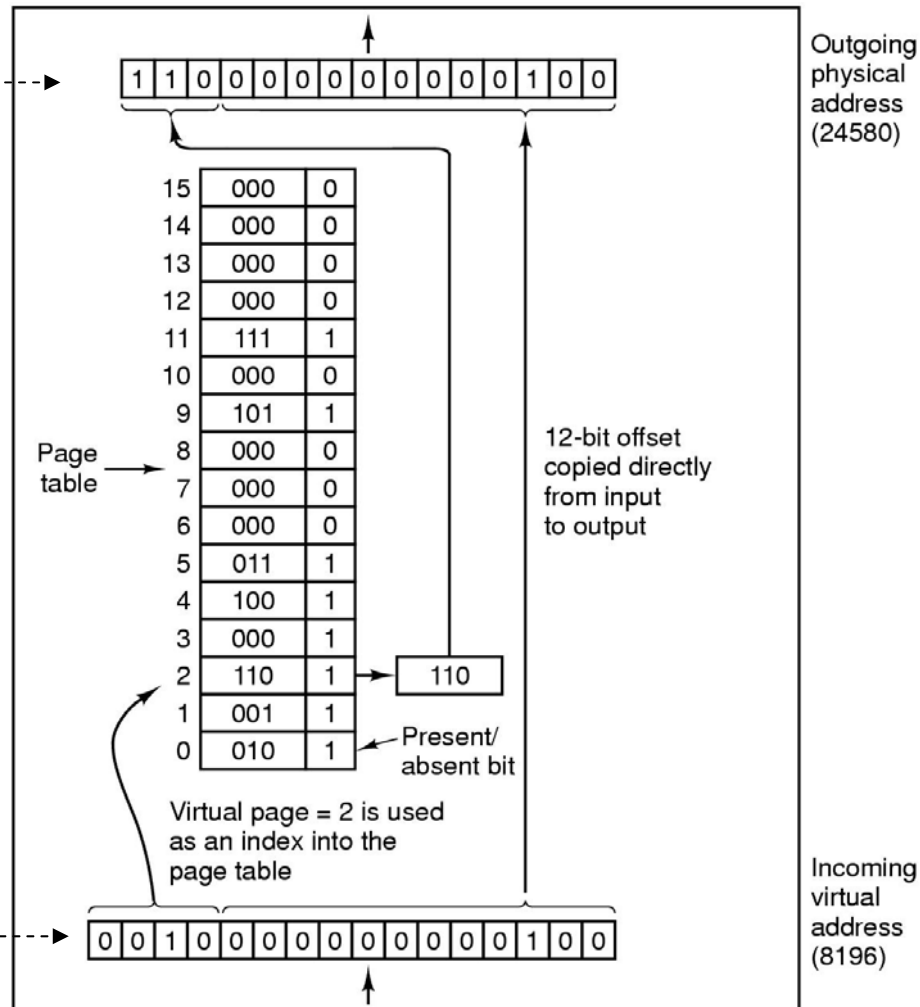
Usado como índice dentro de uma tabela de páginas, o qual contém um end. base de cada página na memória física

Combinado com end. base para definir o end. físico a ser enviado à unidade de memória



Exemplo de translação de endereços

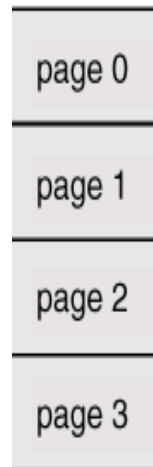
End. Físico
24580



End. Lógico
8196

Exemplo de paginação

Tabela de páginas
guia o mapeamento
de endereços

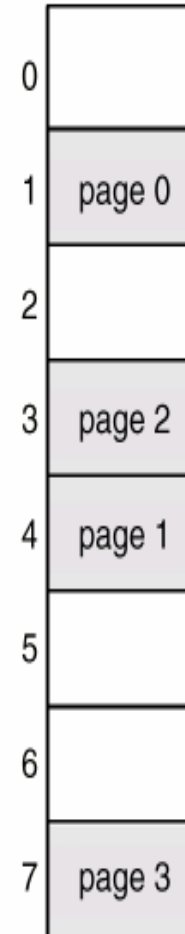


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

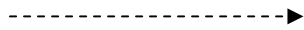


physical
memory

Página e frame têm
o mesmo tamanho

Exemplo de paginação (cont.)

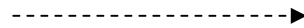
Página 0



0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

Página 3



0	5
1	6
2	1
3	2

page table

Tamanho da página: 4 bytes

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

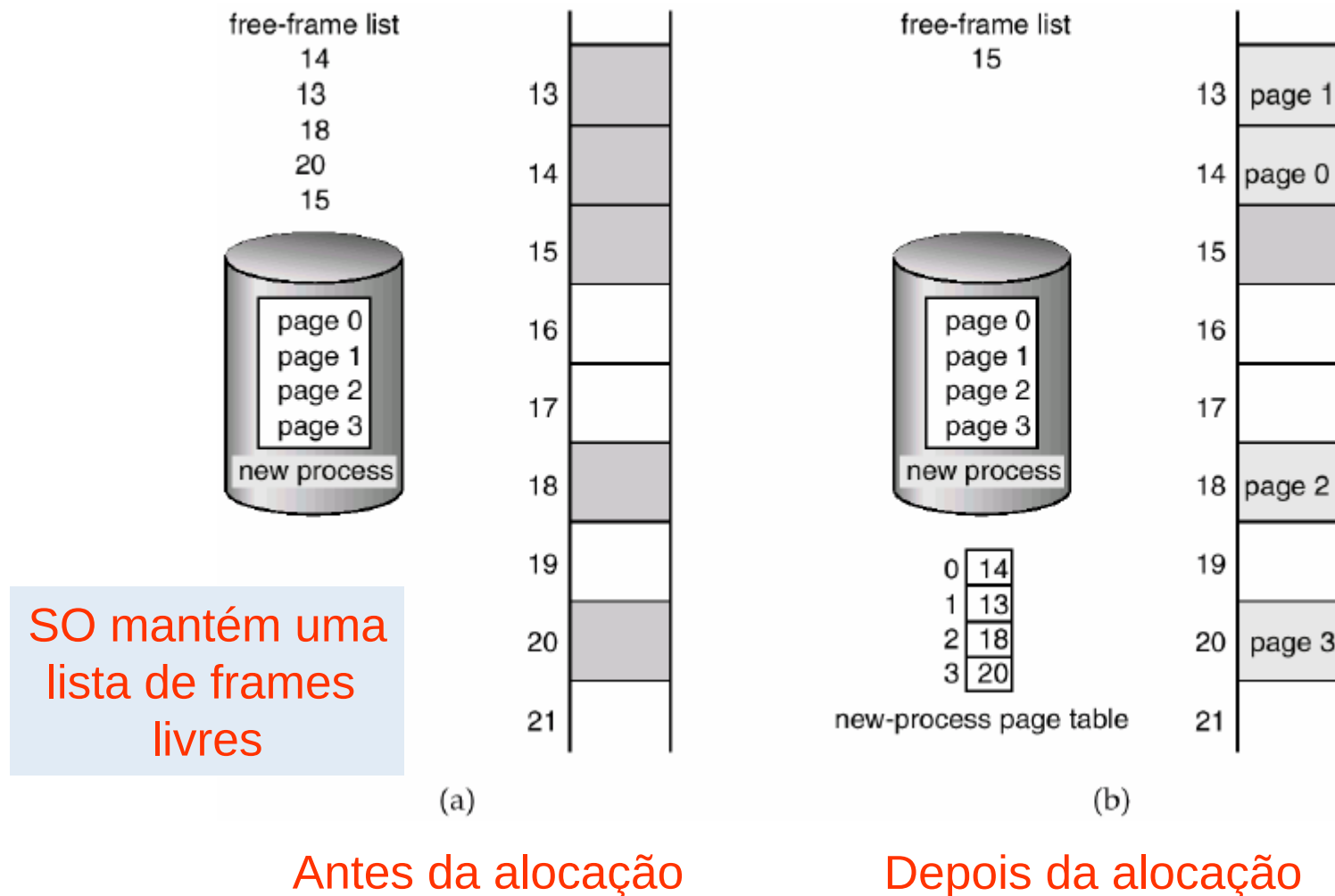
Frame 0

Frame 1

Frame 7

physical memory

Alocação de frames livres

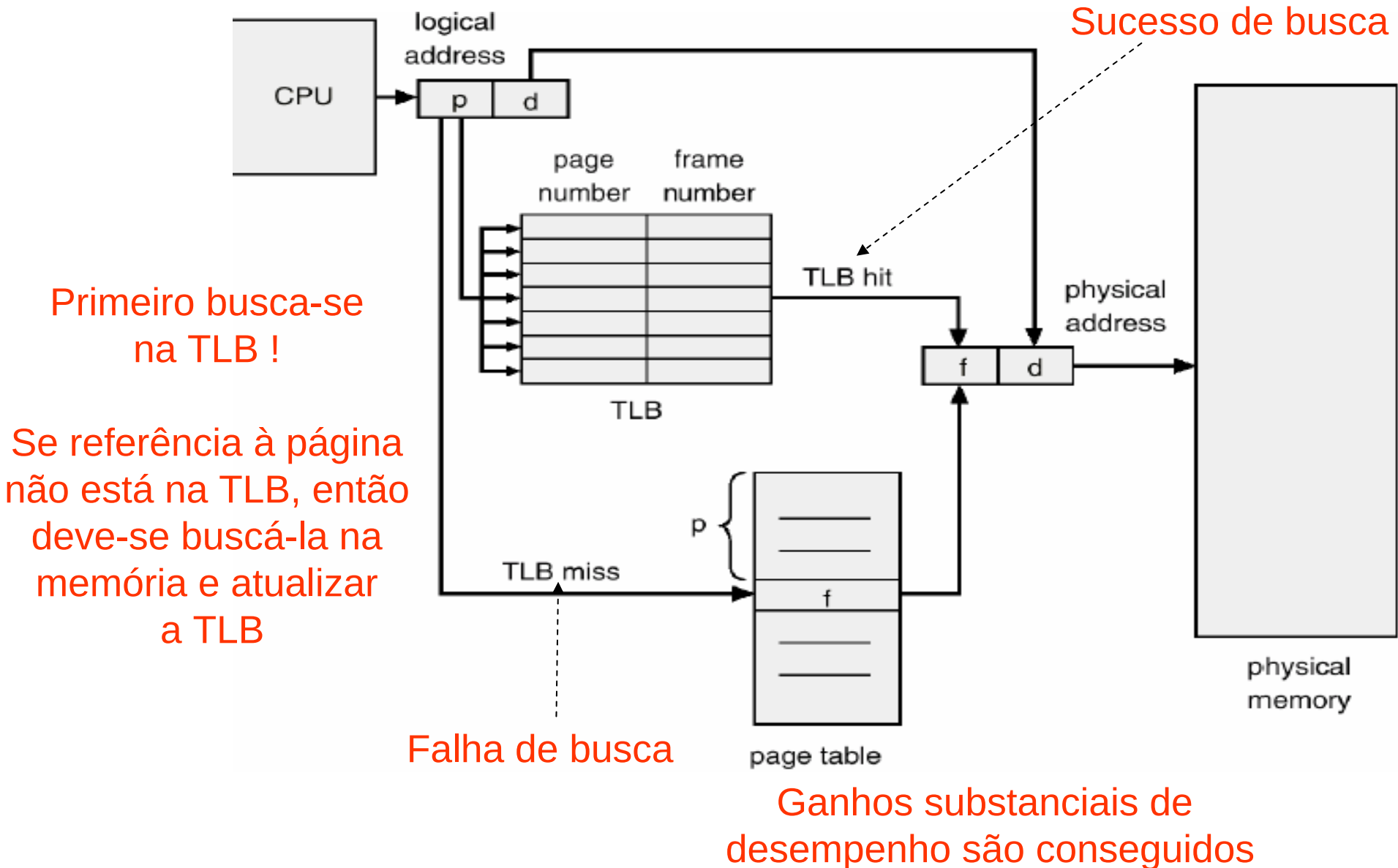


Implementação da tabelas de páginas

Duas possibilidades

- Tabela na memória principal
 - Dois acessos à memória, um para ler a tabela e outro para buscar a instrução/dado
- Tabela em memória cache the busca rápida
 - Essa memórias são chamadas **TLBs** (Translation Look-aside Buffers)

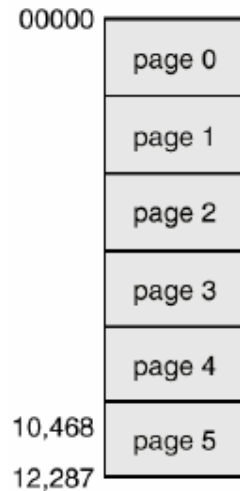
Tabelas de páginas com as TLBs



Proteção de memória

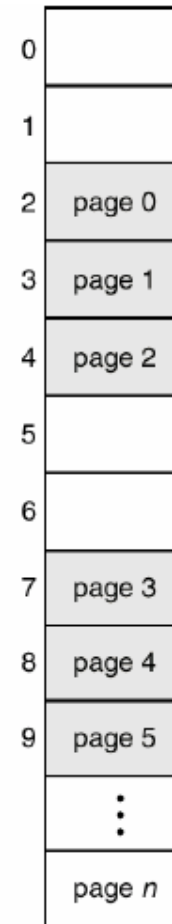
- É conseguida associando-se um bit de proteção, o bit the validade (v), a cada frame.

Raramente um processo usa toda a sua faixa de endereçamento



frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

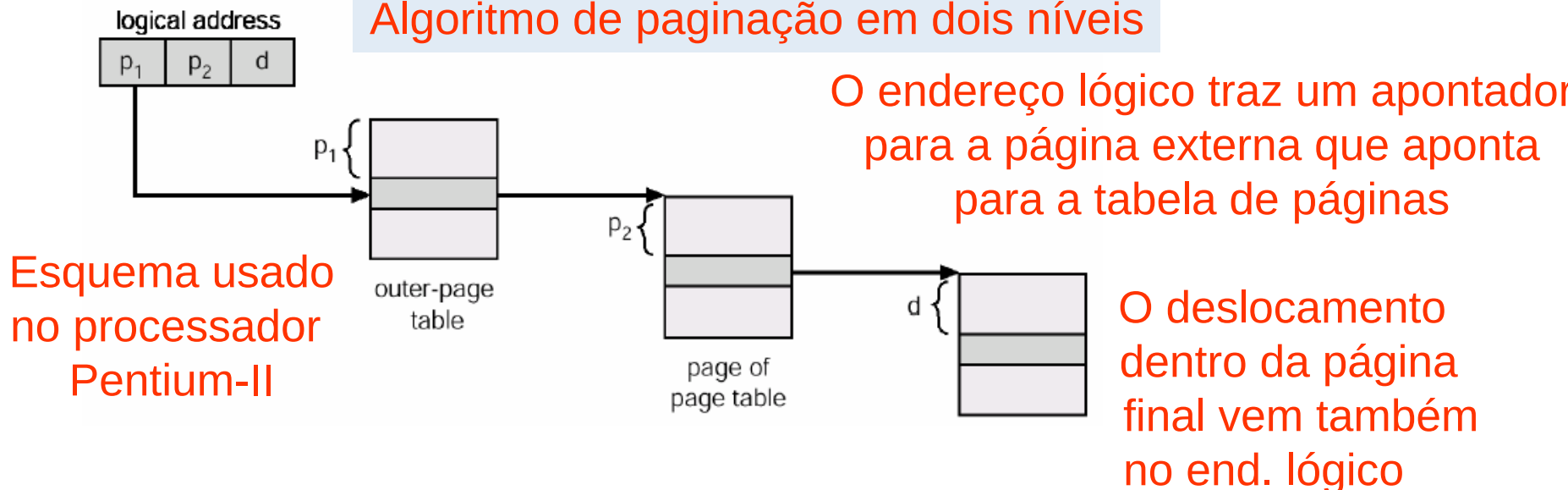


Neste exemplo, o processo não possui valor nenhum nas páginas 6 e 7. Tentativa de acesso a elas, gera um erro !

PAGINAÇÃO HIERÁRQUICA

- Hardwares atuais suportam endereçamentos lógicos bastante extensos (2^{32} a 2^{64} bits)
- **Resultado** → tabelas de páginas extremamente longas
 - Ex.: endereçamento de 32 bits e páginas de 4K
→ tabelas de até 1 milhão de registros
- Solução → dividir a tabela em sub-tabelas
- Sub-tabelas podem ser espalhadas pela memória

Algoritmo de paginação em dois níveis



Paginação em dois níveis: ilustração

- Este esquema é apropriado para até 32 bits de endereçamento
- Para sistemas com mais de 32 bits, a tabela externa torna-se muito longa
- Outras técnicas devem ser usadas
 - Tabelas de páginas Hash
 - Tabelas de páginas invertidas

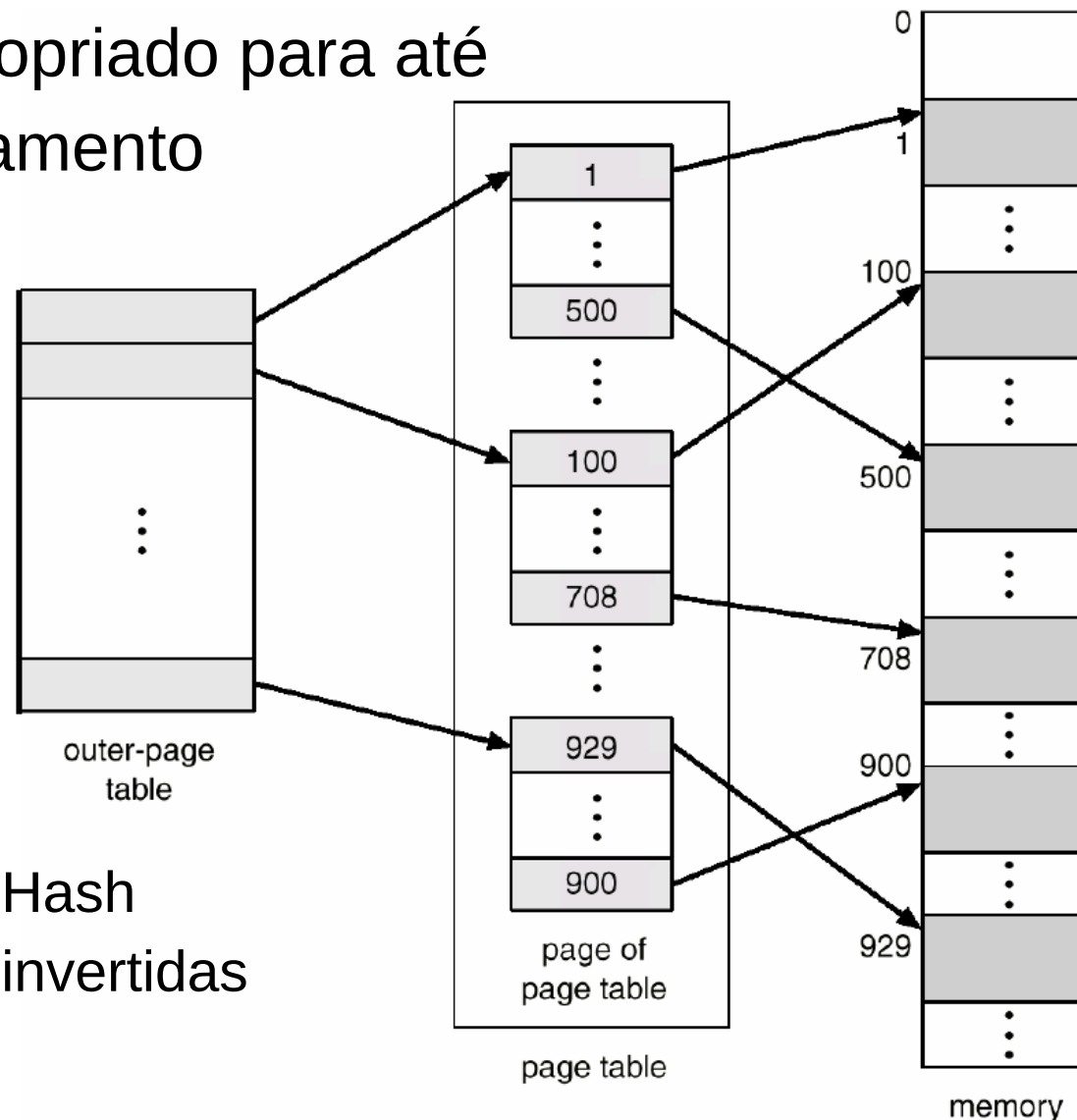
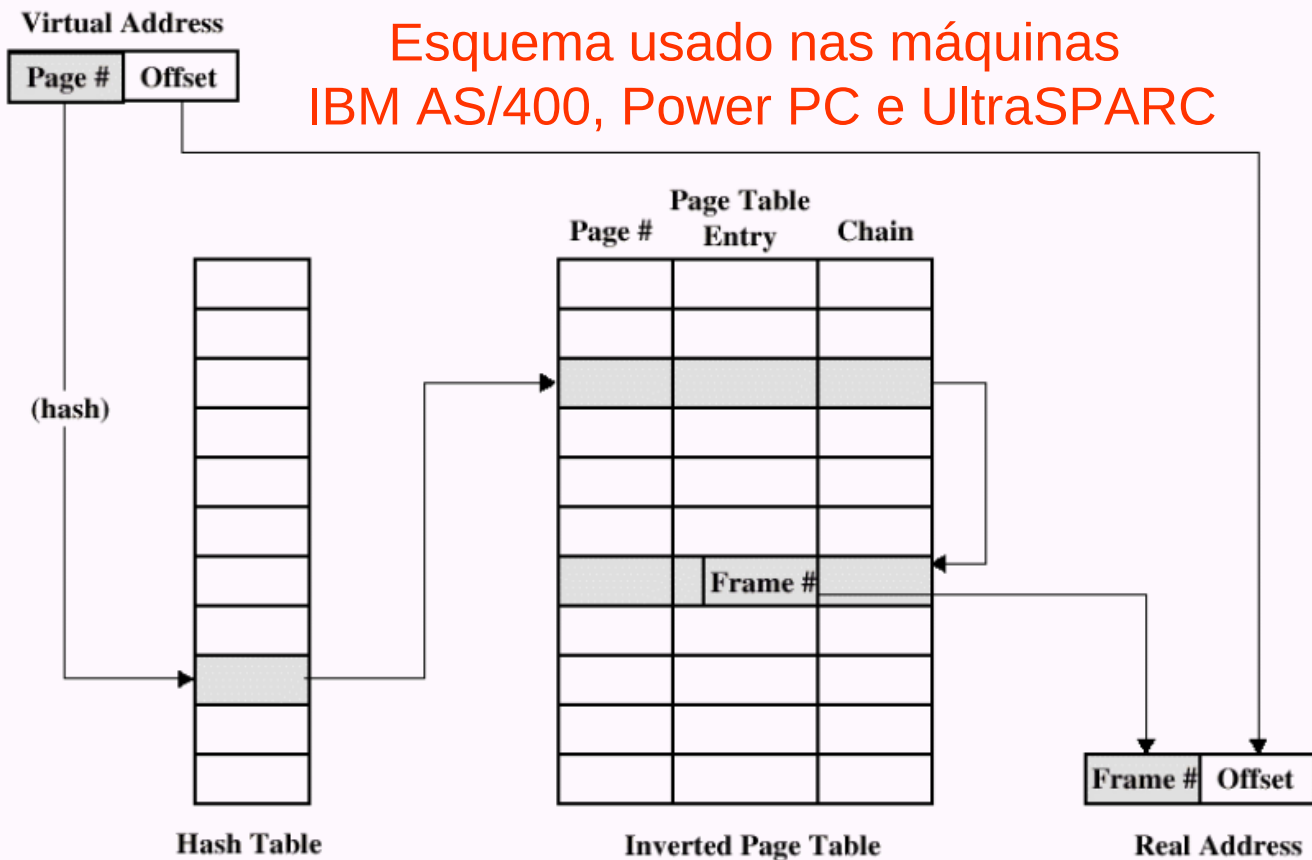


Tabela de páginas Invertida

- Diminui tamanho da tabela de páginas → necessário em sistemas de 64 bits
- Tabela Hash possibilita procura mais rápida

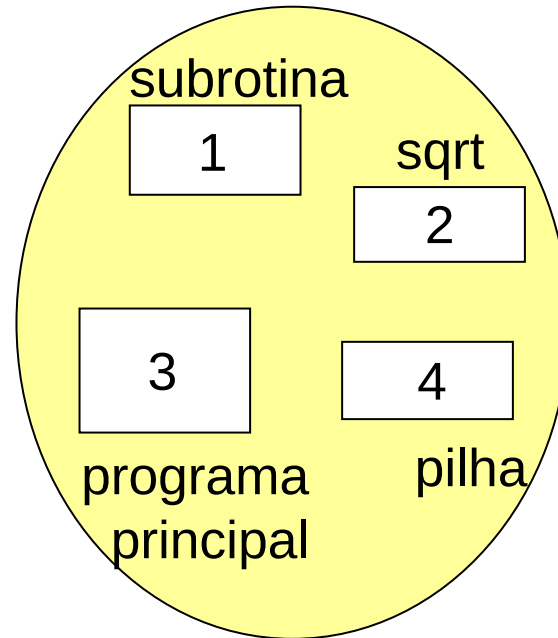


A qtd de registros da tabela de páginas é limitada pela qtd de frames (reais) e não pelo número de páginas virtuais

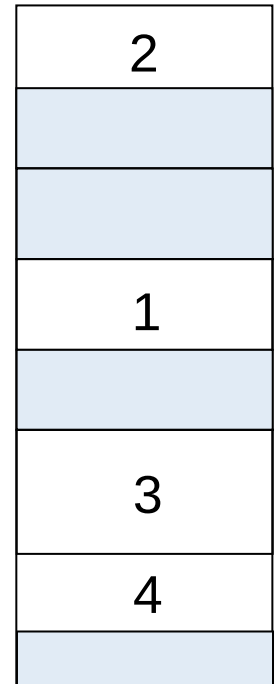
Referências Hash repetidas são manuseados pela técnica Chain

SEGMENTAÇÃO

- Possibilita gerenciar a memória com base na visão do programador → facilita a programação, modularização
- Um programa compõe-se de vários segmentos
 - Programa principal
 - Subrotina
 - Função
 - Pilha
 - Variáveis
 - etc



Programa de usuário

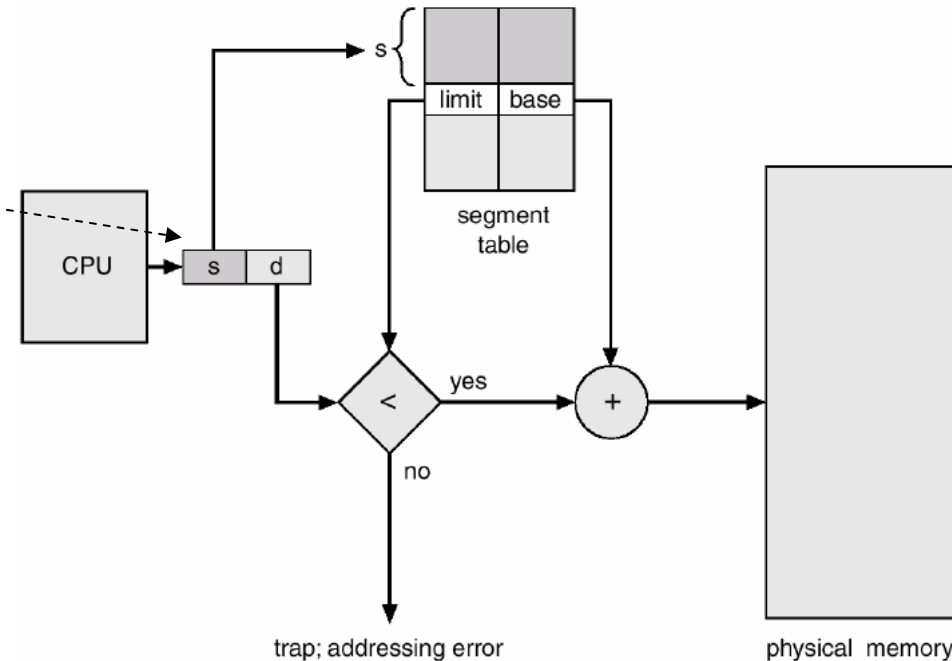


Memória física

Fragmentação externa
está presente !

Implementação da segmentação

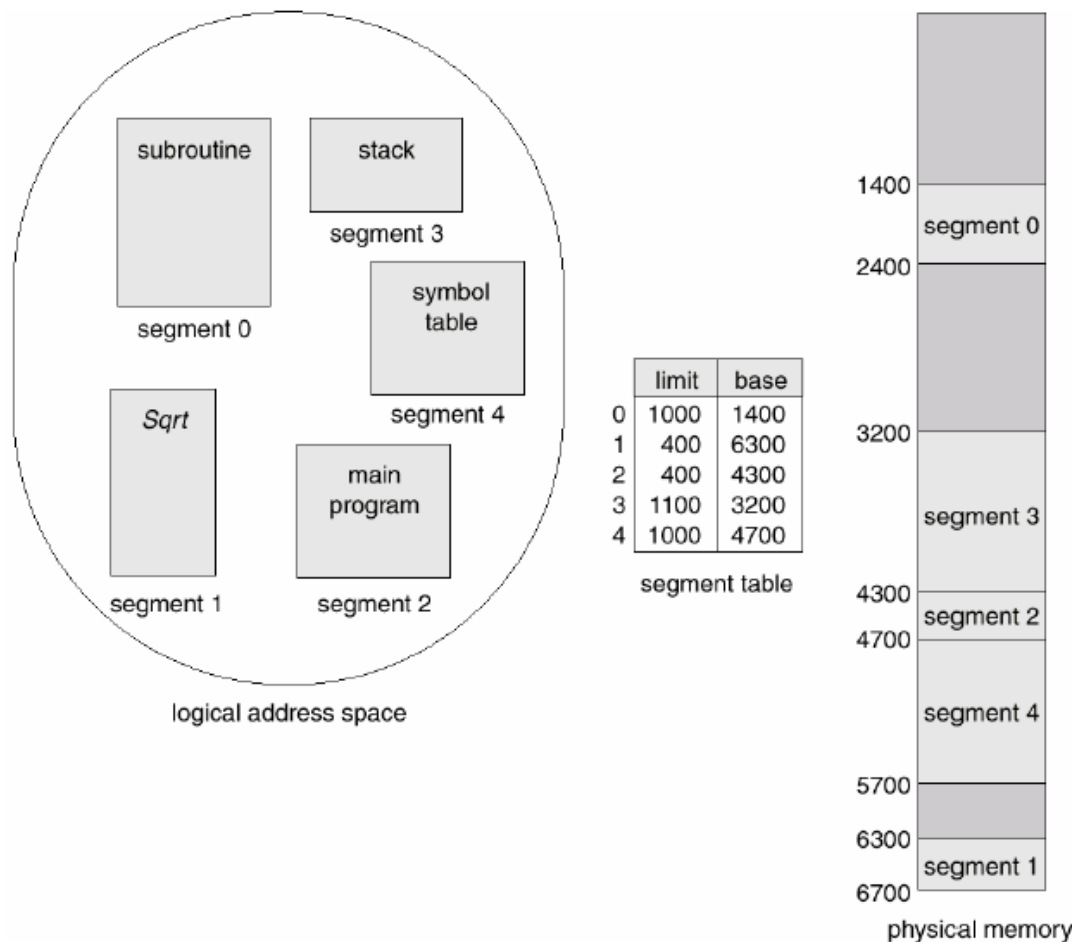
- Como na paginação, uma tabela é utilizada para o mapeamento: end. lógico $\rightarrow$ end. físico
- Cada endereço lógico é formado de:
<Nr. de seg., desl>
- Tabela de segmentos contém em cada registro:
base: end. inicial onde o segmento está na memória física
limit: tamanho do segmento



O mapeamento é realizado em Hardware !

Exemplo de segmentação

- Cada endereço é mapeado para dentro de seu segmento correspondente conforme a tabela de segs.



Há mais dois registradores para a segmentação

STBR (Segment-Table Base Register)

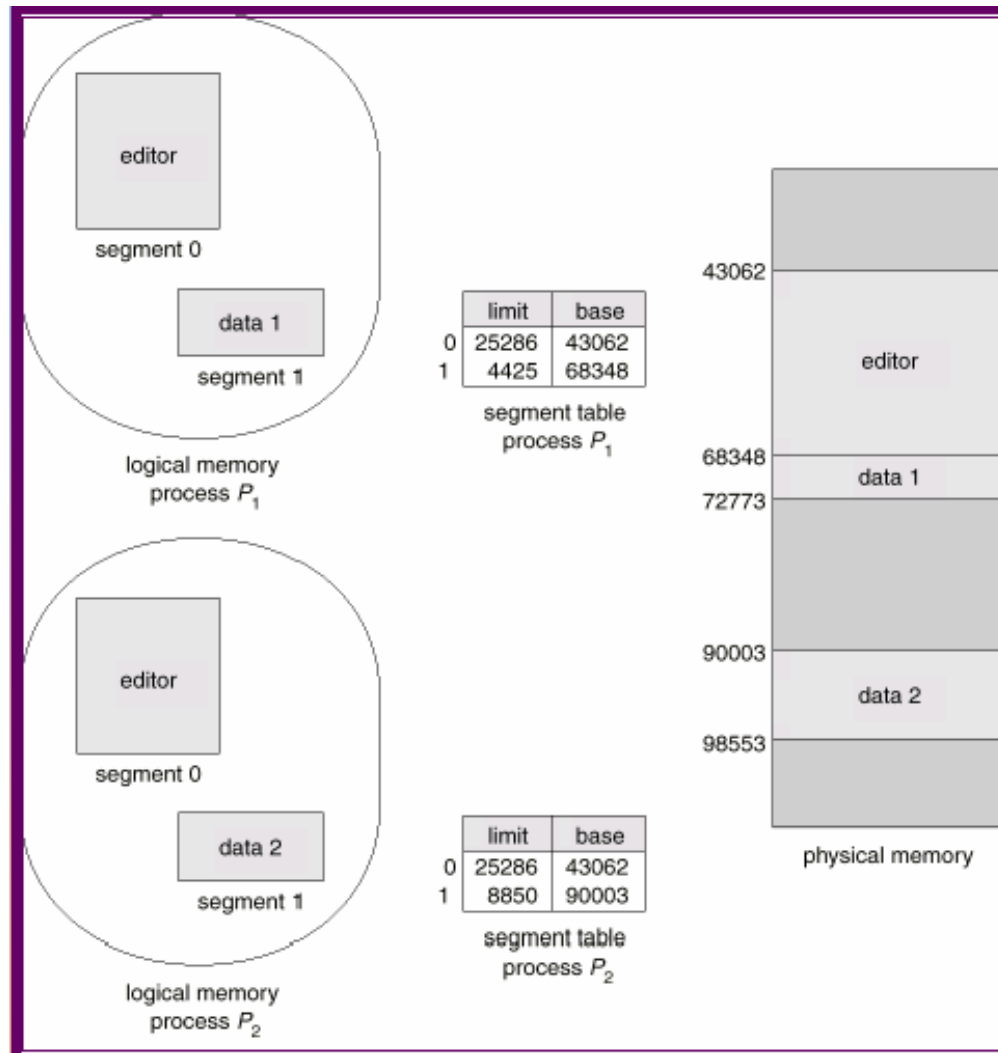
→ aponta para a tabela de segs. na memória

STLR (Segment-Table Length Register)

→ Indica a qtd de segs. usada por um programa

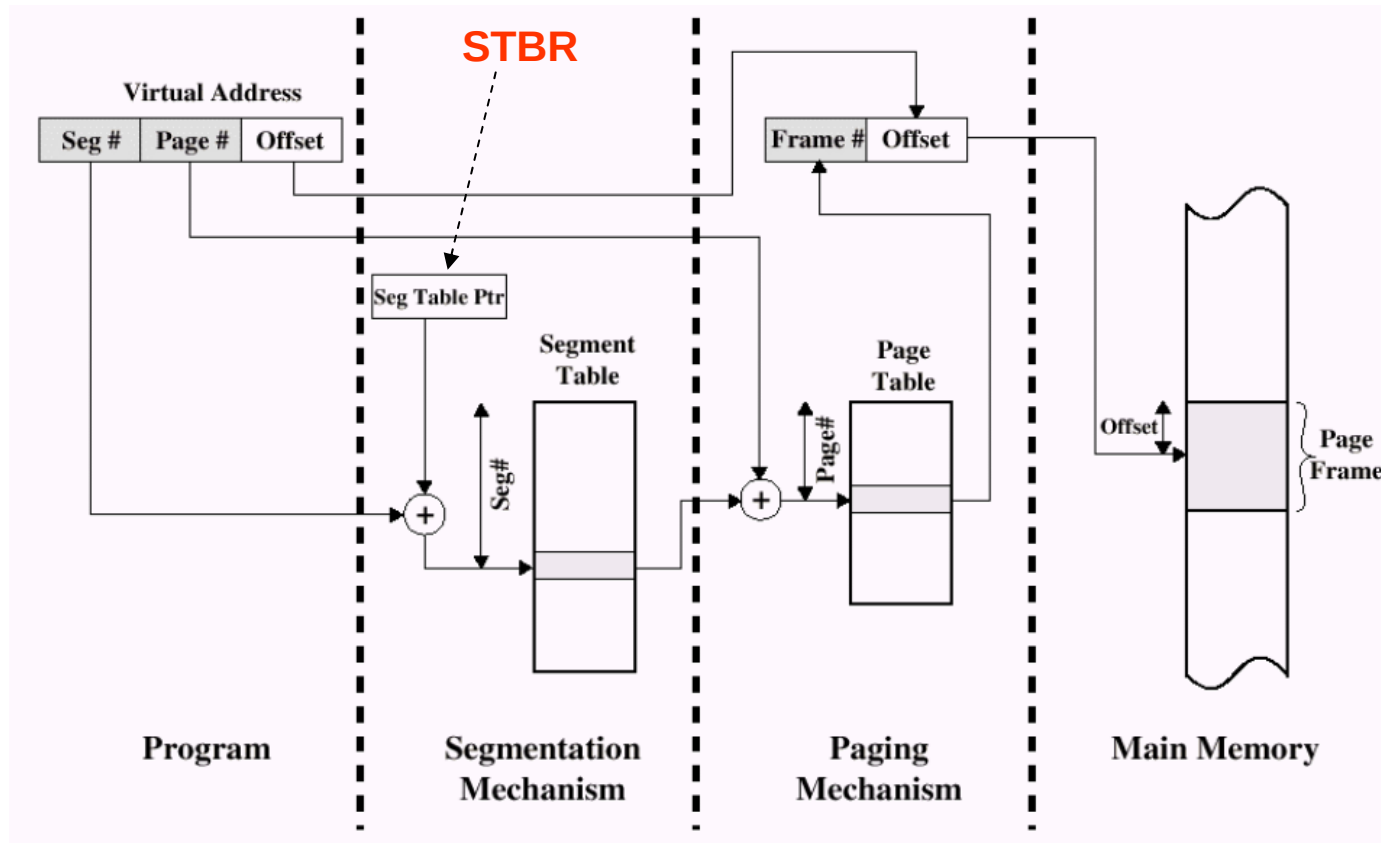
Segmentação com Compartilhamento

- Segmentos podem ser compartilhados
- Segmentos podem ser do tipo: **somente leitura** ou então **somente execução**
- O Hardware detecta erros de acesso
- Ex.: **editor de texto**
somente um segmento para o código, mas vários segmentos para dados de cada usuário
- Compartilham. parcial: subrotina, variáveis, ...



Segmentação com paginação

- Para minimizar o problema da fragmentação externa, os segmentos podem ser paginados



combina-se as vantagens da segmentação e paginação

Programador lida com a segmentação, e o sistema (hardware) gerencia a paginação

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004

Sistemas Operacionais

Memória Virtual

Conteúdo

- Memória Virtual
- Paginação sob demanda
- Desempenho da paginação sob demanda
- Algoritmos de substituição de páginas
- Alocação de Frames
- Thrashing
- O modelo working set

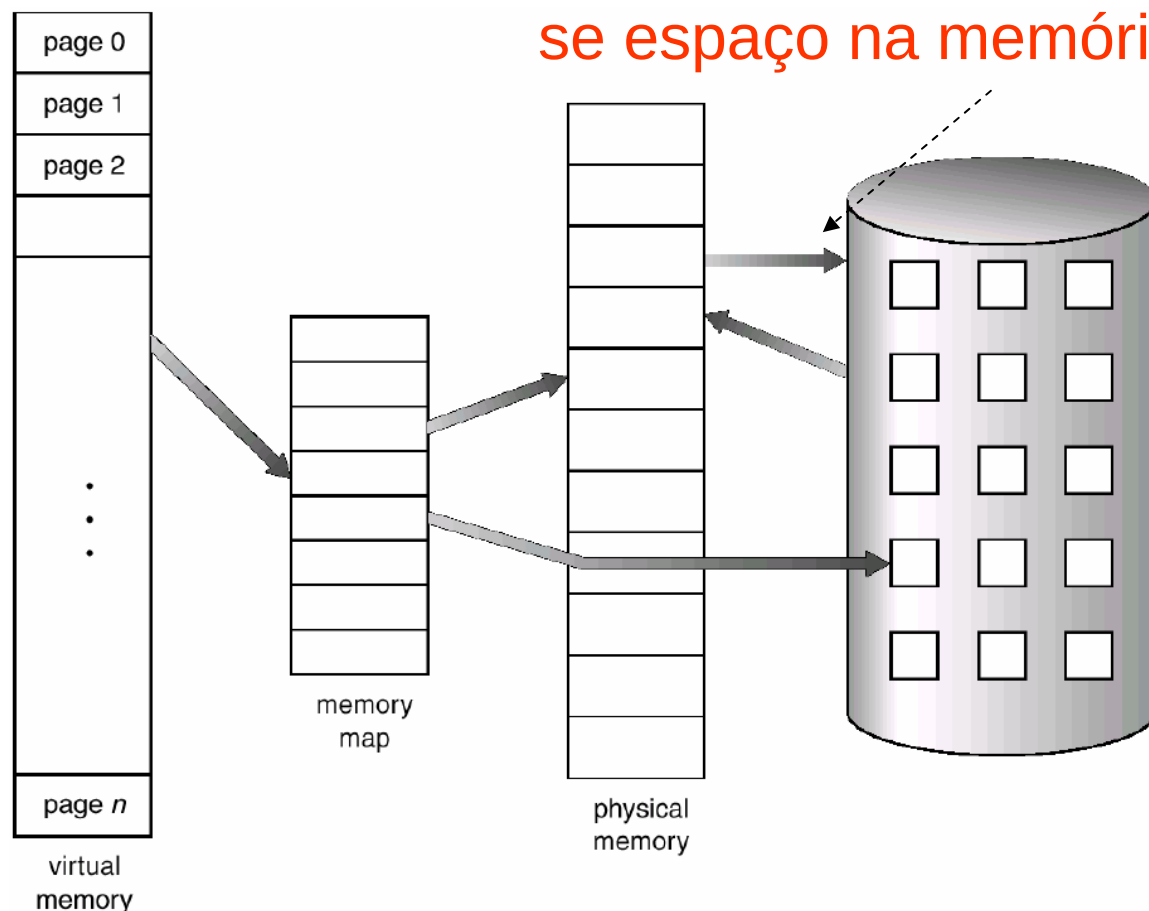
MEMÓRIA VIRTUAL

- Em geral, somente parte do programa precisa estar na memória principal para ser executado
- Permite → faixa de end. Lógico >> faixa end. físico
- End. lógicos são mapeados dinamicamente para end. físicos
- Endereços podem ser compartilhados por vários processos
- Criação de processos podem ser mais eficientes
- Mais processos podem ser mantidos na memória simultaneamente → melhor utilização da **CPU**
- Um processo pode ser maior do que a memória física → usuário não precisa se preocupar com isso

Memória virtual (cont.)

- Páginas que não estejam na memória física são trazidas da memória secundária

Páginas podem ser salvas no HD
se espaço na memória física estiver limitado

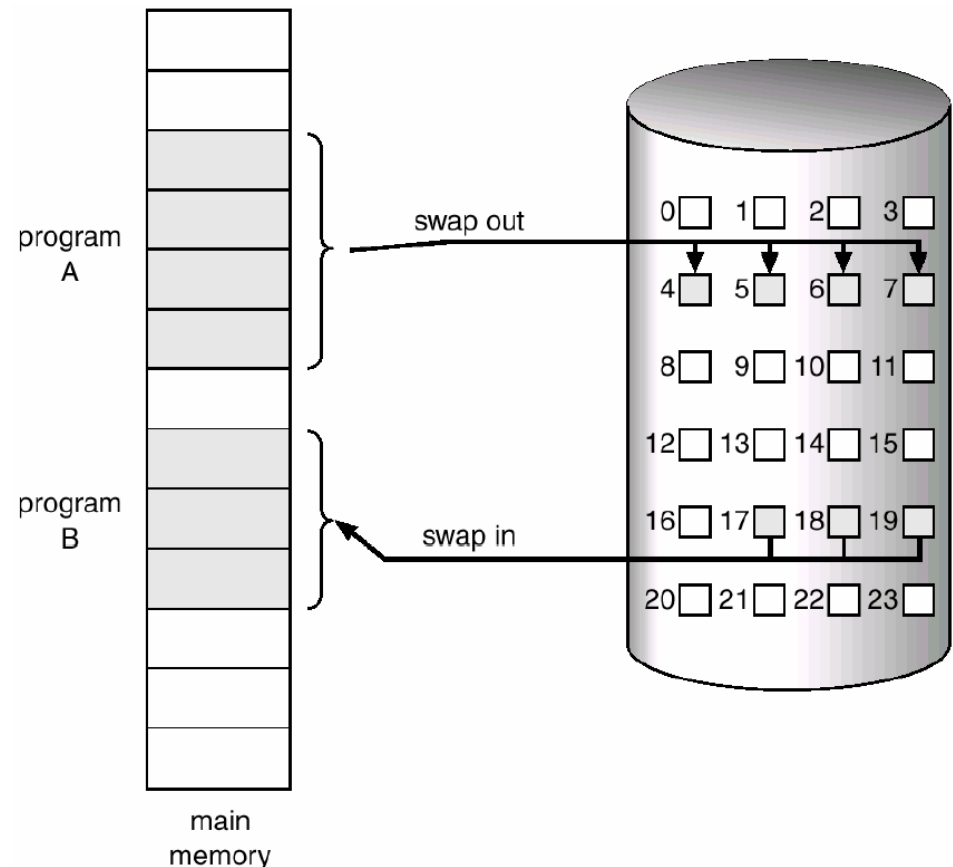


Desafios:

- 1- qdo trocar uma página?
- 2- qual página substituir?
- 3- qdo não é preciso salvar página sobrescrita no HD?
- 4- ...

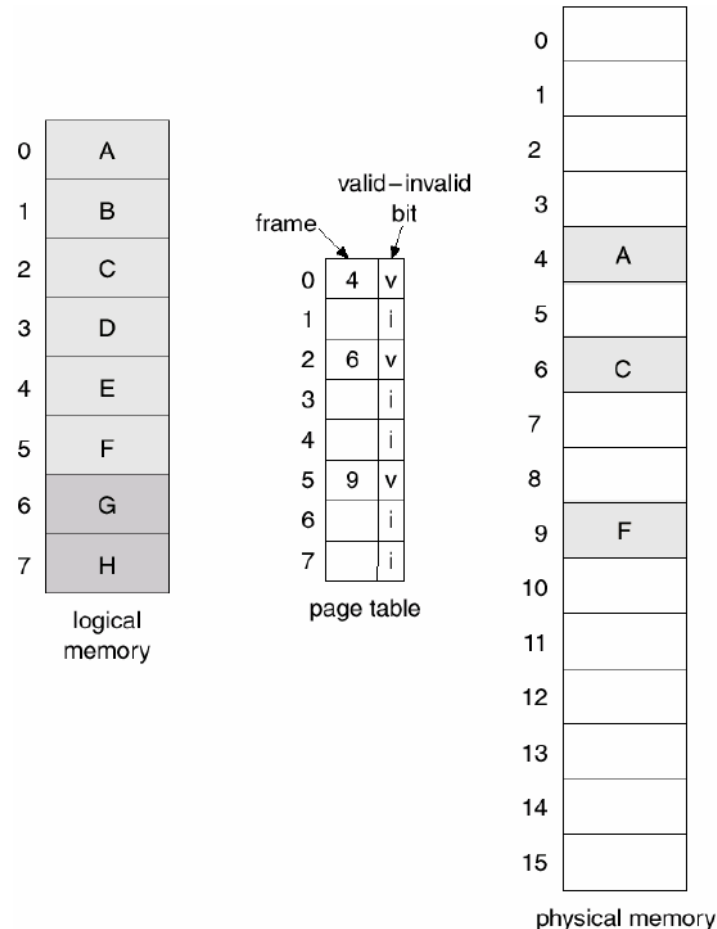
Paginação sob demanda

- Páginas só são trazidas para a memória principal (física) quando necessárias
- Páginas referenciadas:
 - Ausente em memória
→ carregar
 - Inválida
→ abortar por erro
- Referência a uma página ausente, gera uma *system call* do tipo ***page fault***



Paginação sob demanda (cont.)

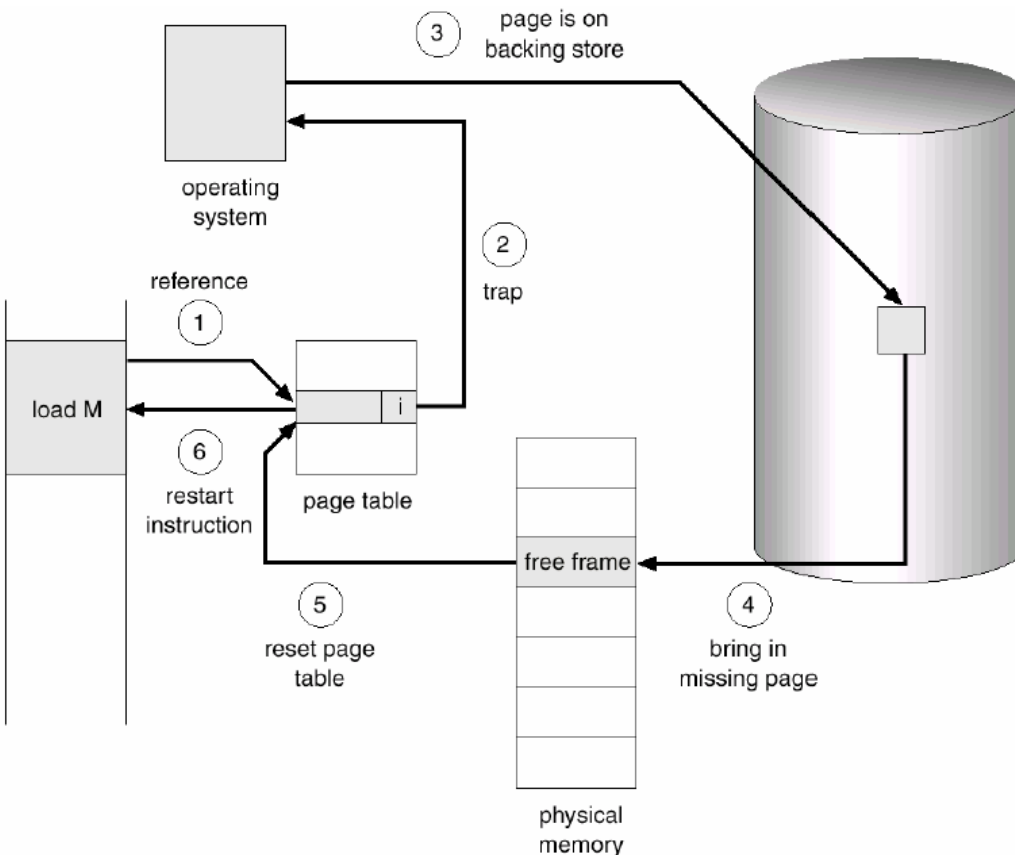
- Cada registro da tabela de páginas inclui o bit de validade “v”:
 - 1 (valid) → em memória, 0 (invalid) → no disco
- Referência a um registro com $v=0$ (invalid) gera uma **page fault**
- Inicialmente todos os registros são ajustados com $v=0$



Procedimento sob *page fault*

1- qdo uma página não é encontrada na memória principal ($v=0$), as seguintes tarefas ocorrem:

- 2- uma system call (trap) invoca o SO que determina se o endereço recebido é válido. Se não, aborta o processo
- 3- um frame livre é encontrado
- 4- página é trazida p/ o frame
- 5- tabelas são atualizadas, $v=1$
- 6- instrução que desencadeou esse procedimento é reiniciada



Desempenho da paginação sob demanda

- O impacto da paginação sob demanda no desempenho do sistema pode ser substancial
- Na ausência de **page faults**, o tempo de acesso efetivo de memória (TAEM) paginada é igual ao tempo de acesso à memória (TAM), o qual varia entre 100 e 200 ns
- Com uma probabilidade de **page faults** p ($0 < p < 1$), o TAEM fica:

$$\text{TAEM} = (1-p) \times \text{TAM} + p \times \text{TPF}$$

Tempo de
Page Fault

Fatores que influem o RFP:

- tratamento da interrupção de page fault
- leitura da página do disco
- reinício do processo

Desempenho da pag. sob demanda (cont.)

EXEMPLO

- Um HD típico tem os seguintes atrasos médios:
 - Latência: 8 ms, busca: 15 ms, transferência: 1 ms
- Somando-se aos tempos de tratamento da interrupção de **page fault** e reinício da instr, tem-se um total de cerca de 25 ms com **page fault** (TPF)
- Então, com TPF=25 ms e TAM=100 ns:
$$TAEM = (1-p) \times 100 \text{ ns} + p \times (25 \text{ ms}) = 100 + 24999900 \times p$$
- Para se ter menos de 10% de degradação, precisa-se:
$$110 > 100 + 25000000 \times p \rightarrow 10 > 25000000 \times p \rightarrow$$

$$\rightarrow p < 0.0000004$$

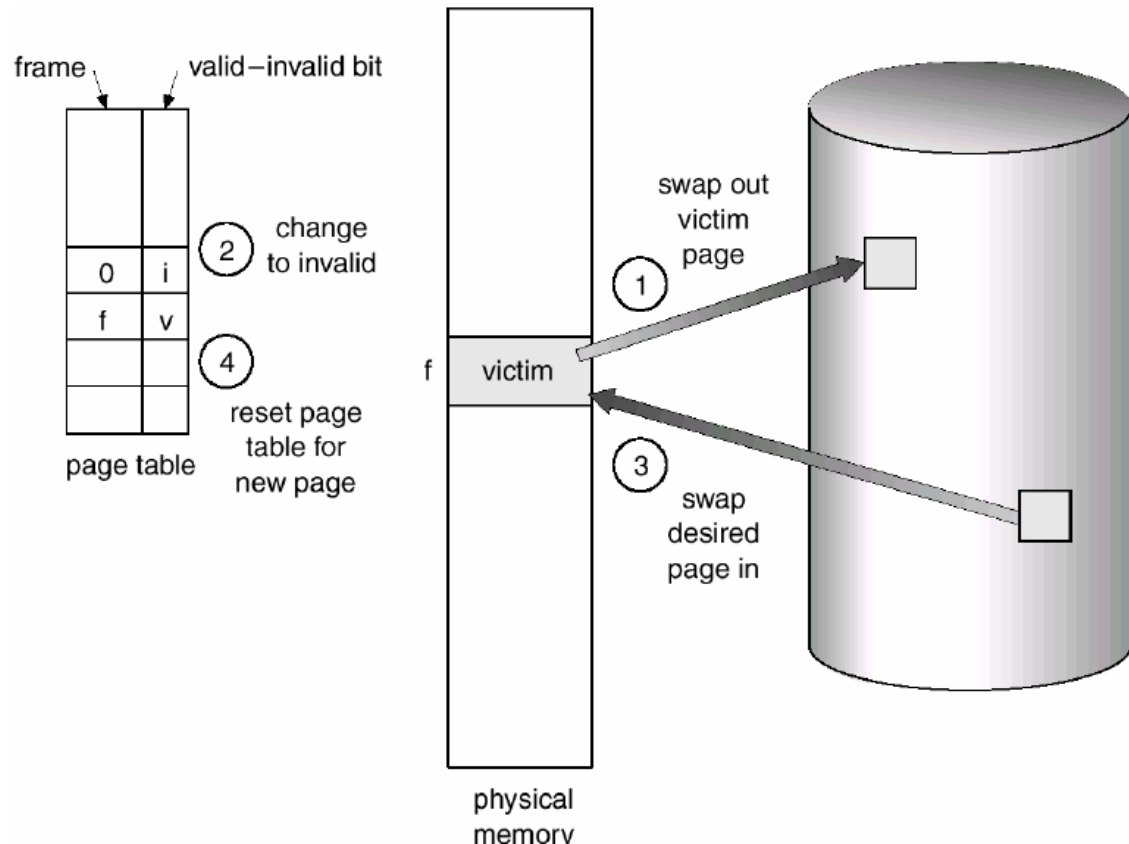
Menos de uma **page fault** dentre 2500000 acessos à memória !!!

Substituição de páginas

- Qdo ocorre **page fault** com necessidade de busca de uma página do disco e não há frames livres
 - Um frame vítima deve ser copiado para o disco para liberar espaço
- Assim, uma extensa memória virtual pode funcionar com uma memória física menor
- O mecanismo que lida com **page faults** deve incluir o algoritmo de substituição mostrado

modify (ou dirty) bit reduz
tempo de **page fault**

→ Páginas não alteradas,
não copiar p/ o disco



Algoritmos de substituição de páginas

- Objetivam a menor taxa de *page faults* possível
- Para avaliá-los, necessita-se de
 - uma cadeia de seqüência de referências à memória
 - conhecer a quantidade de frames disponíveis
- Na maioria dos exemplos que se seguem será utilizada a seguinte cadeia de seqüência:
 - 1 2 3 4 1 2 5 1 2 3 4 5
- Portanto, a página 1 é referenciada primeiramente, depois a página 2, e assim por diante

Algoritmo de substituição ideal (optimal)

- Usado como padrão de referência para outros algoritmos, pois um algoritmo perfeito não existe
- Com este algoritmo
 - Obtém-se a menor taxa de **page faults**
 - Nunca ocorrerá a anomalia de Belady (vista mais adiante)
- Idéia→ substitua a página que demorará mais a ser usada
- Exemplo de **4 frames**

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

1	2	3	4	5	5
-	1	2	3	3	4
-	-	1	2	2	2
-	-	-	1	1	1

6 page faults

- Algoritmo de substituição ideal é difícil de ser implementado porque requer conhecimento prévio da sequência de **page faults**

Algoritmo de substituição FIFO

- Muito simples de ser implementado

EXEMPLOS:

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

- **3 frames** (3 págs podem estar na memória por processo)

1	2	3	4	1	2	5	3	4
-	1	2	3	4	1	2	5	3
-	-	1	2	3	4	1	2	5

9 page faults

- **4 frames**

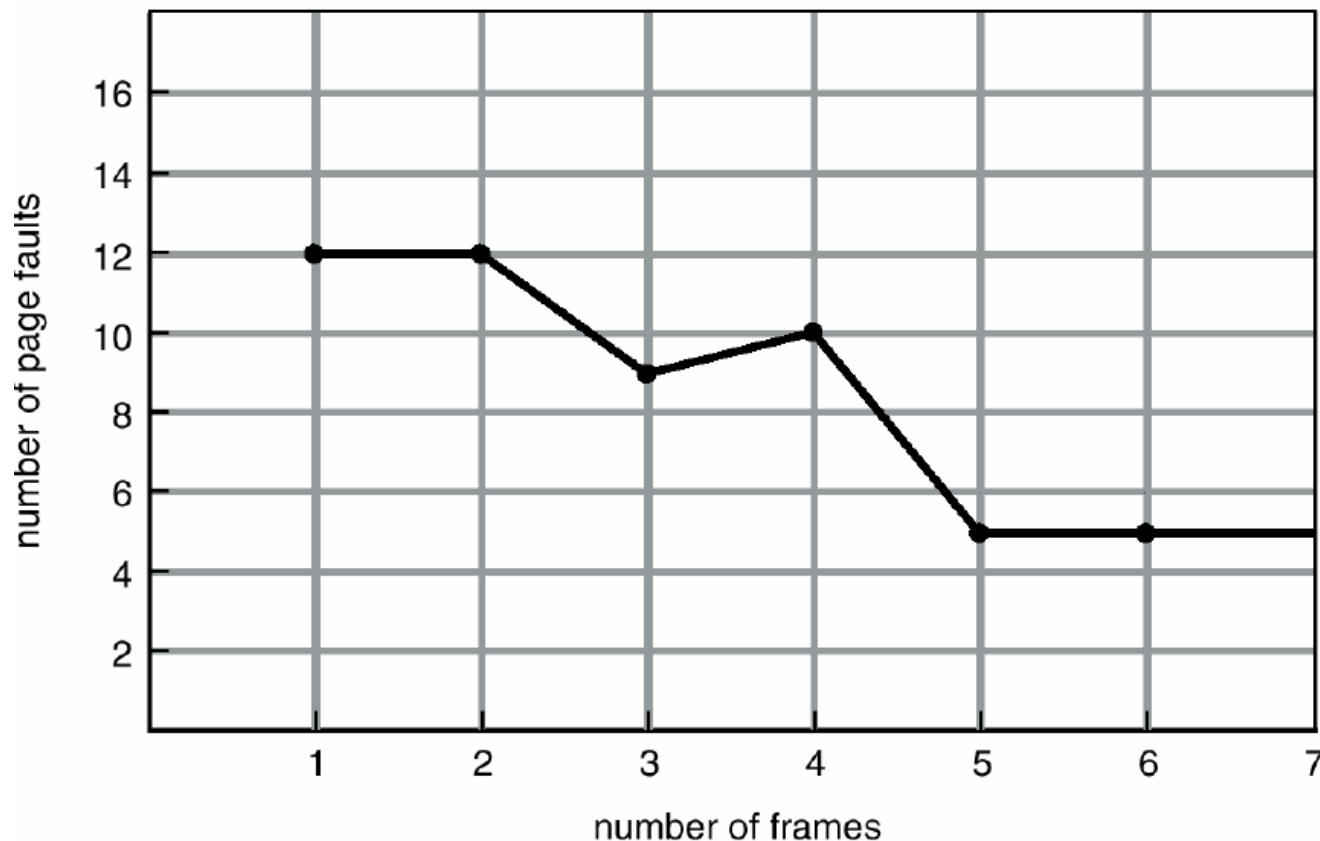
1	2	3	4	5	1	2	3	4	5
-	1	2	3	4	5	1	2	3	4
-	-	1	2	3	4	5	1	2	3
-	-	-	1	2	3	4	5	1	2

10 page faults

- Comportamento inesperado:
 - mais frames livres → mais *page fault* → **Anomalia de Belady**

Algoritmo FIFO com anomalia de Belady

- Gráfico mostra que com 4 frames livres tem-se mais **page faults** (10) do que com 3 frames livres (9)

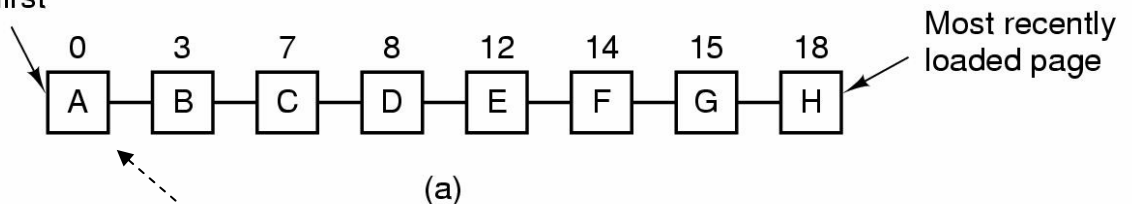


Algoritmo de substituição Segunda Chance

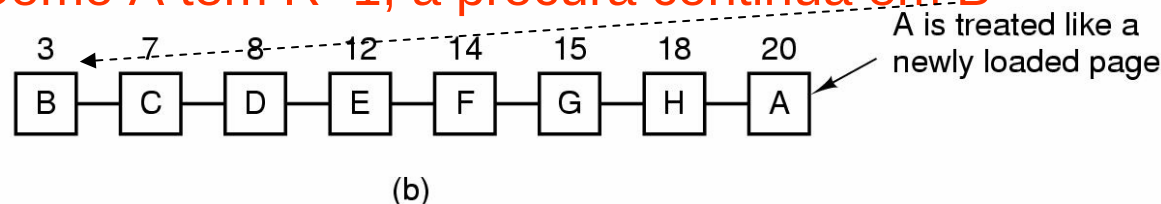
- Pequena melhora ao algoritmo FIFO
- Registros da tabela de páginas incluem *bit de referência R*
- Página referenciada $\rightarrow R=1$
- A fila FIFO é organizada com base no tempo de carga de cada página
- $R=0 \rightarrow$ página velha e não usada $\rightarrow$ subst. imediatamente
- $R=1 \rightarrow R$ é colocado em 0, a página vai para o fim da fila, e o seu tempo de carga é atualizado com se a mesma tivesse acabado de ter sido carregada

Este algoritmo evita o problema de se substituir páginas que estão sendo muito usadas !

Page loaded first

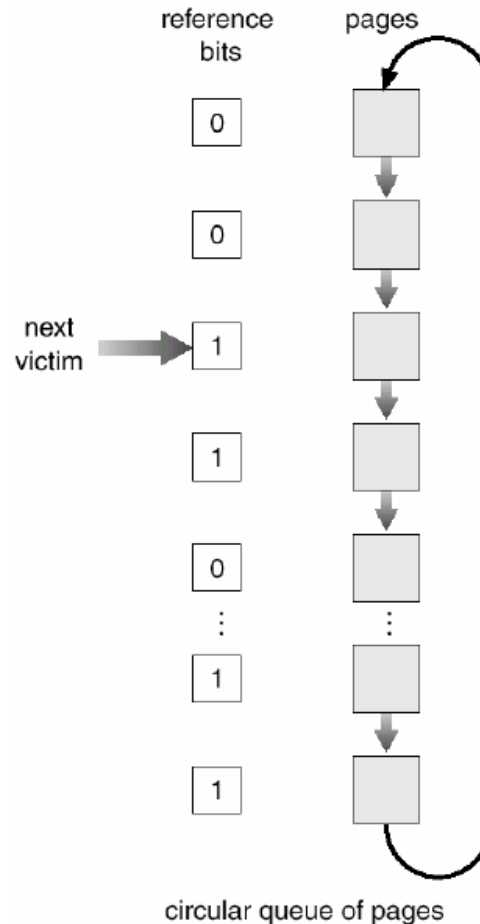


Como A tem $R=1$, a procura continua em B

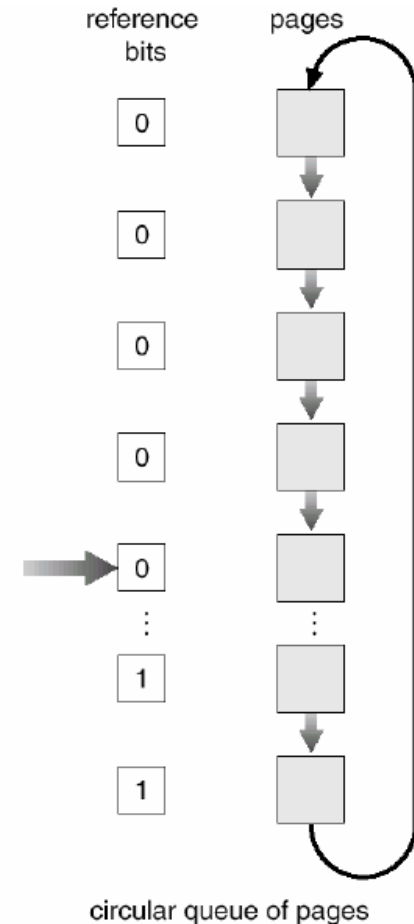


Algoritmo de substituição Clock

- É uma implementação particular do algorit. Segunda Chance
- Diferentemente do algoritmo Segunda Chance, não precisa deslocar páginas dentro da fila
- Páginas a serem substituídas são pesquisadas por ordem circular
- Se $R=1$, então
 - Fazer $R=0$
 - Não subst. página
 - Ir para próx. página



(a)



(b)

Se todas as páginas estiverem com $R=1$, o algoritmo fará uma busca completa até retornar à página inicial

Algoritmo de substituição LRU

- LRU (Least Recently Used): menos usado recentemente
- LRU representa uma aproximação do algoritmo ideal
 - Substitua a página não usada a mais tempo
- Associa-se a cada página, o tempo de seu último uso
- Exemplo de **4 frames**

cadeia de referência: 1 2 3 4 1 2 5 1 2 3 4 5

1	2	3	4	4	3	3	3
-	1	2	3	5	5	4	4
-	-	1	2	2	2	2	2
-	-	-	1	1	1	1	5

8 page faults

- Este algoritmo é bastante usado, mas normalmente demanda substancial suporte do hardware

Comparação entre alguns dos algoritmos

- Neste exemplo, o LRU é o algoritmo que mais se aproxima do algoritmo ideal

Page address
stream

2 3 2 1 5 2 4 5 3 2 5 2

OPT

2	2	2	2	2	2	4	4	4	2	2	2
	3	3	3	3	3	3	3	3	3	3	3
			1	5	5	5	5	5	5	5	5
				F		F			F		

3 page faults

LRU

2	2	2	2	2	2	2	2	3	3	3	3
	3	3	3	5	5	5	5	5	5	5	5
			1	1	1	4	4	4	2	2	2
				F		F		F	F		

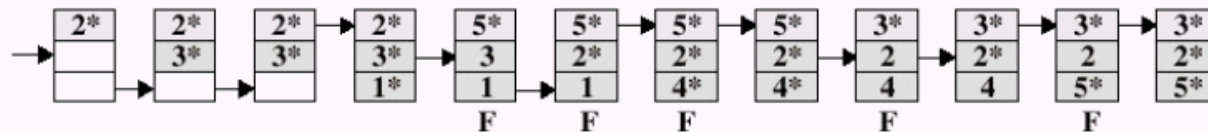
4 page faults

FIFO

2	2	2	2	5	5	5	5	3	3	3	3
	3	3	3	3	2	2	2	2	2	5	5
			1	1	1	4	4	4	4	4	2
				F	F	F		F		F	F

6 page faults

CLOCK



5 page faults

* indica R=1

Exercício

- Considere a seguinte cadeia de referência de páginas
1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6
- Quantas **page faults** ocorrerão com os algoritmos de substituição de páginas IDEAL, FIFO e LRU?
- Considere duas situações distintas, com 3 e 5 frames.

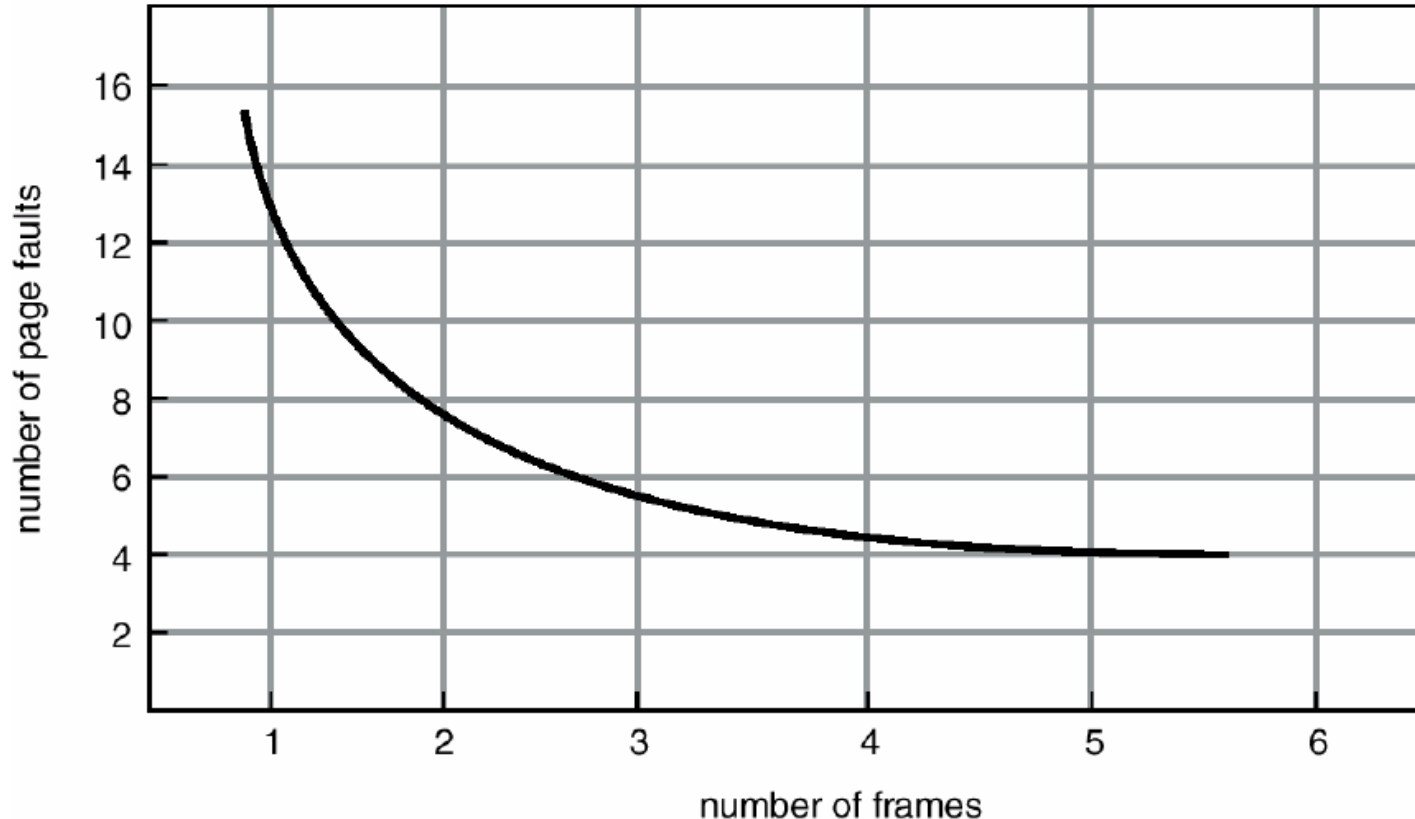
Alocação de Frames

- Cada processo precisa de um número mínimo de frames livres
 - No mínimo alguns poucos frames para uma instrução completa
 - Depende da arquitetura de instrução do processador em uso
- Mínimo de frames → depende da arquitetura
- Máx. de frames → função da memória física disponível
- Maneira mais fácil p/ alocação de frames é dividir os frames disponíveis igualmente entre os processos
 - Problema: processos têm necessidades distintas
- Uma solução alternativa → alocação proporcional
- Uma terceira possibilidade é a alocação por prioridade

Alocação de Frames (cont.)

- Comportamento esperado:

A medida que o número de frames disponíveis aumenta, a taxa de ***page fault*** torna-se constante e pequena



Alocação Local vs. Alocação Global

Na substituição dos frames, duas possibilidades são possíveis

Substituição Local

- Cada processo seleciona frames somente do seu próprio conjunto de frames alocados

Substituição Global

- Processos selecionam frames do conjunto total de frames alocados. Assim, processos podem pegar frames uns dos outros

Thrashing

- Se um processo não tem páginas o suficiente, a taxa de **page faults** é muito alta. Isso ocasiona
 - Baixa utilização de CPU
 - SO adiciona outro processo ao sistema → thrashing

O princípio da localidade

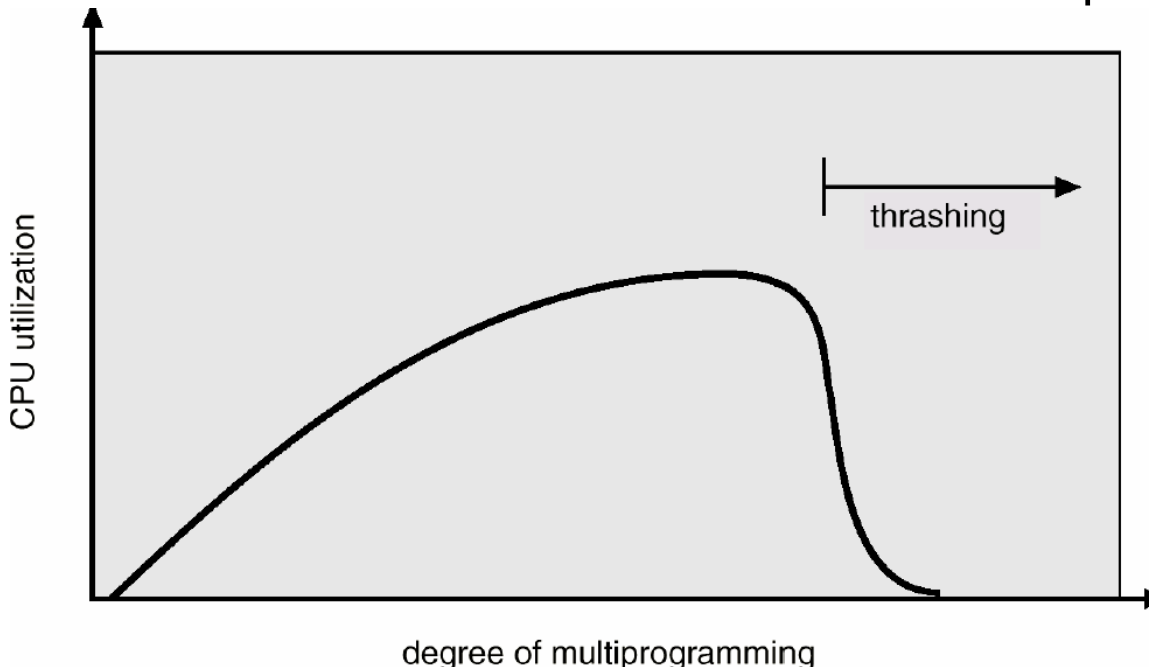
programas e dados de um processo tendem a ficar localizados em intervalos de tempo

frames
alocados < localidade
atual

Thrashing

um processo está mais ocupado com swaps do que com trabalho efetivo

Uma mesma página entra e sai da memória repetidamente

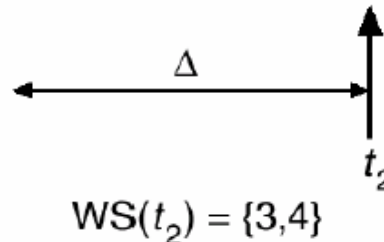
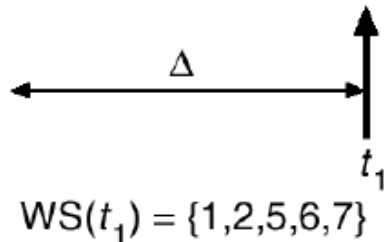


O modelo Working Set

- O working set é uma aproximação da localidade de um processo
- A idéia é manter na memória, um número mínimo de páginas que satisfa a localidade do momento
- Definindo-se um intervalo de tempo delta, o working set WS_i de um processo no instante t_i compõe-se das páginas referenciadas no último intervalo delta
- Páginas do working set não devem ser substituídas

page reference table

... 2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



O working set

- 1- evita thrashing
- 2- otimiza uso da CPU

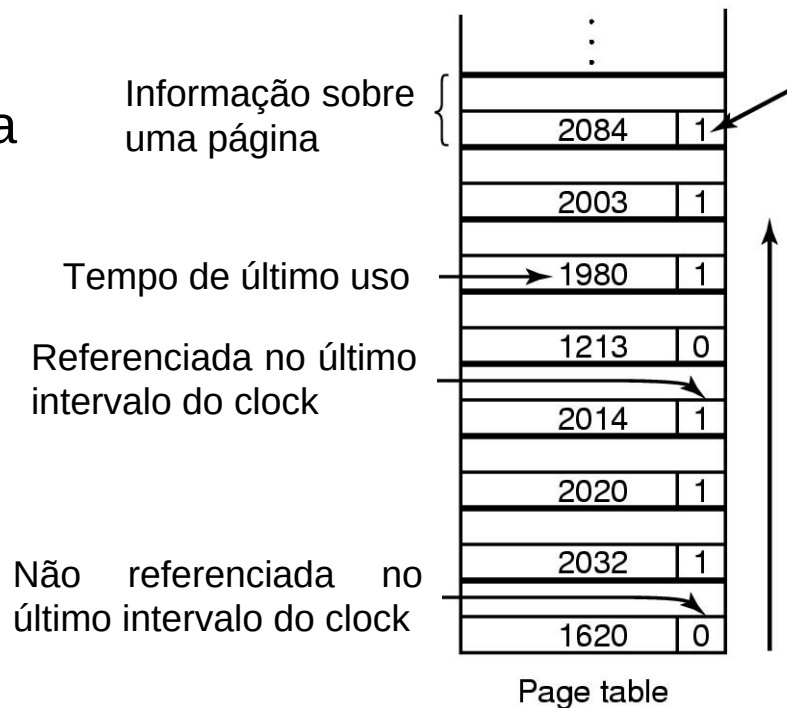
Implementação do modelo Working Set

- Normalmente usa-se um temporizador mais um bit de referência (R) para controle do working set
- Em intervalos regulares, o working set é inspecionado

tempo efetivo de uso da CPU -----> 2204 Tempo virtual corrente

Cada registro da tabela contém:

- Bit de referência
- Tempo do último uso da página



R: bit de referência

A cada pulso do clock → R=0

verifica o bit R de todas as páginas
se(R==1)

Tempo de último uso ← tempo virtual corrente

se (R==0 e idade>delta)

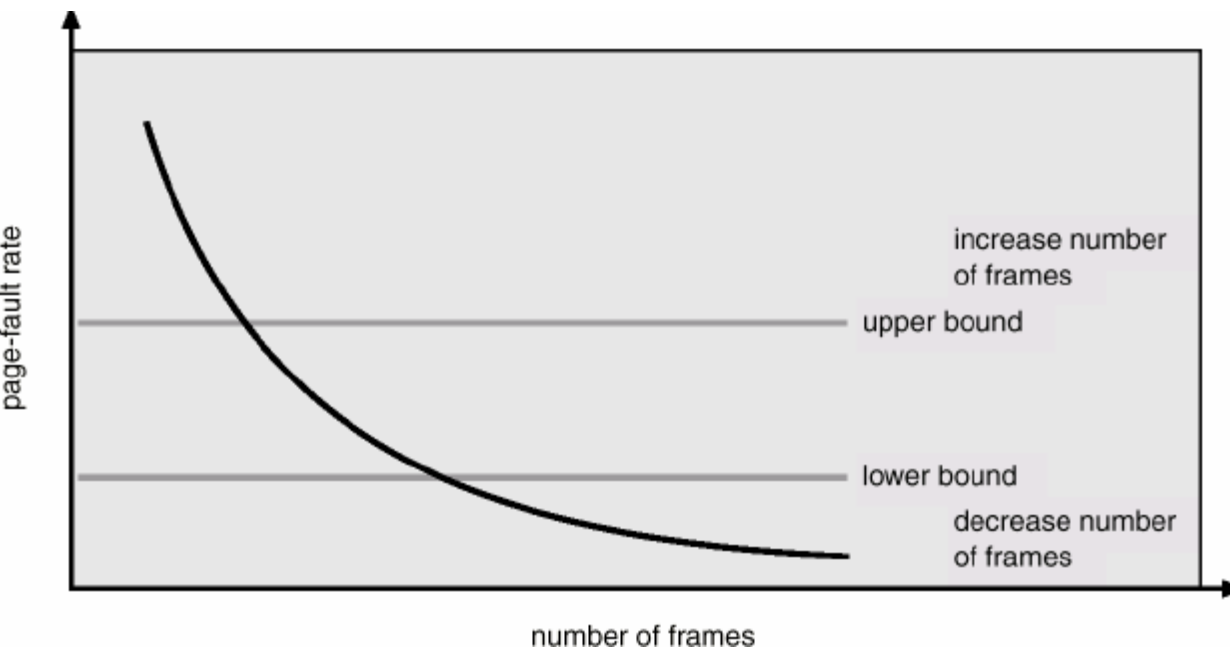
remova a página

se (R==0 e idade≤delta)
anote tempo de último uso da página

Seqüência sob page fault

Esquema de frequência de *page faults*

- Controla o thrashing monitorando a frequência de *page faults*
- Mais fácil de se implementar do que o working set
- Define-se uma faixa aceitável de taxas de *page faults*
- Acima do limite aloca mais frame ao processo, e libera frames se taxa cai abaixo do limite



Na falta de frames
livres a serem
alocados, suspender
um processo e
liberar seus frames

Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004